

**Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»
Кафедра интеллектуальных информационных технол**

**Технологии и инструментальные средства
проектирования интеллектуальных систем
Лабораторный практикум**

**Рекомендовано в области информатики и радиоэлектроники
в качестве учебно-методического пособия для студентов
учреждений, обеспечивающих получение высшего образования
по специальности
«Искусственный интеллект»**

Минск БГУИР 2023

Оглавление

[Лабораторная работа №1. Изучение принципов работы интеллектуальных экспертных обучающих систем на примере системы NIKA. описание выбранной предметной области и фактографических высказываний для базы знаний системы NIKA](#)

[Лабораторная работа №2. Обучение классификатора сообщений Wit.ai системы NIKA](#)

[Лабораторная работа №3. Проектирование логических правил для ответа на сообщения системы NIKA](#)

[Лабораторная работа №4. Разработка модуля адаптации знаний для системы NIKA](#)

Введение

В настоящее время существует необходимость разработки компьютерных систем, которые не только обладают высоким уровнем интеллекта, но и обладают семантической совместимостью, взаимопониманием, способностью координировать свою деятельность с другими системами при коллективном решении сложных задач.

Очевидно, что эти требования предполагают существенное развитие стандартов интеллектуальных компьютерных систем и, соответственно, технологий проектирования семантических совместимых гибридных интеллектуальных компьютерных систем. Важнейшая особенность технологии проектирования таких компьютерных систем заключается в том, что для этого интеллектуальные компьютерные системы должны использовать:

- один и тот же язык внутреннего представления знаний;
- один и тот же язык их коммуникации;
- одну и ту же систему понятий;
- обладать достаточно большим количеством общих (одинаковых) знаний.

К этому можно добавить возможность и целесообразность одинаковой для всех интеллектуальных компьютерных систем реализации целого ряда их способностей:

- способности понимать информацию, которой они обмениваются между собой;
- способности договариваться и координировать свои действия при коллективном решении сложных интеллектуальных задач в условиях нештатных (аномальных, нестандартных, непредусмотренных) ситуаций;
- способности принимать решения на основе их глубокого обоснования;
- способности обучаться;
- многие другие способности, обеспечивающие необходимый уровень интеллекта разрабатываемых интеллектуальных компьютерных систем.

Сформировать у студентов навыки в области искусственного интеллекта можно только путем поэтапного и непосредственного их включения в реальную

деятельность в этой области. Данная проблема частично решается соответствующей образовательной дисциплиной, в процессе обучения которой и изучается технология проектирования интеллектуальных систем.

Целью данной учебной дисциплины является приобретение знаний об особенностях принципов организации и способов реализации интеллектуальных систем. В рамках данного курса студенты должны освоить:

- приобретение знаний из области использования и классификации интеллектуальных систем;
- приобретение навыков о составе и организации знаний в интеллектуальных системах;
- приобретение умений по извлечению знаний из различных источников;
- изучение принципов построения интеллектуальных систем;
- овладение методами представления и структурирования знаний;
- овладение технологией проектирования систем, основанных на знаниях.

Данное методическое пособие содержит теоретический материал, необходимый для изучения интеллектуальных компьютерных систем и технологии их проектирования, эксплуатации и реинжиниринга, называемой технологией OSTIS, а также лабораторный практикум, предусматривающий изучение принципов работы экспертных интеллектуальных систем на примере интеллектуальной системы NIKA, возможность её обучения для решения комплексных прикладных задач и другое.

Лабораторный практикум по курсу ТПИС состоит из 4 лабораторных работ. Каждая из лабораторных работ включает: задание вместе с описанием исходных данных и описанием целей задания; возможного решения вместе с описанием промежуточных состояний, ведущих к достижению описанной цели; ответа, который описывает правильный результат, соответствующий и удовлетворяющий поставленной в задании цели.

Первая лабораторная работа предусматривает ознакомление с принципами работы экспертных интеллектуальных компьютерных систем на примере существующей интеллектуальной системы NIKA и разработку фрагментов базы знаний по выбранной предметной области для этой системы. Вторая и третья лабораторная работа включают классификацию сообщений при ведении диалога с системой NIKA по выбранной теме из первой лабораторной работы и проектирование логических правил для управления диалогом с системой NIKA. Четвёртая лабораторная работа предполагает проектирование программного модуля как части решателя задач системы NIKA, задачей которого является адаптация её знаний, то есть возможность создания и выявления новых знаний на основе существующих в системе. Четвёртая лабораторная работа необязательна для выполнения.

Теоретические сведения

При выполнении лабораторных работ в рамках данного курса можно столкнуться с некоторыми из понятий и терминов, описываемых в рамках стандарта технологии проектирования интеллектуальных систем. **Их знание важно не только при прохождении аттестации по выполненной лабораторной работе, но и при выполнении всего лабораторного практикума.**

Технология - комплекс моделей, методов и средств, обеспечивающих решение соответствующего класса задач, выполнение соответствующего класса действий, реализацию соответствующего вида деятельности. Технология проектирования чего-либо обычно сопровождается разработкой стандарта проектирования этого чего-либо. В рамках данного курса детальное внимание уделяется технологии проектирования интеллектуальных систем, а конкретно Технологии OSTIS (Open Semantic Technology for Intelligent Systems).

Технология OSTIS - это комплекс (семейство) технологий, обеспечивающих проектирование, производство, эксплуатацию и реинжиниринг интеллектуальных компьютерных систем (ostis-систем), предназначенных для автоматизации самых различных видов человеческой деятельности и в основе которых лежит смысловое представление и онтологическая систематизация знаний, а также агентно-ориентированная обработка знаний. Другими словами, **Технология OSTIS** – это технология принципиально нового уровня, целью и задачами которой ставятся:

- разработка интеллектуальных компьютерных систем (ostis-систем) по принципам, лежащих в основе этой технологий, семантически совместимых между собой, способных к самостоятельному взаимодействию, адаптироваться к пользователям и адаптировать (обучать) самих пользователей более эффективному взаимодействию с интеллектуальными компьютерными системами;
- интеграция и конвергенция самых различных видов знаний и самых различных моделей решения задач, неразрывно связанных с процессом разработки интеллектуальных компьютерных систем и процессом повышения квалификации разработчиков этих систем.

Проектирование технологии проектирования интеллектуальных систем осуществляется едино, по всем направлениям Искусственного интеллекта.

Искусственный интеллект - научно-техническая деятельность, направленная на построение теории интеллектуальных систем, а также на

создание технологии проектирования и производства искусственных интеллектуальных систем (интеллектуальных компьютерных систем); область человеческой деятельности, основными целями которой являются:

- построение теории интеллектуальных систем;
- создание технологии разработки интеллектуальных компьютерных систем (искусственных интеллектуальных систем);
- переход на принципиально новый уровень комплексной автоматизации всех - видов человеческой деятельности, который основан на массовом применении интеллектуальных компьютерных систем и который предполагает:
 - не только наличие интеллектуальных компьютерных систем, способных понимать друг друга и согласовывать свою деятельность,
 - но и построение Общей теории человеческой деятельности, осуществляемый в условиях нового уровня её автоматизации, (теории деятельности smart-общества), которая должна быть "понятна" используемым интеллектуальным компьютерным системам и которая потребует существенного переосмысления современной организации человеческой деятельности.

Под любой **интеллектуальной компьютерной системой** (искусственной интеллектуальной системой) понимается искусственная кибернетическая система, обладающая высоким уровнем интеллекта (высоким уровнем знаний и умений), а также высоким уровнем обучаемости. Частным случаем таких систем являются *ostis-системы*, проектируемые на основе Технологии OSTIS.

OSTIS-система - интеллектуальная компьютерная система, которая построена в соответствии с требованиями и стандартами Технологии OSTIS, что обеспечивает существенное развитие целого ряда свойств (способностей) этой компьютерной системы, позволяющих значительно повысить уровень интеллекта этой системы (и, прежде всего, ее уровень обучаемости и уровень социализации); интеллектуальная компьютерная система, построенная по Технологии OSTIS, что обеспечивает:

- семантическую совместимость (взаимопонимание) всех этих систем между собой;
- их способность к самостоятельному взаимодействию, к координации своей деятельности при возникновении заранее непредсказуемых (нештатных) ситуаций.

Если интеллектуальные компьютерные системы не будут обладать указанными выше способностями, то ни о каких smart-предприятиях, smart-учреждениях, smart-городах, ни о каком smart-обществе и речи быть не может, т.к. все обстоятельства их деятельности заранее на этапе проектирования предусмотреть принципиально невозможно. Это означает, что

интеллектуальные компьютерные системы должны научиться самостоятельно "отрабатывать" все заранее непредусмотренные обстоятельства.

В данном курсе объектом изучения являются *интеллектуальные обучающие экспертные системы*. Каждая **интеллектуальная обучающая система** в качестве простейшего средства изучения учебного материала предполагает наличие средств навигации по этому материалу и средств задания по нему различных вопросов. Системы, обладающие только таким ограниченным набором возможностей, названы интеллектуальными справочными системами. Таким образом, можно сказать, что интеллектуальная обучающая система обязательно реализует в себе функции интеллектуальной справочной системы. **Интеллектуальная экспертная система** - интеллектуальная система, включающая в себя знания об определенной слабо структурированной и трудно формализуемой узкой предметной области и способная предлагать и объяснять пользователю разумные решения.

Как упоминалось выше, важными процессами при проектировании и эксплуатации интеллектуальных компьютерных систем, являются конвергенция и интеграция.

Конвергенция - конвергенция пар конкретных искусственных сущностей (например, технических систем) есть стремление их унификацию (в частности, к стандартизации), т.е. стремление к минимизации многообразия форм решения аналогичных практических задач – стремление к тому, чтобы все, что можно сделать одинаково, сделалось одинаково, но без ущерба требуемого качества. Последнее очень важно, так как безграмотная стандартизация может привести к существенному торможению прогресса. Ограничение многообразия форм не должно приводить к ограничению содержания, возможностей. Образно говоря, "словам должно быть тесно, а мыслям – свободно".

Часто под конвергенцией может пониматься конвергенция баз знаний, либо конвергенция моделей решения задач, либо конвергенция гибридных решателей задач.

Интеграция - объединение нескольких разных сущностей, в результате чего возникает некоторая объединённая целостная сущность.

Понятие интеграции имеет тесную связь с понятием конвергенции. Чем выше степень конвергенции (степень сближения) интегрируемых объектов, тем выше качество результата интеграции.

Лабораторная работа №1. Изучение принципов работы интеллектуальных экспертных обучающих систем на примере системы NIKA, описание выбранной предметной области и фактографических высказываний для базы знаний системы NIKA

Цель

- Изучить структуру и принципы работы баз знаний интеллектуальных систем, освоить навыки формализации предметных областей и фактографических высказываний в базах знаний ostis-систем, научиться навигироваться по базе знаний, выявлять закономерности и противоречия в ней.

Задание

1) Выбрать тему предметной области согласно варианту (вариант выбирается по желанию, но должен быть уникальным в потоке и не совпадать с темой для курсового проекта, также необходимо согласовать вариант с преподавателем);

2) Формализовать структуру предметной области (выделить классы объектов исследования и исследуемые отношения);

3) Описать классы объектов исследования и исследуемые отношения (суммарно не менее 30 абсолютных и относительных понятий):

- между формализуемыми понятиями нужно обязательно задавать теоретико-множественные отношения;
- для формализуемых отношений необходимо задавать свойства и домены;
- при описании понятий задавать примечания, пояснения и определения к ним;
- при формализации предметной области необходимо пользоваться текстами из открытых источников.

4) Для каждого понятия описать не менее одного экземпляра:

- полученные экземпляры связать между собой при помощи относительных понятий;
- полученная структура должна соответствовать выбранным текстам по теме предметной области.

5) Протестировать полученные фрагменты базы знаний в системе NIKA, используя графовый редактор, контекстное меню и другие возможности;

6) По результатам работы составить электронный отчёт, где указать:

- цель выполненной работы,
- задание лабораторной работы,

- индивидуальный вариант,
- иллюстрацию и тестирование фрагментов базы знаний,
- выводы.

7) Уметь отвечать на вопросы и решать задачи, указанные в конце лабораторной работы.

8) Запустить свои изменения в форк на github.

Алгоритм опубликования результатов работы:

1) Сделать форк проекта системы NIKA <https://github.com/ostis-apps/nika>

2) Установить форк проекта системы у себя локально:

```
git clone https://github.com/{имя_Вашего_аккаунта}/nika
git checkout tpis-2023
```

3) Перейти в папку kb/extra и погрузить туда исходники фрагментов базы знаний;

4) Вернуться в корень проекта, сделать коммит:

```
git add kb/extra/
git commit -m "feat(kb): <название вашей предметной области>"
```

и сделать пуш изменений:

```
git push
```

5) По решению преподавателя сделать pull-request в <https://github.com/ostis-apps/nika> (пул-реквесты необходимы для получения автомата и выполняются только в случае, если лабораторная работа выполнена качественно, т.е. на 9-10, и одобрена проверяющим).

Критерии аттестации

- 1) Структурированность понятий в предметной области - до 1 балла;
- 2) Наполненность понятиями - до 1 балла;
- 3) Связность экземпляров понятий - до 1,5 баллов;
- 4) Тестируемость - до 0,5 балла;
- 5) Качество отчёта - до 1 балла;
- 6) Умение отвечать на вопросы и решать задачи - до 3 баллов;
- 7) Срок защиты - до 2 баллов.

Максимальный балл - 10.

Срок сдачи - до 8 октября.

Примеры вариантов:

ГР. 321701

ГР. 321702

ГР. 321703

1) Транспорт

2) Активный отдых

3) Путешествия

4) Рестораны

5) Магазины

6) Наука

7) Косметика

8) История

9) Биология

10) Искусственный интеллект

11) Преподаватели кафедры ИИТ

12) Журналистика

13) Болезни

14) Медицина

15) Музыка

16) Атракционы

17) Праздники

18) Сказки

19) Достопримечательности

20) Домохозяйство

21) Производственные товары

22) Торговые центры

23) Драгоценности

24) Кондитерские изделия

25) Мебель

26) Языки программирования

27) Настольные игры

28) Механика

29) Строительство

30) Университет

31) Семейные традиции

32) Еда

33) Одежда

34) Животный мир

35) Почтовые марки

36) Спортивные олимпиады

37) Писатели РБ

- 38) Великие математики
- 39) Программное обеспечение
- 40) Печатные издания
- 41) Динозавры
- 42) Хобби
- 43) Профессии
- 44) Достижения
- 45) Рыбное дело
- 46) Морские обитатели
- 47) Мультфильмы
- 48) Технические науки
- 49) Спортсмены
- 50) Водные ресурсы
- 51) Земельные ресурсы
- 52) Диеты
- 53) Фитнес
- 54) Здоровый образ жизни
- 55) Чай
- 56) Атракционы
- 57) Фильмы ужасов
- 58) Продовольствие
- 59) Химические удобрения
- 60) Танцы
- 61) Валютно-денежные отношения
- 62) Настольные игры
- 63) Искусство
- 64) Сыры
- 65) Танки
- 66) Достопримечательности РБ
- 67) Научные журналы
- 68) Конференции
- 69) Колбасные изделия
- 70) Персоны
- 71) Человекообразные
- 72) Культура
- 73) Этимология
- 74) Квантовая физика
- 75) Космос
- 76) Планеты солнечной системы
- 77) Созвездия
- 78) Спутники

- 79) Космические аппараты
- 80) Астронавты
- 81) Великие географические открытия
- 82) Звёзды Голливуда
- 83) Звёзды Болливуда
- 84) Смысловое пространство
- 85) Компьютерная техника
- 86) Технологии проектирования интеллектуальных систем
- 87) Интеллектуальные системы
- 88) Операционные системы
- 89) Аппаратное обеспечение
- 90) Конные скачки
- 91) Естественные языки
- 92) Турниры
- 93) Напитки
- 94) Тайны человечества
- 95) Домашние животные
- 96) Комнатные растения
- 97) Садовые растения
- 98) Компьютерные игры
- 99) Банковское дело
- 100) Финансовое дело
- 101) Бухгалтерский учёт
- 102) Кибернетические системы
- 103) Системы автоматизации производства
- 104) Проблемы науки
- 105) Проблемы человечества
- 106) Парниковый эффект
- 107) Безопасность жизнедеятельности человека
- 108) Зазеркалье
- 109) Фокусы
- 110) Охранные мероприятия
- 111) Древний Рим
- 112) Древняя Греция
- 113) Древняя Русь
- 114) Дворцы и замки
- 115) Президенты
- 116) Механизмы синхронизации процессов
- 117) Модели памяти
- 118) Нейробионика
- 119) Коллективная деятельность

- 120) Образовательная деятельность
- 121) Технологии нового поколения
- 122) Компьютеры нового поколения
- 123) Семантические технологии
- 124) Модели обработки информации
- 125) Алгоритмы сортировки
- 126) Графовые алгоритмы
- 127) Религия
- 128) Буддизм
- 129) Православие
- 130) Конфуцианство
- 131) Католицизм
- 132) Протестантизм
- 133) Пластиковые изделия
- 134) Вредные привычки
- 135) Полезные привычки
- 136) Деликатесы
- 137) Вымершие животные
- 138) Охота
- 139) Фотоохота
- 140) Фотографии
- 141) Дизайн
- 142) Разработка программного обеспечения
- 143) Жизненный цикл программного обеспечения
- 144) Тестирование программного обеспечения
- 145) Архитектуры и паттерны проектирования программного обеспечения
- 146) Робототехника
- 147) Цветы
- 148) Бисероплетение
- 149) Керамические изделия
- 150) Лепка из глины
- 151) Картины
- 152) Карточные игры
- 153) Футбол
- 154) Хоккей

Теоретические сведения

В состав каждой ostis-системы входит база знаний, решатель задач и интерфейс.

В базе знаний описывается вся информация, соответствующая некоторой части действительного мира, а также, например, знания о самой системе (её документация) и многое другое.

База знаний - совокупность знаний, хранимых в памяти интеллектуальной компьютерной системы и достаточных для того, чтобы указанная система удовлетворяла соответствующим предъявляемым к ней требованиям (в частности, чтобы она имела соответствующий уровень интеллекта).

База знаний - информация, хранимая в памяти кибернетической системы и имеющая высокий уровень качества по всем показателям и, в частности, высокий уровень:

- семантической мощности языка представления информации хранимой в памяти кибернетической системы (в базе знаний указанный язык должен быть универсальным);
- гибридности информации, хранимой в памяти кибернетической системы;
- многообразия видов знаний, хранимых в памяти кибернетической системы;
- формализованности информации, хранимой в памяти кибернетической системы;
- структурированности информации, хранимой в памяти кибернетической системы.

В качестве языка универсального представления знаний в базах знаний *ostis*-систем используется SC-код. Особенность данного языка заключается в том, что он обеспечивает единую форму записи разных видов знаний (декларативных и процедурных). *SC-код является формальным языком универсального представления знаний в памяти интеллектуальных компьютерных систем и представляет собой множество нелинейных sc-текстов.* Универсальность SC-кода обеспечивается и тем, что элементами текстов SC-кода могут быть знаки описываемых сущностей любого вида, в том числе, и знаки связей между описываемыми сущностями и/или их знаками. Тексты SC-кода являются графовыми структурами расширенного вида, в которых знаки описываемых связей могут соединять не только вершины (узлы) графовой структуры, но и знаки других связей. Таким образом на SC-коде можно записать любую информацию, в том числе информацию (программу), которая будет обрабатывать другую информацию. SC-код обладает простым синтаксисом, благодаря чему прост в понимании, представлении и обработке. Так как SC-код является графовым нелинейным формальным языком, то абстрактная память *ostis*-системы является графодинамической (т.е. нелинейной (графовой) и структурно перестраиваемой).

Для представления баз знаний *ostis*-систем используется целый ряд как универсальных языков, так и специализированных языков, как формальных языков, так и естественных языков, как внутренних языков, обеспечивающих представление информации в памяти *ostis*-систем, так и внешних языков,

обеспечивающих представление информации, вводимой в память ostis-систем, либо выводимой из этой памяти. Естественные языки используются исключительно для представления файлов, хранимых в памяти ostis-системы и формально специфицируемых в рамках базы знаний этой ostis-системы.

Для эксплуатации интеллектуальных компьютерных систем, построенных на основе SC-кода, кроме способа абстрактного внутреннего представления баз знаний (SC-кода) потребуются несколько способов внешнего изображения абстрактных sc-текстов, удобных для пользователей и используемых при оформлении исходных текстов баз знаний указанных интеллектуальных компьютерных систем и исходных текстов фрагментов этих баз знаний, а также используемых для отображения пользователям различных фрагментов баз знаний по пользовательским запросам. В качестве таких способов внешнего отображения sc-текстов и предлагаются указанные выше внешние языки ostis-систем (SCg-код, SCs-код и SCn-код).

База знаний каждой ostis-системы представляет собой иерархическую систему разделов, к которым должны "привязываться" исходные тексты каждой новой информации, вводимой в базу знаний, и, прежде всего, исходные тексты достаточно крупных фрагментов баз знаний. *База знаний ostis-системы имеет достаточно развитую иерархическую структуру. База знаний делится на разделы. Разделы бывают атомарными и неатомарными. Неатомарный раздел состоит из сегментов. Атомарные разделы не имеют сегментов. Разделы базы знаний ostis-системы могут иметь самое различное назначение.*

Знание, описываемое в базе знаний - дискретная информационная конструкция, представленная в некотором (дополнительно уточняемом) языке. Sc-знание - знание, представленное на SC-коде. Не каждый sc-текст является sc-знанием. В отличие от sc-текстов в знаниях важна семантическая целостность (полнота) информационных конструкций. Так, например, логическая формула со свободными переменными не является знанием. Но полный текст высказывания, включающий в себя все компоненты всех логических связок (вплоть до атомарных логических формул) является знанием. Второй пример: sc-текст, в состав которого входит sc-коннектор или связка, но не входят все компоненты этого sc-коннектора или связки, знанием не является.

Качество системы в большей части зависит от качества формализованной и структурированной базы знаний. Так, высокий уровень обучаемости ostis-систем обеспечивается:

- высоким уровнем семантической гибкости информации, хранимой в памяти ostis-системы, поскольку каждое удаление или добавление синтаксически элементарного фрагмента хранимой информации, а также удаление или добавление каждой связи инцидентности между такими элементами имеет четкую семантическую интерпретацию;

- высоким уровнем стратифицированности хранимой информации, что обеспечивается онтологически ориентированной структуризацией базы знаний ostis-системы;
- высоким уровнем рефлексии ostis-системы, что обеспечивается мощными метаязыковыми возможностями языка внутреннего представления информации (знаний) в памяти ostis-систем.

Достоинства ostis-систем обеспечиваются:

- достоинствами SC-кода – языка внутреннего кодирования информации, хранимой в памяти ostis-систем;
- достоинствами организации sc-памяти – памяти ostis-систем;
- достоинствами sc-моделей баз знаний ostis-систем – средствами структуризации таких баз знаний;
- достоинствами sc-моделей решения задач – агентно-ориентированных моделей решения задач, используемых в ostis-системах.

В формализации Вы часто будете сталкиваться с таким понятием, как понятие множества. Под **множеством** понимается соединение в некое целое M определённых хорошо различимых предметов m нашего созерцания или нашего мышления (которые будут называться «элементами» множества M). **Множество** – мысленная сущность, которая связывает одну или несколько сущностей в целое. Более формально **множество** – это абстрактный математический объект, состоящий из элементов. Связь множеств с их элементами задается бинарным ориентированным отношением принадлежность*.

Множество может быть полностью задано следующими тремя способами:

- путем перечисления (явного указания) всех его элементов (очевидно, что таким способом можно задать только конечное множество);
- с помощью определяющего высказывания, содержащего описание общего характеристического свойства, которым обладают все те и только те объекты, которые являются элементами (т.е. принадлежат) задаваемого множества.
- с помощью теоретико-множественных операций, позволяющих однозначно задавать новые множества на основе уже заданных (это операции объединения, пересечения, разности множеств и др.).

Более подробно структуру базы знаний, понятия, используемые при формализации, и пример формализации предметной области будут описаны в разделе “Ход работы”.

Полезные источники

- 1) [Стандарт Технологии OSTIS](#)
- 2) [Краткая презентация о Технологии OSTIS](#)
- 3) [Презентация о текущем состоянии и направлениях развития ИИ](#)
- 4) [Документация платформы, на которой спроектирована НИКА](#)
- 5) [Метасистема OSTIS](#)

Ход работы

Кратко о системе.

НИКА - интеллектуальная обучающая экспертная система, интеллектуальный ассистент, управляемый знаниями, способный вести диалог с пользователем на заданную тему или множество тем (*NIKA is an Intelligent Knowledge-driven Assistant*). НИКА является ostis-системой и спроектирована по принципам, лежащим в основе Технологии OSTIS. Она состоит из гибридной базы знаний, гибридного решателя задач и интеллектуального интерфейса.

В базе знаний системы НИКА описываются её документация; предметные области, содержащие основные понятия, необходимые при решении логических задач и задач управления диалогом с пользователем; логические правила, по которым строится диалог с пользователем; агенты управления диалогом и их формальная спецификация; а также все прикладные предметные области, на тему которых может вестись диалог с пользователем.

Решатель задач и интерфейс системы НИКА будут рассмотрены в других лабораторных работах.

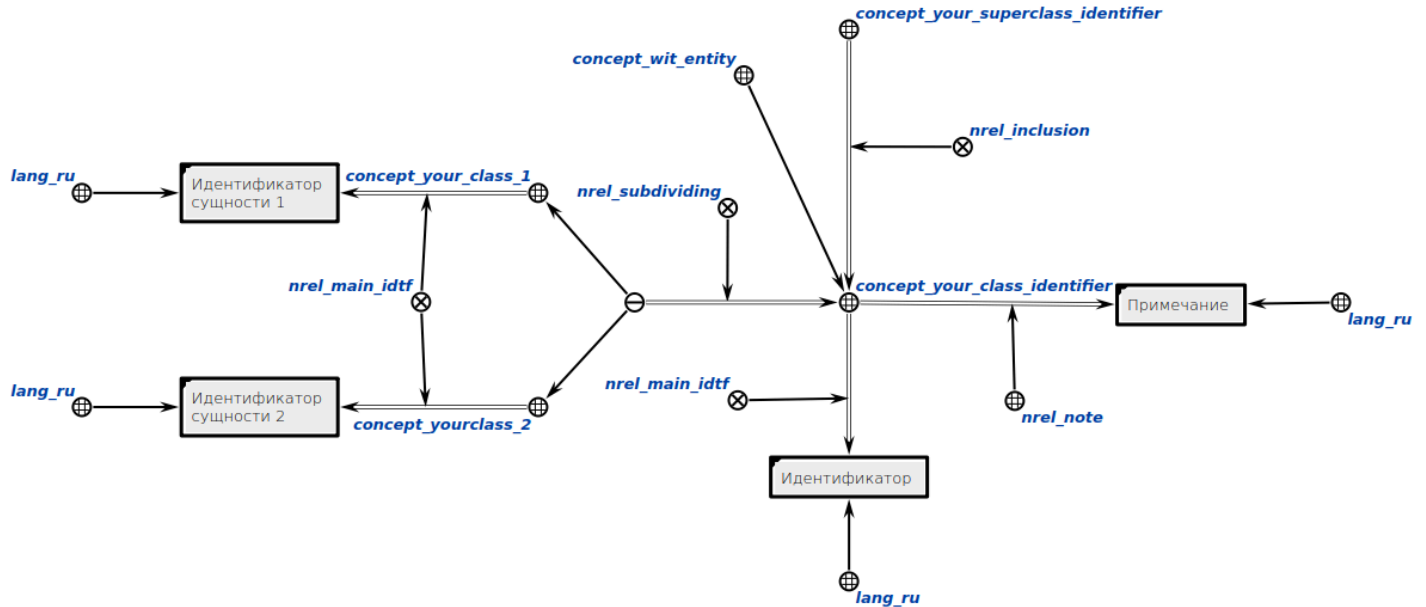
Ника может отвечать на самые различные вопросы, диалог с ней можно вести на *русском языке*. **Документация по системе** доступна на русском и английском языках по [ссылке](#). Документация по платформе доступна по следующей [ссылке](#).

Примеры формализованных фрагментов.

В базе знаний Ники хранятся полезные шаблоны (на SCg и SCs), которые можно использовать либо для ознакомления, либо в качестве основы для построения своих конкретных фрагментов базы знаний. Расположены шаблоны в папке pattern: **“nika/kb/pattern”**. В данной папке есть три модуля – *science*, *gwf*, *scs*.

В папке *gwf* есть шаблоны, написанные на SCg. Их можно использовать как основу для построения своих логических формул (правил), фраз ответа, для формализации сущностей, отношений, и т.д. – то есть всего того, что придется формализовывать в ЛР №1-3.

Пример формализации класса с указанием его декомпозиции и определения (примечания):



В папке *science* есть конкретный пример формализации сущности “science”, правила классификации сообщения с вопросом про науку, правила ответа и фразы ответа на вопрос о науке: “Что такое наука?”.

В папке *scs* представлены шаблоны, которые есть в папке *gwf*, но только на SCs. Например, ниже формализован класс, его разбиение и определение (примечание).

```
concept_your_class_identifier
<- sc_node_class;
<- concept_wit_entity;
=> nrel_main_idtf:
    [Идентификатор] (* <- lang_ru;; *);
=> nrel_note:
    [Примечание] (* <- lang_ru;; *);
<= nrel_subdividing:
    ...
    (*
        <- sc_node_tuple;;
        -> ..instance_1;;
        -> ..instance_2;;
    *);;

..instance_1
=> nrel_main_idtf:
    [Идентификатор] (* <- lang_ru;; *);;

..instance_2
=> nrel_main_idtf:
    [Идентификатор] (* <- lang_ru;; *);;
```

Проверка правильности формализуемых фрагментов базы знаний

Чтобы проверить правильность формализуемых понятий, можно спросить у Ники о сущности, которая должна быть формализована в её базе знаний.

Например, если формализована сущность *треугольник*, то тогда можно задать вопрос: “Что такое треугольник?”. Если формализовано разбиение треугольника (его декомпозиция), то можно задать вопрос: “Каким бывает треугольник?”. Примеры того, как правильно формализовывать сущности, их разбиение и т.д. приведены в пункте выше, где указаны примеры формализации.

Примечание:

Может быть такое, что Ника не отвечает на вопрос “Что такое”, в данном случае это означает, что нужно обучить Нику с помощью wit-ai (а это можно будет сделать уже в ЛР №2).

Подробнее информацию о вопросах, на которые умеет отвечать Ника, можно найти в разделе с лабораторной работой №2 в пункте “Вопросы, на которые умеет отвечать Ника”. Там представлены примеры фрагментов, необходимые для того, чтобы Ника отвечала на соответствующий вопрос.

Установка проекта.

Чтобы приступить к работе с системой НИКА, необходимо развернуть её локально на компьютерном устройстве. Проект системы НИКА располагается в репозитории на GitHub по ссылке <https://github.com/ostis-apps/nika>. Если у вас установлена ОС из семейства Debian, то вы можете установить и развернуть систему, выполнив следующие действия:

1) Установка системы:

```
git clone -c core.longpaths=true -c core.autocrlf=true
--recursive https://github.com/ostis-apps/nika
```

```
cd nika
```

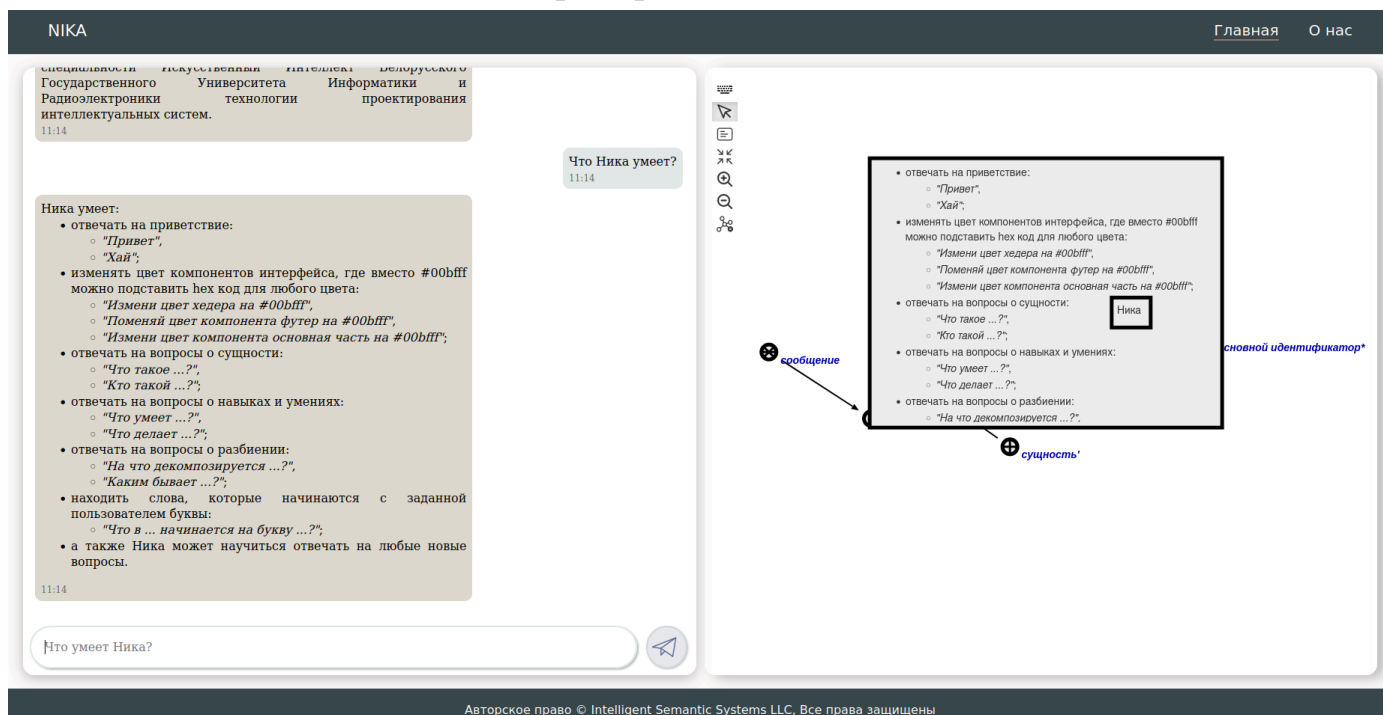
1) Запуск документации:

```
pip3 install mkdocs markdown-include mkdocs-material
mkdocs-static-i18n
mkdocs serve
```

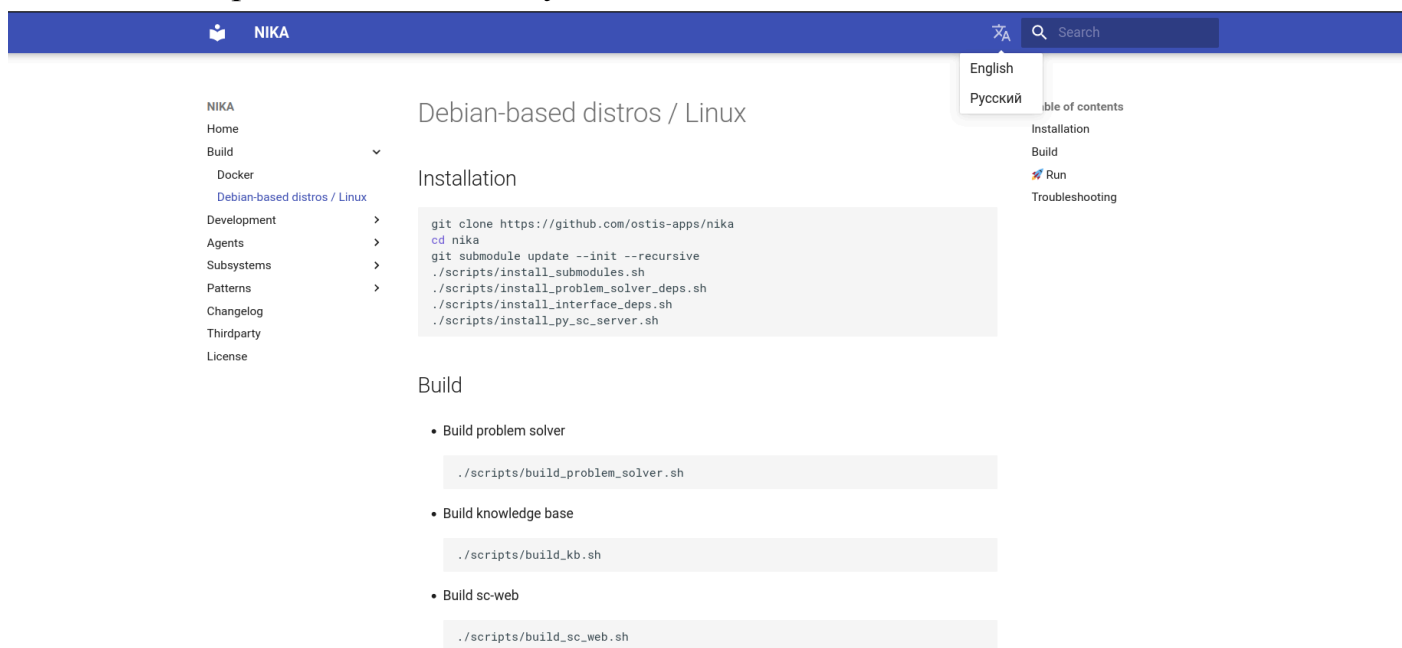
Документацию также можно найти по ссылке ostis-apps.github.io/nika/

*В документации можно увидеть информацию по установке и решению проблем, если они возникли в процессе установки системы. Вам понадобится страница *Build/Debian-based distros / Linux* или *Build/Docker*. Заметьте, что документация доступна и на русском языке)*

Диалоговое окно выглядит примерно так:



Переключить языки в документации можно с помощью кнопки на верхней панели в правой её части. Документация:



Примечание: при возникновении вопросов в процессе установке системе можно обратиться к преподавателю за помощью.

Чтобы изучить, как работает система, можно открыть документацию в <https://github.com/ostis-apps/nika>, а также документацию платформы <https://github.com/ostis-ai/ostis-web-platform>, на которой она спроектирована.

При необходимости можно настроить сервер, который запускался первой командой. Для этого можно перейти в nika и открыть конфигурационный файл nika.ini. Информацию о настройке можно найти здесь [config file docs](#).

Средства разработки БЗ

Чтобы приступить к выполнению заданий, указанных в лабораторной работе, необходимо установить графический редактор баз знаний:

- на ОС из семейства Debian или MacOS, необходимо перейти по ссылке [Linux KBE](#);

- на Windows ОС, необходимо перейти по ссылке [Windows KBE](#).

При всех вариантах архив с редактором начнёт скачиваться автоматически. Чтобы запустить редактор, необходимо разархивировать его и выполнить исполняемый файл:

- Debian или MacOS - `cd KBE & ./kbe;`

- Windows - `cd KBE 4.0 & kbe.exe.`

Примечание: при возникновении вопросов в процессе работы с редактором можно обратиться к преподавателю за помощью.

Также можно воспользоваться любым текстовым редактором, с помощью которого можно сохранять файлы. В текстовом редакторе можно описывать фрагменты базы знаний в строковом формате (на языке SCs).

Пример выполнения заданий

После успешной установки всех необходимых средств можно приступить к выполнению заданий, указанных в лабораторной работе.

1) Допустим, тема лабораторной работы - “Интеллектуальные обучающие системы”. Тогда название формализуемой предметной области и соответствующего ей раздела в базе знаний - “Предметная область интеллектуальных обучающих систем” и “Раздел. Предметная область интеллектуальных обучающих систем”.

2) В папке nika/kb/extra необходимо создать папку для выбранной предметной области. В нашем случае название директории будет выглядеть так: `section_subject_domain_of_intelligent_study_systems`, а иерархия директорий базы знаний, соответственно, так:

3)

kb/

- extra/
 - section_subject_domain_of_intelligent_study_systems/

Далее создаём исходный файл с расширением `.scs` с аналогичным названием `section_subject_domain_of_intelligent_study_systems.scs`, в котором можно начинать описывать структуру предметной области и соответствующего ей раздела:

```
/* Таким образом можно оставлять комментарии в исходных файлах
  базы знаний. Очень важно отметить, что названия SC-элементов
  должны быть написаны на латинице и соответствовать требованиям,
  предъявляемые к синтаксису внешних знаков в SC-коде. Со всеми
  требованиями можно ознакомиться в Стандарте Технологии
  проектирования интеллектуальных систем OSTIS (смотрите ссылки
  выше). */
section_subject_domain_of_intelligent_study_systems
=> nrel_main_idtf:
  [Раздел. Предметная область интеллектуальных обучающих систем]
  (* <- lang_ru;; *);
  [Section. Subject domain of intelligent study systems]
  (* <- lang_en;; *);
<- atomic_section;;

/* Структура предметной области (классы объектов исследования и
  исследуемые отношения) */
section_subject_domain_of_intelligent_study_systems = [*
subject_domain_of_intelligent_study_systems
<- sc_node_struct;
=> nrel_main_idtf:
  [Предметная область интеллектуальных обучающих систем]
  (* <- lang_ru;; *);
  [Subject domain of intelligent study systems]
  (* <- lang_en;; *);
<- subject_domain;
-> rrel_maximum_studied_object_class:
  concept_intelligent_study_system;
  concept_intelligent_study_ostis_system;
-> rrel_not_maximum_studied_object_class:
  concept_lab_work;
-> rrel_explored_relation:
  nrel_knowledge_base;
  nrel_aim;
  nrel_lab_workshop;
  nrel_lab_work_condition;;
*];;

section_subject_domain_of_intelligent_study_systems
```

```
-> rrel_key_sc_element:
    subject_domain_of_intelligent_study_systems;
    concept_intelligent_study_system;
    concept_intelligent_study_ostis_system;
    nrel_knowledge_base;
    nrel_aim;
    nrel_lab_workshop;
    nrel_lab_work_condition;
    concept_lab_work;;
```

Исходный текст 1. Пример предметной области интеллектуальных обучающих систем на SCs

Примечание: приведенная в качестве примера предметная область описана недостаточно - при выполнении заданий лабораторной работы необходимо соблюдать требования, указанные вместе с ними.

Формально, **предметная область** – это результат интеграции (объединения) частичных семантических окрестностей, описывающих все исследуемые сущности заданного класса и имеющих одинаковый (общий) исследования (то есть один и тот же набор отношений, которым должны принадлежать связи, входящие в состав интегрируемых семантических окрестностей. В спецификации любой предметной области указываются максимальный класс объектов исследования (через связку ролевого отношения `rrel_maximum_studied_object_class`), немаксимальные классы объектов исследования (через связки ролевого отношения `rrel_not_maximum_studied_object_class`) и исследуемые отношения (через связки ролевого отношения `rrel_explored_relation`). Важно понимать *различие между максимальным классом объектов исследования и просто классом объектов исследования* - для максимального класса объектов исследования в заданной предметной области отсутствует другой класс объектов исследования, который был бы его надмножеством.

Основываясь на всём сказанном выше, **предметная область** – структура, в состав которой входят:

- основные исследуемые (описываемые) объекты – первичные и вторичные;
- различные классы исследуемых объектов;
- различные связи, компонентами которых являются исследуемые объекты (как первичные, так и вторичные), а также, возможно, другие такие связи – то есть связи (как и объекты исследования) могут иметь различный структурный уровень;
- различные классы указанных выше связей (то есть отношений);

- различные классы объектов, не являющихся ни объектами исследования, ни указанными выше связками, но являющихся компонентами этих связок.

При этом все классы, объявленные исследуемыми понятиями, должны быть полностью представлены в рамках данной предметной области вместе со своими элементами, элементами элементов и т.д. вплоть до терминальных элементов.

Можно говорить о типологии предметных областей по разным структурным признакам:

- наличие метасвязей;
- наличие исследуемых структур, входящих в состав предметной области;
- наличие исследуемых (смежных, дополнительных) объектов, которых исследуются в других предметных областях;

Каждой предметной области можно поставить в соответствие:

- семейство соответствующих ей онтологий разного вида;
- некий язык (в нашем случае – язык, построенный на основе SC-кода), тексты которого представляют различные фрагменты соответствующей предметной области.

Для эффективной коллективной разработки и эксплуатации базы знаний ostis-системы важна не просто её структуризация, а такая структуризация, которая носит максимально объективный характер, имеющая четкую семантическую интерпретацию и позволяющая на основе семантических связей между структурно выделяемыми фрагментами базы знаний легко определять (локализовывать) "местоположение" либо искомым знаний, либо новых знаний, вводимых в состав базы знаний. Такая семантическая структуризация базы знаний, формирование системы семантически связанных между собой "семантических полочек", на которых размещаются конкретные знания, удовлетворяющие четко заданным требованиям, существенно упрощает навигацию по базе знаний и четко локализует эволюцию знаний, находящихся на каждой "семантической полочке". В основе указанной семантической структуризации базы знаний лежит иерархическая система предметных областей.

С содержательной точки зрения *предметная область представляет собой совокупность фактографических высказываний, описывающих все элементы заданного множества объектов исследования с помощью заданного набора отношений и параметров (характеристик).* Максимальный класс указанных объектов исследования, некоторые специально выделяемые подклассы этого класса, а также указанные отношения и параметры задают систему понятий, лежащих в основе предметной области,

которую будем называть схемой предметной области. Подчеркнем то, что схема предметной области, определяющая семантическую "систему координат" в рамках соответствующей предметной области, в известной степени имеет субъективный характер и является предметом соглашения (консенсуса) между специалистами в этой области. Подчеркнем также, что схема предметной области может эволюционировать (может меняться набор понятий, может уточняться их денотационная семантика).

Понятие предметной области можно считать обобщением понятия алгебраической системы, ориентированным на решение проблемы семантической структуризации базы знаний. В памяти ostis-системы предметная область представляется обычно *бесконечной sc-структурой, которая задается (1) неким множеством исследуемых сущностей, (2) семейством отношений, алгебраических операций и параметров (свойств), заданных на множестве исследуемых сущностей, каждое из которых либо рассматривается в рамках данной предметной области, либо "наследуется" из другой предметной области более высокого уровня.* Каждая предметная область в базе знаний может быть представлена своим разделом. При этом на множестве таких разделов могут быть заданы не только рассмотренные выше "синтаксические" отношения, но и целый ряд семантически интерпретируемых отношений, например, отношение "быть частной предметной областью*". Примером связки этого отношения является связка между Предметной областью геометрических фигур в Евклидовом пространстве и Предметной областью планарных фигур Евклидова пространства. Таких отношений, заданных на множестве предметных областей и уточняющих характер соотношения между множествами исследуемых сущностей, а также рассматриваемыми или "наследуемыми" отношениями, алгебраическими операциями и параметрами, существует достаточно много. Но, если к этому добавить анализ соотношения не только между самими предметными областями, но и соотношения между представляющими их sc-структурами в рамках базы знаний конкретной ostis-системы, а также в рамках глобального смыслового пространства (SC-пространства), то число семантически интерпретируемых отношений, заданных на множестве предметных областей существенно расширится. К числу важных отношений, заданных на множестве предметных областей относятся также отношения, связывающие предметные области с различными структурно выделяемыми фрагментами базы знаний, которые предметными областями не являются.

4) После описания структуры выбранной предметной области можно приступать к описанию всех указанных понятий. Перед этим в папке nika/kb/extra/section_subject_domain_of_intelligent_study_systems необходимо создать папки для абсолютных и относительных понятий - concepts и

relations, соответственно, где будут описываться абсолютные и относительные понятия. Иерархия директорий базы знаний, соответственно, обновится:

```
kb/  
- extra/  
  - section_subject_domain_of_intelligent_study_systems/  
    - section_subject_domain_of_intelligent_study_systems.scs  
    - concepts/  
    - relations/
```

В качестве примера абсолютного понятия опишем понятие интеллектуальной обучающей системы с целью дальнейшего описания сущности самой системы Ники. Таким образом в следующих лабораторных работах можно будет строить диалог на тех знаниях, которые имеются о Нике в её базе знаний. Это могут быть такие типовые вопросы, как “Кто такая Ника?”, “Что нужно сделать в лабораторной работе №1?”, “Когда нужно сдать первую лабораторную работу?” и так далее. Содержание и глубина вопросов зависит от качества формализованной Вами предметной области.

```
concept_intelligent_study_system  
<- sc_node_class;  
<- class;  
=> nrel_main_idtf:  
  [интеллектуальная обучающая система]  
  (* <- lang_ru;; *);  
<= nrel_inclusion:  
  concept_intelligent_system  
  (* => nrel_main_idtf:  
    [интеллектуальная система]  
    (* <- lang_ru;; *));  
  *);  
=> nrel_note:  
  [Такого рода системы по сравнению с традиционными системами  
  электронного обучения (например, электронными учебниками)  
  предоставляют обладают рядом существенных преимуществ.]  
  (* <- lang_ru;; *);  
<= nrel_inclusion:  
  concept_intelligent_help_system  
  (*  
    => nrel_main_idtf:  
      [интеллектуальная справочная система]  
      (* <- lang_ru;; *));  
    => nrel_note:  
      [Каждая интеллектуальная обучающая система в качестве  
      простейшего средства изучения учебного материала
```

предполагает наличие средств навигации по этому материалу и средств задания по нему различных вопросов. Системы, обладающие только таким ограниченным набором возможностей, названы интеллектуальными справочными системами. Таким образом, можно сказать, что интеллектуальная обучающая система обязательно реализует в себе функции интеллектуальной справочной системы.]

```
(* <- lang_ru;; *);;
*);
=> nrel_inclusion:
concept_intelligent_study_ostis_system
/* Иногда есть необходимость не описывать некоторые понятия,
участвующие в описании формализуемого понятия. Это может
быть объяснено тем, что связанное понятие принадлежит
другой предметной области и никак не связано с этой. Но для
дополнительной визуализации фрагментов базы знаний
можно задавать идентификаторы на конкретном языке. */
(* => nrel_idtf:
[интеллектуальная обучающая система, построенная
на основе Технологии OSTIS]
(* <- lang_ru;; *);;
*);;
```

Исходный текст 2. Пример абсолютного понятия предметной области интеллектуальных обучающих систем на SCs

Как видно, в формализованном понятии указаны идентификаторы понятий, примечания, подмножества и надмножества класса. Данное описание не является полным, его можно улучшать и дополнять. Также примеры формализации абсолютных понятий Вы можете найти в [Стандарте Технологии OSTIS](#) и на ims.ostis.net.

В качестве исследуемого отношения можно описать понятие бинарного ориентированного отношения “лабораторный практикум*”:

```
nrel_lab_workshop
<- sc_node_norole_relation;
<- relation;
<- binary_relation;
<- oriented_relation;
<- antireflexive_relation;
<- asymmetric_relation;
<- antitransitive_relation;
=> nrel_main_idtf:
[лабораторный практикум по системе*]
```

```

(* <- lang_ru;; *);;
=> nrel_first_domain: concept_intelligent_system;
=> nrel_second_domain: concept_lab_work;
=> nrel_definitional_domain:
...
(*
  <= nrel_combination: {
    concept_intelligent_system;
    concept_lab_work
  };;
*);
<- rrel_key_sc_element:
...
(*
  => nrel_main_idtf:
    [Опр. (лабораторный практикум по системе*)]
    (* <- lang_ru;; *);
  <- definition;;
  <= nrel_sc_text_translation:
    ...
    (*
      -> rrel_example:
        [Лабораторный практикум по системе* - бинарное
        ориентированное отношение, связывающее
        интеллектуальную систему и множество лабораторных
        работ, являющихся лабораторным практикумом по
        этой системе.]
        (* <- lang_ru;; *);;
      *);;
    <= nrel_using_constants: {
      concept_intelligent_system;
      concept_lab_work
    };;
  *);;

```

Исходный текст 4. Пример относительного понятия предметной области интеллектуальных обучающих систем на SCs

Необходимо ознакомиться с используемыми при формализации понятиями. **Понятие** - класс сущностей, который входит в состав по крайней мере одной предметной области в качестве (в роли) ключевого исследуемого понятия. Каждому понятию соответствует по крайней мере одна предметная область, в которой это понятие является исследуемым понятием и в которой рассматриваются основные характеристики этого понятия. Между понятиями можно строить разные теоретико-множественные отношения: включение

одного понятия в другое, принадлежность одного понятия другому, различные теоретико-множественные операции над понятиями и так далее.

Все понятия (классы объектов исследования и исследуемые отношения), описываемые в рамках базы знаний, делятся на *абсолютные (классы) и относительные (отношения) понятия*.

Класс - множество элементов, обладающих какими-либо явно указываемыми общими свойствами. Конкретный элемент, принадлежащий классу, называется **экземпляром класса**. Таким примером класса может быть, например: группа людей, занимающихся некоторой деятельностью, либо множество фруктов, обладающих схожими признаками, а экземпляром класса: конкретный человек, либо фрукт, соответственно, принадлежащие этим классам.

Отношение, заданное на множестве M – это подмножество декартового произведения этого множества самого на себя n раз. В более широком смысле отношение – это математическая структура, которая формально определяет свойства различных объектов и их взаимосвязи. Отношения делятся на бинарные и небинарные, ориентированные и неориентированные, ролевые и неролевые.

Бинарное отношение – это множество таких связок на множестве M , являющихся подмножеством декартова произведения множества M самого на себя. Если бинарное отношение R задано на множестве M и два элемента этого множества a и b связаны данным отношением, то будем обозначать такую связь как aRb . Бинарные отношения могут быть рефлексивными, антирефлексивными или частично рефлексивными; симметричными, антисимметричными или частично симметричными; транзитивными, антитранзитивными или частично транзитивными.

Небинарное отношение – это множество связок на множестве M , хотя бы одна из связок каждого из которых имеет значение мощности больше двух. Унарное отношение не является небинарным отношением.

Ориентированное отношение – это множество таких отношений, каждая связка которых является кортежем.

Неориентированное отношение – это множество таких отношений, каждая связка которых является неориентированным множеством.

Ролевое отношение – это отношение, являющееся подмножеством отношения принадлежности. В конце каждого идентификатора, соответствующего экземплярам класса ролевое отношение, не являющегося системным, ставится знак “ $\langle \rangle$ ”.

Например: *ключевой экземпляр*’.

Из-за ограничений в разрешенном алфавите символов, в системном идентификаторе не может быть использовать знак “ $\langle \rangle$ ”, поэтому в начале каждого системного идентификатора, соответствующего экземплярам класса ролевое отношение ставится префикс “ rel_ ”.

Например: *rrel_key_sc_element*.

Неролевое отношение – отношение, не являющееся подмножеством отношения принадлежности.

В конце каждого идентификатора, соответствующего экземплярам класса неролевое отношение, не являющегося системным, ставится знак “*”.

Из-за ограничений в разрешенном алфавите символов, в системном идентификаторе не может быть использоваться знак “*”, поэтому в начале каждого системного идентификатора, соответствующего экземплярам класса неролевое отношение ставится префикс “nrel_”.

Например: *nrel_inclusion*.

С остальными определениями понятий и примерами формализации понятий можно подробно ознакомиться в [Стандарте Технологии OSTIS](#) и на ims.ostis.net.

После описания всех абсолютных и относительных понятий Ваша директория базы знаний будет выглядеть таким образом:

```
kb/
-   extra/
    - section_subject_domain_of_intelligent_study_systems/
      - section_subject_domain_of_intelligent_study_systems.scs
      - concepts/
        - concept_intelligent_study_system.scs
      - relations/
        - nrel_lab_workshop.scs
```

При выполнении заданий лабораторной работы для каждого описываемого понятия необходимо создавать свой файл с расширением *.scs*.

5) На заключительном этапе формализации предметной области следует описать все необходимые экземпляры используемых Вами понятий, а также указать связи между ними, принадлежащие как отношениями формализуемой Вами предметной области, так и другим смежным предметным областям.

Проиллюстрируем эти требования на примере сущности Ника как экземпляра абсолютного понятия “интеллектуальная обучающая система”. Для этого рядом в папке *concepts/* следует создать папку *instances/*:

```
kb/
-   extra/
    - section_subject_domain_of_intelligent_study_systems/
      - section_subject_domain_of_intelligent_study_systems.scs
      - concepts/
        - concept_intelligent_study_system.scs
        - instances/
```

```
- relations/  
- nrel_lab_workshop.scs
```

В ней можно описывать все необходимые Вам экземпляры абсолютных понятий. Так, в файле `nika.scs` опишем следующие краткие сведения о системе:

```
nika  
<- concept_intelligent_study_system;  
<- concept_expert_system;  
=> nrel_main_idtf:  
  [Ника]  
  (* <- lang_ru;; *);  
=> nrel_aim:  
  [расширить круг решаемых задач, устранить недостатки ранее  
  использованного своего аналога, упростить и автоматизировать  
  процесс обучения студентов, изучающих интеллектуальные  
  экспертные системы.]  
  (* <- lang_ru;; *);  
  [обучить студентов специальности Искусственный Интеллект  
  Белорусского Государственного Университета Информатики и  
  Радиоэлектроники технологии проектирования интеллектуальных  
  систем.]  
  (* <- lang_ru;; *);  
=> nrel_lab_workshop:  
  nika_lab_workshop  
  (* => nrel_main_idtf:  
    [Лабораторный практикум по системе Ника]  
    (* <- lang_ru;; *));;  
    <= nrel_inclusion: concept_lab_work;;  
    -> rrel_1: nika_lab_work_1;;  
  *);;  
  
nika_lab_work_1  
<- concept_lab_work;  
<- concept_wit_entity;  
=> nrel_main_idtf:  
  [Лабораторная работа №1]  
  (* <- lang_ru;; *);  
=> nrel_idtf:  
  [Лабораторная работа №1. Изучение принципов работы  
  интеллектуальных экспертных обучающих систем на примере  
  системы NIKA, описание выбранной предметной области и  
  фактографических высказываний для базы знаний системы NIKA]  
  (* <- lang_ru;; *);  
=> nrel_lab_work_number:  
  [№1]  
  (* <- lang_ru;; *);
```

=> nrel_lab_work_condition:

[1) Выбрать тему предметной области согласно варианту (вариант выбирается случайным образом и уникален в рамках потока);

2) Формализовать структуру предметной области (выделить классы объектов исследования и исследуемые отношения);

3) Описать классы объектов исследования и исследуемые отношения (суммарно не менее 30 абсолютных и относительных понятий):

- между формализуемыми понятиями нужно обязательно задавать теоретико-множественные отношения;
- для формализуемых отношениях необходимо задавать свойства и домены;
- при описании понятий задавать примечания, пояснения и определения к ним;
- при формализации предметной области необходимо пользоваться текстами из открытых источников.

4) Для каждого понятия описать не менее одного экземпляра:

- полученные экземпляры связать между собой при помощи относительных понятий;
- полученная структура должна соответствовать выбранным текстам по теме предметной области.

5) Протестировать полученные фрагменты базы знаний в системе NIKA, используя графовый редактор, контекстное меню и другие возможности;

6) По результатам работы составить отчёт, где указать:

- цель выполненной работы,
- задание лабораторной работы,
- индивидуальный вариант,
- иллюстрацию и тестирование фрагментов базы знаний
- и выводы.

7) Уметь отвечать на вопросы и решать задачи, указанные в конце лабораторной работы.

8) После одобрения преподавателем результатов работы залить изменения в свой форк проекта NIKA на github.]

```
(* <- lang_ru;; *)
```

=> nrel_deadline:

[до 1 октября текущего года]

```
(* <- lang_ru;; *)
```

Исходный текст 4. Пример экземпляра понятия предметной области интеллектуальных обучающих систем на SCs

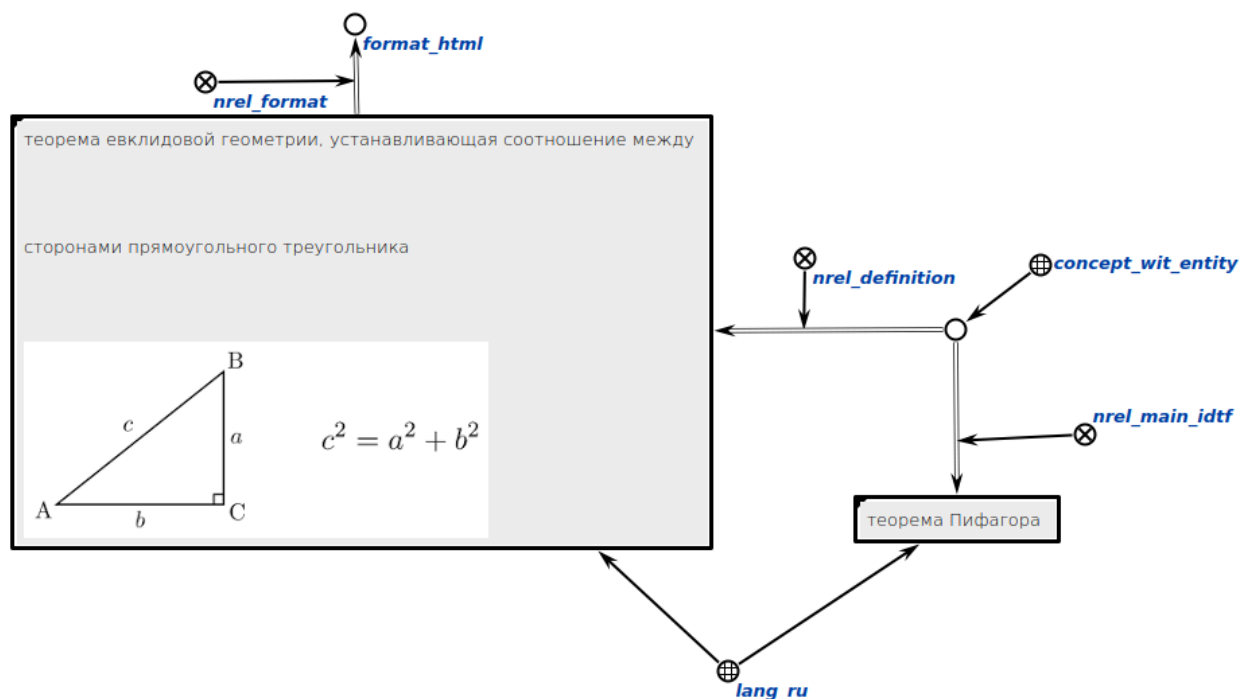
В данном фрагменте текста можно увидеть описание сущности системы Ника, её цели, лабораторный практикум по ней и задачи его первой лабораторной работы.

После выполнения первых 4-х заданий лабораторной работы, должна сформироваться какая-то логически целостная часть предметной области на выбранную вами тему.

Дополнительные примеры формализации

В качестве формата файлов может быть выбран *html*. Это означает, что в ходе формализации понятий может быть использован не только текст, но и картинки, видео – все, что угодно (точнее, что может быть представлено в *html*).

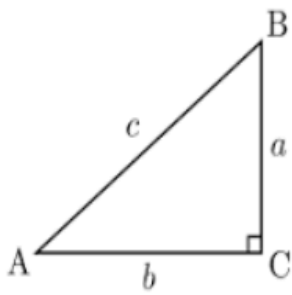
Пример приведен ниже.



Для того, чтобы внешний файл (sc-ссылка), был распознан как файл с форматом *html*, необходимо провести бинарную ориентированную дугу от файла к узлу *format_html* и обозначить данное отношение как *nrel_format*. В данном случае картинка была вставлена с помощью `<img src=СсылкаИзИнтернета>`.

Ссылку из интернета нужно найти самостоятельно. Используя данный фрагмент базы знаний, Ника сможет ответить следующим образом:

теорема Пифагора - теорема евклидовой геометрии, устанавливающая соотношение между сторонами прямоугольного треугольника



$$c^2 = a^2 + b^2$$

19:53

Что умеет Ника?



Еще один пример на SCs указан ниже:

```
***
nika
<- concept_wit_entity;
=> nrel_main_idtf:
  [Ника]
    (* <- lang_ru;; *);
  [Nika]
    (* <- lang_en;; *);
=> nrel_skill:
  [
    <ul>
      <li>отвечать на приветствие:</li>
      <ul>
        <li><i>"Привет"</i>,</li>
        <li><i>"Хай"</i>;</li>
      </ul>
      <li>изменять цвет компонентов интерфейса, где вместо
#00bfff можно подставить hex код для любого цвета:</li>
      <ul>
        <li><i>"Измени цвет хедера на #00bfff"</i>,</li>
        <li><i>"Поменяй цвет компонента футер на
#00bfff"</i>,</li>
        <li><i>"Измени цвет компонента основная часть на
#00bfff"</i>;</li>
      </ul>
      <li>отвечать на вопросы о сущности: </li>
```

```

      <ul>
      <li><i>"Что такое ...?"</i>,</li>
      <li><i>"Кто такой ...?"</i>;</li>
      </ul>
    <li>отвечать на вопросы о навыках и умениях:</li>
      <ul>
      <li><i>"Что умеет ...?"</i>,</li>
      <li><i>"Что делает ...?"</i>;</li>
      </ul>
    <li>отвечать на вопросы про условия и срок сдачи
лабораторных работ, где спросить можно не только про первую
лабу, но и про остальные три:</li>
      <ul>
      <li><i>"Когда защита первой лабы?"</i>,</li>
      <li><i>"Когда нужно сдавать первую лр?"</i>;</li>
      <li><i>"Какое условие 1 лр?"</i>,</li>
      <li><i>"Что нужно делать в лабораторной работе
№1?"</i>;</li>
      </ul>
    <li>отвечать на вопросы о разбиении: </li>
      <ul>
      <li><i>"На что декомпозируется ...?"</i>,</li>
      <li><i>"Каким бывает ...?"</i>;</li>
      </ul>
    <li>находить слова, которые начинаются с заданной
пользователем буквы:</li>
      <ul>
      <li><i>"Что в ... начинается на букву ...?"</i>;</li>
      </ul>
    <li> а также Ника может научиться отвечать на любые новые
вопросы.</li>
  </ul>]
  (* <- lang_ru;;
    => nrel_format: format_html;;
  *);;
***

```

Используя данный фрагмент, Ника сможет ответить следующим образом:

Ника умеет:

- отвечать на приветствие:
 - "Привет",
 - "Хай",
- изменять цвет компонентов интерфейса, где вместо #00bfff можно подставить hex код для любого цвета:
 - "Измени цвет хедера на #00bfff",
 - "Поменяй цвет компонента футер на #00bfff",
 - "Измени цвет компонента основная часть на #00bfff",
- отвечать на вопросы о сущности:
 - "Что такое ...?",
 - "Кто такой ...?",
- отвечать на вопросы о навыках и умениях:
 - "Что умеет ...?",
 - "Что делает ...?",
- отвечать на вопросы про условия и срок сдачи лабораторных работ, где спросить можно не только про первую лабу, но и про остальные три:
 - "Когда защита первой лабы?",
 - "Когда нужно сдавать первую лр?",
 - "Какое условие 1 лр?",
 - "Что нужно делать в лабораторной работе №1?",
- отвечать на вопросы о разбиении:
 - "На что декомпозируется ...?",
 - "Каким бывает ...?",
- находить слова, которые начинаются с заданной пользователем буквы:
 - "Что в ... начинается на букву ...?",
- а также Ника может научиться отвечать на любые новые вопросы.

Рекомендации

1. По результату лабораторной работы должна получиться предметная область, на тему которой в следующих лабораторных работах необходимо будет строить диалог с системой. При формализации описаний понятий и экземпляров этих понятий старайтесь думать о перспективности Вашего решения. Соблюдайте вариативность определений понятий, задавайте больше связей между сущностями, старайтесь описывать несколько сущностей одного класса - это поможет Вам сделать диалог с системой более интеллектуальным и разнообразным.
2. Думайте не только о Вашем решении. Кроме Вашей предметной области другими коллегами могут быть формализованы другие предметные области, возможно даже схожие с Вашей. Проблема интеграции решений различного класса заключается вовсе не в сложности реализаций этих решений, а прежде всего в договороспособности экспертов. Всегда оставайтесь в курсе новостей о существующих решениях. Ваша задача интегрировать и переиспользовать имеющийся опыт, а не повторять его.
3. Творчески относитесь к задачам любой лабораторной работы. Лабораторная работа - это творческая задача, для которой решение придумываете Вы сами.

Лабораторная работа стимулирует развитие Ваших творческих навыков. В это же время помните, что лабораторная работа хоть и создаёт творческую атмосферу, но в то же время определяет её границы.

К защите

1. Протестировать фрагменты предметной области базы знаний в системе;
2. Сделать отчёт (электронная версия) с иллюстрацией тестирования фрагментов базы знаний в системе;
3. Залить изменения в форк на GitHub со всеми выполненными заданиями;
4. Подготовиться к вопросам и задачам. Повторить теоретический материал.

Ход защиты

1. Проверка отчёта
2. Иллюстрация и тестирование формализованных фрагментов базы знаний;
3. Ответы на вопросы и решение задач от преподавателя;
4. Выставление оценки.

Вопросы

1. Что такое Технология проектирования интеллектуальных систем? Какие цели она ставит и какие задачи решает?
2. Какие виды интеллектуальных систем Вы знаете?
3. Что является объектом изучения Искусственного интеллекта? Какие направления в нём Вам известны?
4. Что такое интеллектуальная компьютерная система? Какими свойствами они должны обладать?
5. В чём отличие интеллектуальной компьютерной системы от искусственной интеллектуальной системы?
6. Из каких компонентов состоит любая *ostis*-система? Возможно ли существование *ostis*-системы без какого-то компонента? Аргументируйте.
7. Что такое база знаний *ostis*-системы? Какие знания можно представлять в ней?
8. Что такое SC-код? Какие задачи решаются с помощью него? Приведите пример фрагмент текста на этом языке.
9. Что такое знание? Приведите пример знания.
10. В чём отличие знание от информации? Приведите пример знания и информации и обоснуйте различие.
11. Какими достоинствами наделяет база знаний *ostis*-систему?

12. Как организуется и структурируется база знаний? Приведите пример разделов и предметных областей базы знаний, формирующих её структуру?
13. Что такое система NIKA? Какими функциями она обладает? Какие цели она преследует?
14. Что такое абсолютное и относительное понятие?
Объясните разницу между ними и сформулируйте критерии выбора класса понятий при их формализации.
15. Из каких основных частей состоит предметная область, описываемая в рамках базы знаний?
16. В чём различие максимального класса объектов исследования и класса объектов исследования? Приведите пример.
17. Приведите пример теоретико-множественных отношений между понятиями различного вида? Зачем необходимо их указывать?
18. Какие существуют правила идентификации элементов SC-кода? Как Вы считаете, какие требования часто забываются при идентификации?
19. Приведите алгоритм формализации предметной области базы знаний. На что необходимо обратить внимание в нём. Какие требования должны выполняться при формализации фрагментов базы знаний?

Задачи

1. Формализовать фактографическое высказывание:

- а. Угол АОВ равен 140° . Луч ОС делит угол АОВ на два угла. Найдите угол, образованный биссектрисами углов АОС и СОВ.
- б. Разность двух углов, образованных при пересечении двух прямых, равна 24° . Найдите меньший из этих углов.
- в. Продолжения хорд AB и CD окружности с диаметром AD пересекаются под углом 25° . Найдите острый угол между хордами AC и BD .
- г. Караси — неприхотливые промысловые рыбы, и ценный объект прудового хозяйства благодаря способности выживать при низком (до 2-3 мг/л) содержании кислорода в воде. Тело высокое с толстой спиной, умеренно сжатое с боков. Чешуя крупная и гладкая на ощупь. Окраска варьирует в зависимости от места обитания.
- д. Планетарная туманность в созвездии Лиры имеет угловой диаметр $83''$ и находится на расстоянии 660 пк. Каковы линейные размеры туманности в астрономических единицах?
- е. Человек — общественное существо, обладающее разумом и сознанием, субъект общественно-исторической деятельности и культуры, относящийся к виду Человек разумный. По научным данным, возник на Земле в результате эволюционного процесса — антропогенеза, детали которого продолжают изучаться.

ж. С помощью sc-элементов можно описать любые связи между sc-элементами и/или между сущностями, которые обозначаются этими sc-элементами. При этом указанные связи трактуются как множества связываемых sc-элементов и обозначаются sc-ребрами, sc-дугами, а в случае небинарных связей – sc-узлами.

з. Чему равен КПД источника тока с ЭДС 12 В и внутренним сопротивлением 0,5 Ом, если в цепи течет ток силой 4,8 А?

2. Формализовать часть предметной области на тему:

- а. Обитатели животного мира
- б. Сотрудники университета
- в. Сотрудники IT-компаний
- г. Великие географические открытия
- д. Модели решения задач
- е. Компьютерные системы
- ж. Правотворчество
- з. Философские направления
- и. Полезные ископаемые
- к. Геометрия
- л. Логика
- м. История
- н. Армия

3. Устранить противоречия во фрагменте текста.

Лабораторная работа №2. Обучение классификатора сообщений Wit.ai системы NIKA

Цель

Изучить принципы работы классификатора сообщений, приобрести навыки определения классов сообщений, выделения сущностей и признаков в них, формально специфицировать сценарий диалога.

Задание

- 1) Описать сценарий диалога по предметной области из первой лабораторной работы. Сценарий диалога должен состоять как минимум из 8 сообщений (4 вопроса по теме и 4 соответствующих ответа);
- 2) Обучить классификатор сообщений Wit.ai на вопрос “Что такое ...?” для каждой формализованной сущности (см. пример) и протестировать с помощью вопроса в диалоговом окне Ники (в результате Ника должна отвечать на данный вопрос для всех понятий);
- 3) Выделить классы и сущности сообщений, используемых в описанном сценарии диалога;
 - каждое сообщение должно иметь хотя бы один класс;
- 4) По результатам работы составить отчёт, где указать цель, задание, вариант и показать тестирование сценария диалога;
- 5) Уметь отвечать на вопросы и решать задачи, указанные в конце лабораторной работы.
- 6) Запустить изменения в свой форк на github.

Алгоритм опубликования результатов работы:

- 1) Сделать форк проекта системы NIKA <https://github.com/ostis-apps/nika>
- 2) Установить форк проекта системы у себя локально:

```
git clone https://github.com/{имя_вашего_аккаунта}/nika
git checkout tpis-2023
```
- 3) Перейти в папку kb/extra и погрузить туда исходники фрагментов базы знаний;
- 4) Также в папку kb/extra/{папка с исходниками предметной области}/classification/ загрузить исходники с результатами классификации сообщений, предварительно сделав экспорт из Wit.ai в формате JSON (описано в конце хода работ лабораторной работы);
- 5) Вернуться в корень проекта, сделать коммит:

```
git add kb/extra/
```

```
git commit -m "feat(kb): <название вашей предметной области>"
```

и сделать пуш изменений:

```
git push
```

6) Залить изменения в свой форк.

Критерии аттестации:

- 1) Качество описанного сценария диалога - до 2 баллов;
- 2) Осмысленность классов сообщений - до 2 баллов;
- 3) Качество отчёта - до 1 балла;
- 4) Умение отвечать на вопросы и решать задачи - до 3 баллов;
- 5) Срок выполнения - до 2 баллов.

Максимальный балл - 10.

Срок сдачи - до 22 октября.

Теоретические сведения

Диалоговым экспертным системам недостаточно иметь только формализацию изучаемой предметной области. Системе также необходимо иметь навыки общения, механизмы понимания пользователя. Одним из таких механизмов выступает классификация сообщений пользователя. Исходя из результата классификации система определяет, как следует ответить на то или иное сообщение. **Задача классификации** - задача разделения множества наблюдений (объектов) на группы, называемые классами, на основе анализа их формального описания.

Классификация сообщений помогает компьютерной системе понять намерения (*intents*) пользователя, который выражает свои мысли на естественном языке, а также извлекать другую информацию из сообщения.

Сообщение можно классифицировать по теме, времени, типу и т.д. **Классификация сообщений по теме** подразумевает определение основной мысли (намерения), которую хотел передать пользователь, а также объекты (сущности), указанные в сообщении. **Классификация сообщений по времени** позволяет выделить то, в каком времени происходят события в сообщении: сообщение о настоящем, прошедшем или будущем. **Классификацией сообщений по типу** выделяется цель высказывания сообщения: повествовательное, вопросительное или восклицательное сообщение. Классификация по времени и типу основывается на грамматических признаках

предложения, в то время как классификация по теме предполагает обучение некоторого классификатора с использованием сообщений и указанных для них классов.

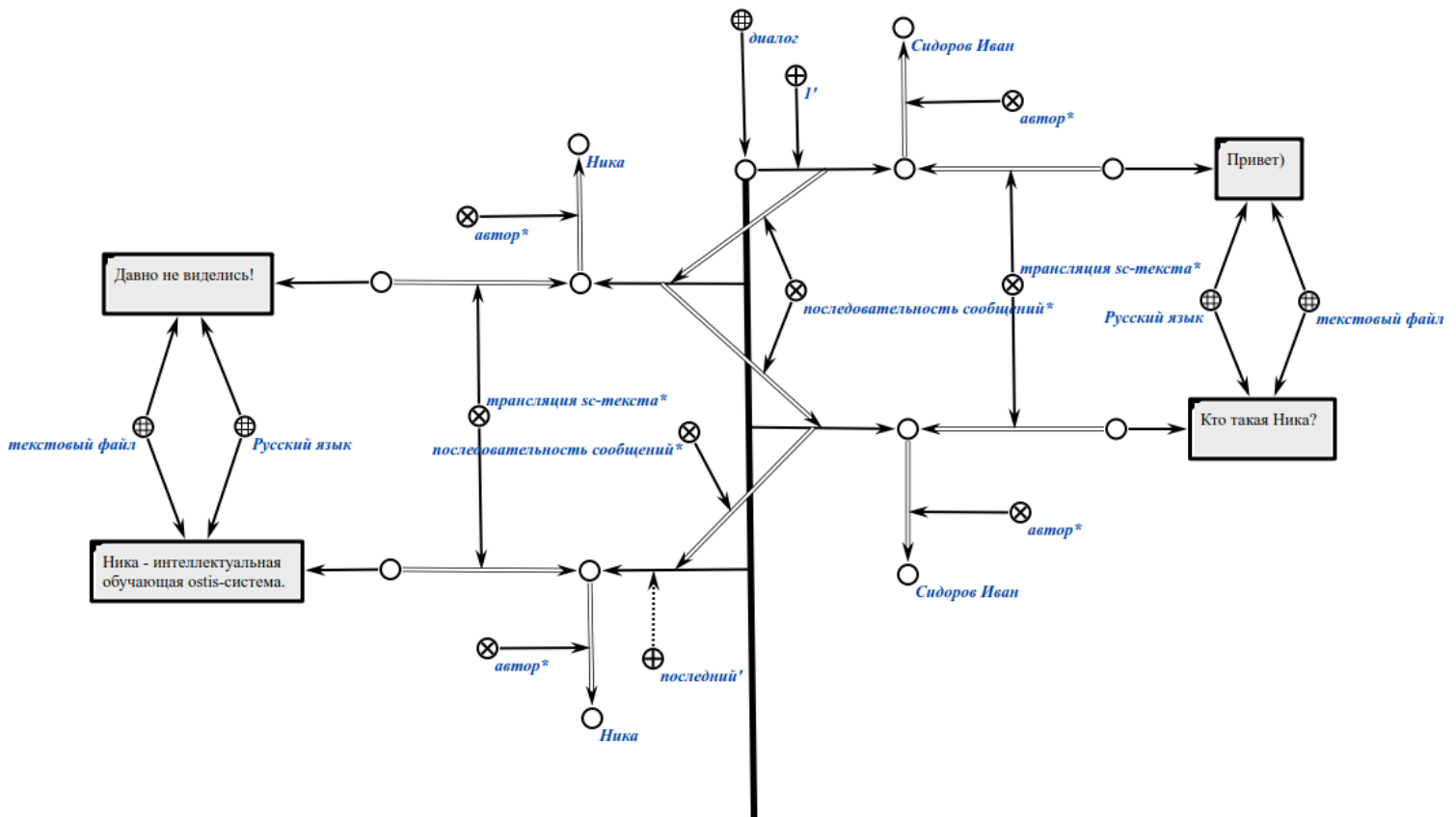
В данной лабораторной работе мы рассмотрим классификацию сообщения по теме. Например, сообщения “Сколько времени?”, “Который час в *Минске*?”, “Подскажите, пожалуйста, сколько сейчас время в *Голландии*?” можно классифицировать как *сообщение запроса времени*. Как Вы можете заметить, некоторые сообщения могут запрашивать время с указанием города или страны, а некоторые без этого уточнения. Для того, чтобы определить ключевые элементы сообщения, выделяются сущности (*entities*) сообщения. Выделение сущностей позволяет различным образом отвечать на сообщения некоторого класса, при этом ответ на сообщение будет зависеть, например, от конкретного города, названия спортивной команды, фильма и других сущностей. В сообщении “Который час в Минске?” можно выделить сущность *Минск*, в сообщении “Подскажите, сколько сейчас время в *Голландии*.” - *Голландия*, а в сообщении “Сколько времени?” сущностей нет, в таком случае система может использовать значения по умолчанию или из других источников, например, определить местоположение пользователя или извлечь знание из истории диалога.

Сообщение необязательно должно иметь только класс намерения (цели) и сущности, определяемые сообщением, также можно классифицировать сообщения по некоторым признакам (*traits*), например, по эмоциональной окраске. Одну и ту же мысль можно выражать по-разному. Сообщение может быть положительной, нейтральной или негативной эмоциональной окраски. “Подскажите, сколько сейчас время в *Голландии*.” можно классифицировать как сообщение с положительной эмоциональной окраской, так как оно содержит вежливые конструкции, сообщение “Сколько времени?” можно считать нейтральным, а “Скажи уже, наконец, сколько времени?” - негативным.

Для того, чтобы научить систему отвечать по новым классам сообщений, необходимо обучить классификатор *wit* на этих классах, правила, по которым система должна отвечать на сообщения и фразы, с помощью которых выводится ответное сообщение по правилам. В данной лабораторной работе необходимо обучить классификатор для ответа на сообщения. Система научится отвечать на новые сообщения после создания логических правил и фраз, то есть выполнения 3 лабораторной работы.

Ход работы

1) После выбора тема предметной области и её написания её формальной спецификации можно приступить к подбору и разработке различных вариантов диалогов. В данной лабораторной работе достаточно привести диалог с системой, состоящий из 8 сообщений. Например, диалог с системой Ника, состоящий из 4 сообщений, может выглядеть таким образом:



На картинке видно, что Сидоров Иван сначала приветствует систему Ника. В то же время система Ника отвечает взаимностью. После собеседник Иван узнаёт назначение системы. Далее можно приступить к обучению системы и описанию ответных фраз.

2, 3, 4) Классификатор сообщений по теме системы НИКА использует открытый облачный сервис wit.ai, который позволяет получать класс намерения сообщения (intent), сущности сообщения (entity) и класс признака (trait). Наглядная инструкция по использованию wit.ai есть на сайте, а также других многочисленных интернет-ресурсах. Далее будет описано, как пользоваться wit.ai, а также подключить своё wit-приложение к системе.

Создание собственного приложения классификации wit.ai

Переходим на wit.ai входим с помощью Facebook (Рис. 4). Если через Google Chrome не удаётся войти, используйте другой веб-браузер (например, Mozilla Firefox).

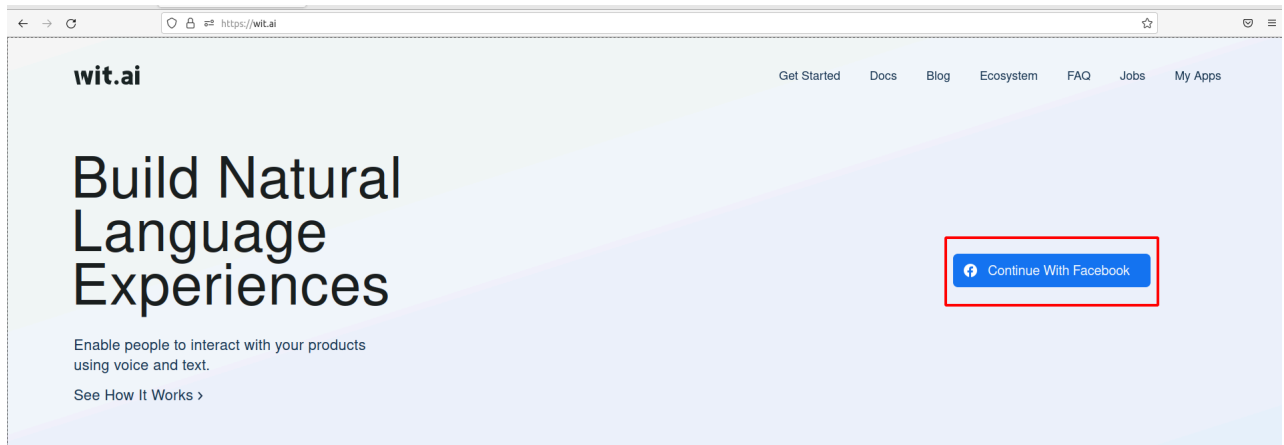


Рис. 4. Главное окно wit.ai

Создаём новое приложение (Рис. 5).

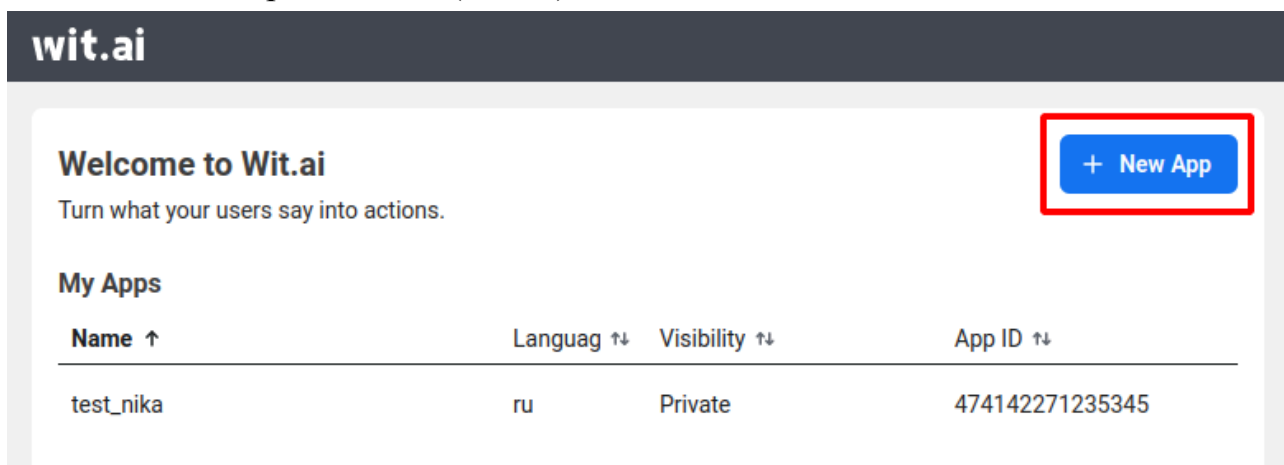


Рис. 5. Создание приложения для классификации

Заполняем поле с именем приложения вписав туда, например, название предметной области, выбираем язык и приватность приложения (Рис. 6).

Create a New App

×

Name

my_subject_domain

Language

Russian (RU) (BETA) ▾

Visibility

☒ Open

☐ Private

Import From a Backup

Browse...

Create

Рис. 6. Настройка создаваемого приложения

Обучение классификатора

Затем во вкладке Management/Intents добавляем классы сообщений, которые мы хотим распознавать (Рис. 7).

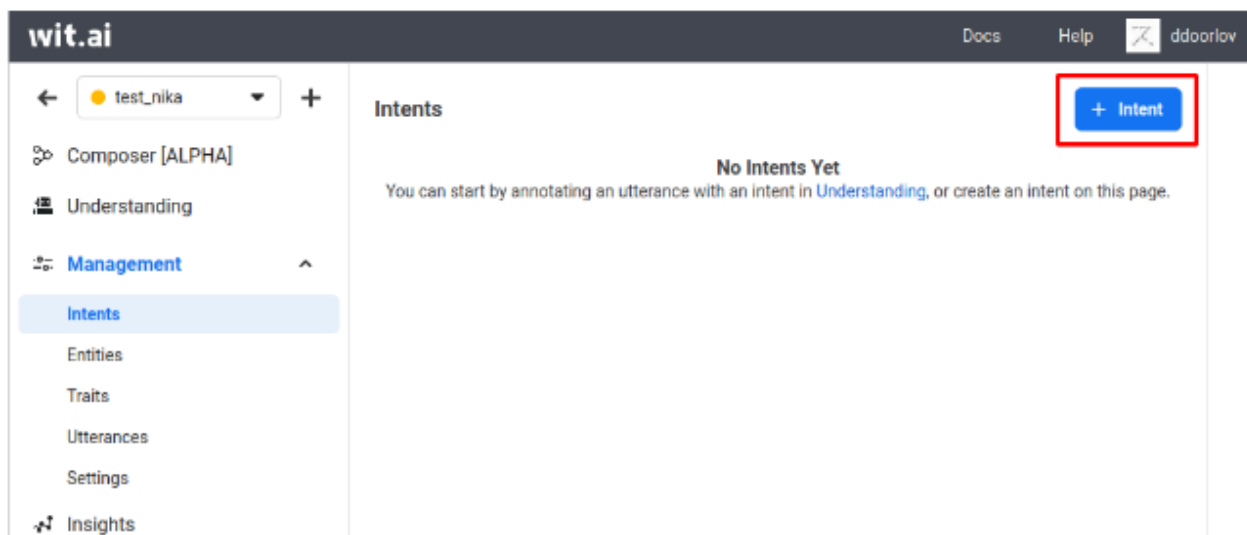


Рис. 7. Создание классов намерений сообщений (wit traits)

Во вкладке Entities необходимо создать роли сущностей, которые мы хотим выделять в сообщении (Рис. 8). При работе агента классификации используется только одна роль - `rel_entity`, остальная информация используется из базы знаний.

New Entity ×

☒ **New custom entity**

Lookup Strategies ⓘ

☐ **Free Text**
An entity that does not belong to a predefined list. You'll use a free-text entity when you're open to new values.

☐ **Keywords**
An entity that belongs to a predefined list.

☒ **Free Text & Keywords**
An entity that is defined by a predefined list, but also open to new values.

☐ **Add built-in entities**

Отмена Next

Рис. 8. Создание ролей сущностей (wit entities)

Переходим в Understanding, вводим разные по формулировке сообщения, которые хотим классифицировать и выбираем их классы, а также выделяем сущности (Рис. 9).

Train Your App

Add a sample utterance and specify an intent. You can also highlight words or phrases in the utterance to annotate.

Utterance ⓘ

“ Кто такая **Ника**? ” 265

Intent ⓘ about_entity ▼ ☐ Out of Scope ⓘ

Entity	Role	Resolved value	Confidence	
rrel_entity	rrel_entity ▼	Ника ▼	N/A	×

+ Add Trait

Train and Validate

Рис. 9. Обучение приложения wit

На данном примере можно использовать такие формулировки как “Что ты знаешь о Нике?” “Что ты можешь сказать о Нике?” и другие.

При обучении можно выделять не только одну сущность, но также и несколько сущностей сразу. В данной ситуации необходимо выделить подходящие роли (intents) для выделяемых сущностей (чтобы отличать их между собой, если это необходимо). Если Вы создаете свою новую роль, то ее необходимо формализовать в базе знаний. Например в вопросе: “Какая погода в Бресте в Беларуси?”, для ответа на вопрос необходимо выделить город Брест и страну Беларусь. Заметьте, что выделяемые сущности лучше распознавать как существительные в именительном падеже.

Add a new utterance

Add a sample utterance and specify an intent. You can also highlight words or phrases in the utterance to annotate.

Utterance ⓘ

“Какая погода в Бресте в Беларуси?”

Intent ⓘ

● about_weather

Entity	Role	Resolved value
rrel_entity	rrel_entity	Брест
rrel_entity	rrel_entity	Беларусь

После этого приложение способно классифицировать похожие сообщения и выделять его сущности. Проверить это можно вводя новые сообщения в окно Utterance, классы и сущности должны определяться автоматически.

Обучение может занять некоторое время, прогресс обучения можно наблюдать в верхнем левом углу на странице Вашего приложения wit.ai.

Подключение классификатора к системе Nika

Переходим в настройки и копируем **Server Access Token** (Рис. 10).

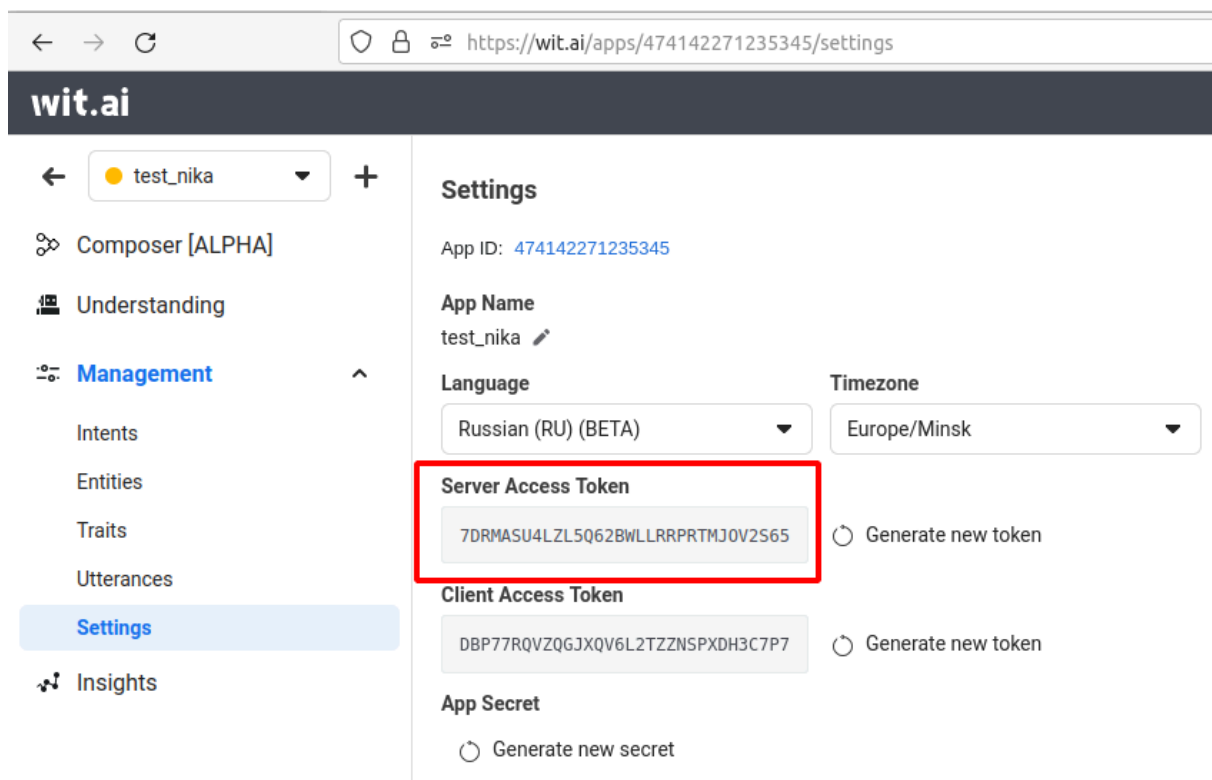


Рис. 10. Получение Server Access Token

Затем записываем данный токен в конфиг по пути “nika/nika.ini” в переменную `server_token`.

```
[wit-ai]
server_token = 7DRMASU4LZL5Q62BWLLRRPRTMJOV2S65
url = https://api.wit.ai/message
```

Далее нужно формализовать новое классифицируемое сообщение. Здесь важно, чтобы содержимое ссылки под отношением *nrel_wit_ai_idtf* было в точности таким же, как и в приложении wit.ai. Фрагмент формализации приведён ниже.

```
concept_message_about_entity
<- sc_node_not_relation;
=> nrel_main_idtf:
  [сообщение о сущности]
  (* <- lang_ru;; *);
=> nrel_wit_ai_idtf:
  [about_entity];
<- rrel_key_sc_element:
  ...
  (*
=> nrel_main_idtf:
  [Опр. (сообщение о сущности)]
  (* <- lang_ru;; *);;
<- definition;;
```



```

<= nrel_sc_text_translation:
    ...
    (*
    -> rrel_example:
[Сообщение о сущности - атомарное сообщение,
 которое содержит сущность, информацию о которой
 необходимо найти.]
    (* <- lang_ru;; *);;
    *);;
<= nrel_using_constants:
    {
        concept_atomic_message
    };;
*);;

```

Исходный текст 5. Пример спецификации класса сообщений о сущности на SCs

Далее во множество распознаваемых сообщений (*concept_intent_possible_class*) необходимо добавить новое классифицируемое сообщение.

```

concept_intent_possible_class
<- sc_node_class;
=> nrel_main_idtf:
    [возможный класс цели Wit.ai]
    (* <- lang_ru;; *);;
<- rrel_key_sc_element:
    ...
    (*
=> nrel_main_idtf:
    [Опр. (возможный класс цели Wit.ai)]
    (* <- lang_ru;; *);;
<- definition;;
<= nrel_sc_text_translation:
    ...
    (*
    -> rrel_example:
        [Возможный класс цели Wit.ai - класс, элементы
 которого могут быть выделены посредством Wit.ai как цели.]
        (* <- lang_ru;; *);;
    *);;
-> concept_greeting_message;
-> concept_message_about_entity;;

```

Исходный текст 6. Спецификация множества распознаваемых сообщений на SCs с добавленным классом сообщений

Аналогичным образом формализуются классы признаков (traits). Необходимо указать идентификатор wit и добавить класс во множество *concept_trait_possible_class*. Классификация, используя traits является необязательной, а используя intent - обязательна.

Для выделения сущностей сообщений необходимо формализовать класс выделяемых сущностей и их роли в сообщении (Рис. 11)

Рекомендуется использовать графический редактор баз знаний [kbe](#).
Сохранённые в формате .gwf файлы также можно погружать в базу знаний.

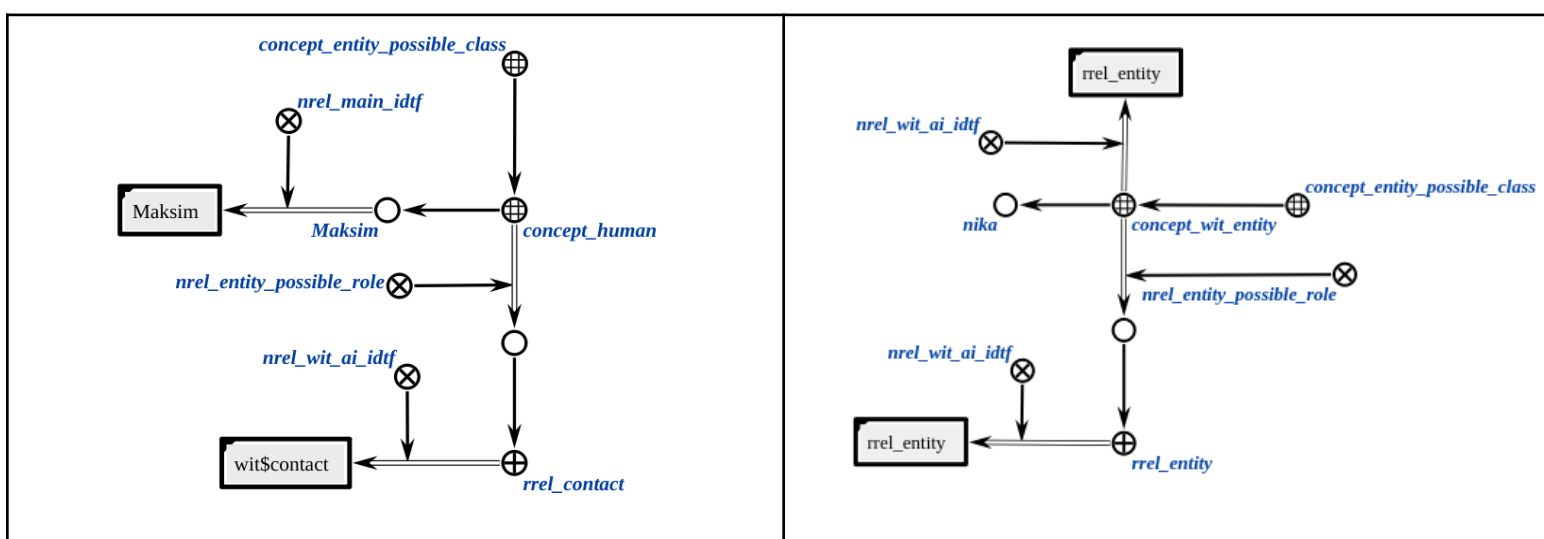


Рис. 11. Пример формализации конструкции, необходимой для выделения сущности

Сервис wit.ai предоставляет некоторые встроенные сущности и классы, например, выделение персоны в сообщении, времени, дистанции, числа, места, денег, электронной почты, признак сообщения по включению/выключению чего-либо и другие (рис. 12). Пользоваться встроенными сущностями и классами разрешается, однако не стоит этим злоупотреблять (максимум две сущности).

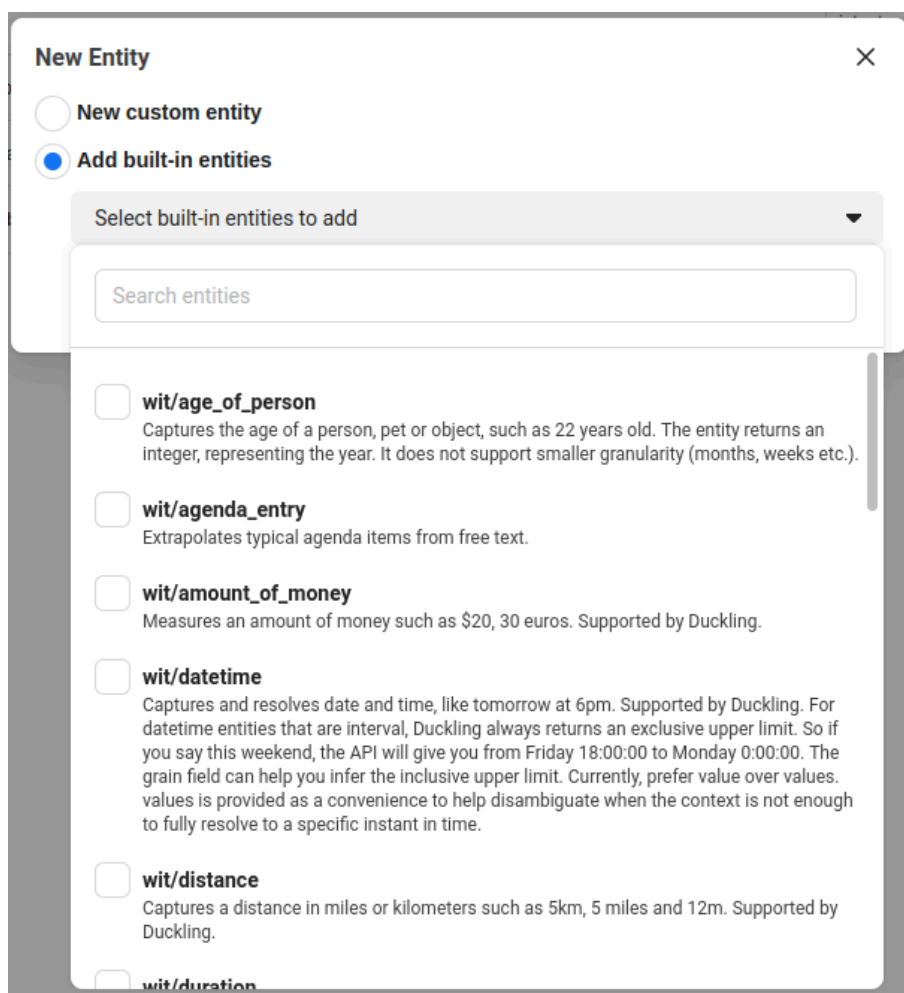


Рис. 12. Заранее обученные сущности wit

Для более полного понимания механизма классификации рекомендуется изучить [документацию агента классификации](#) НИКА и [исходный код агента классификации](#) и связанные с ним фрагменты базы знаний.

После выполнения лабораторной работы, необходимо в папку kb/extra/{папка с исходниками предметной области}/classification/ загрузить исходники с результатами классификации сообщений, предварительно сделав экспорт данных из Wit.ai в формате JSON (Рис. 13).

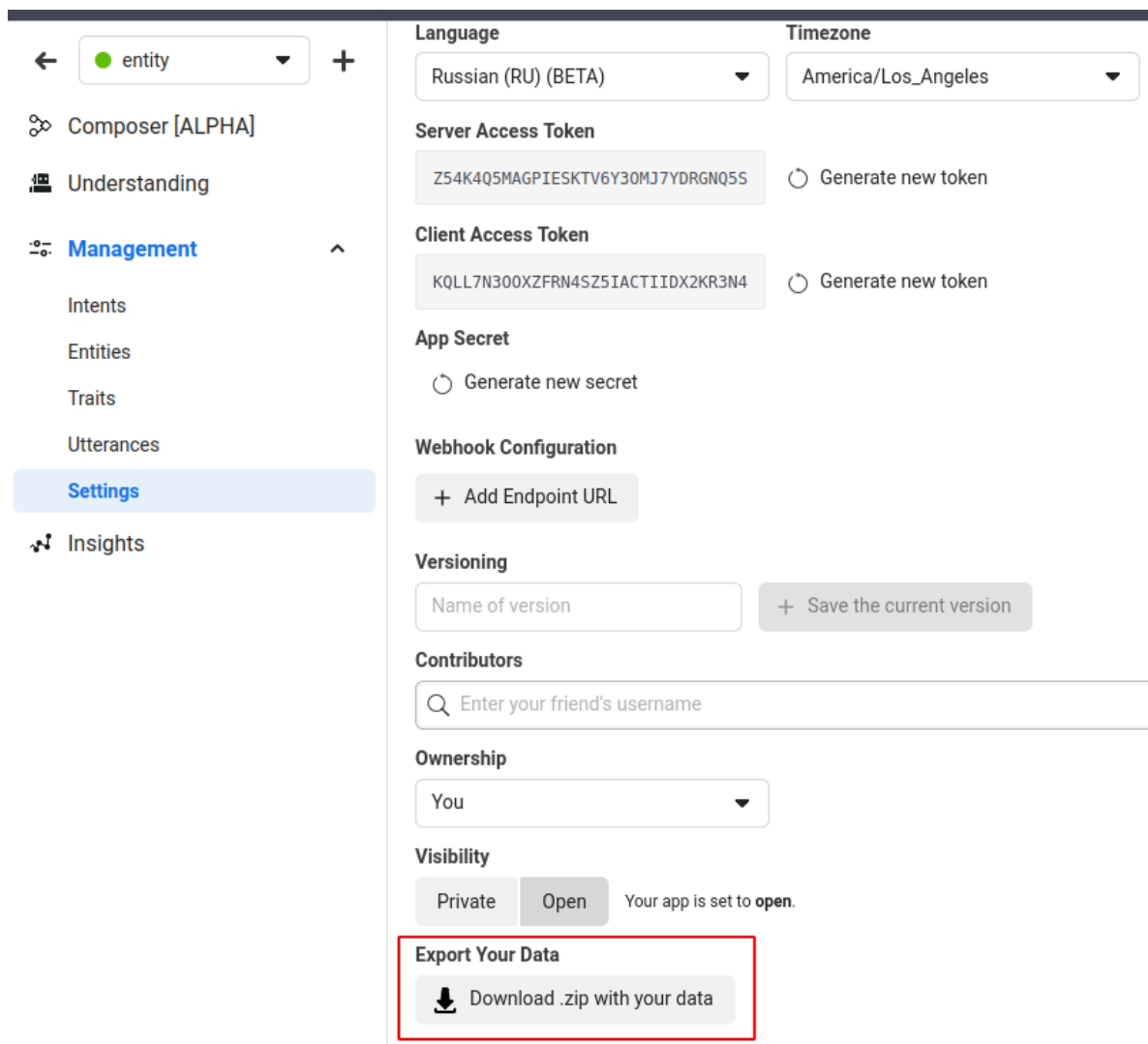


Рис. 13. Экспорт данных wit.ai

К защите

1. Протестировать классификацию сообщений и выделение сущностей, используя wit.ai utterances;
2. Протестировать сообщения с помощью вопроса: “Что такое ...?” для каждой сущности;
3. Сделать отчёт (электронная версия) с иллюстрацией тестирования;
4. Залить изменения в свой форк на GitHub со всеми выполненными заданиями;
5. Подготовиться к вопросам и задачам. Повторить теоретический материал.

Ход защиты

1. Проверка отчёта;
2. Иллюстрация и тестирование классификатора wit;
3. Ответы на вопросы и решение задач от преподавателя;
4. Выставление оценки.

Вопросы

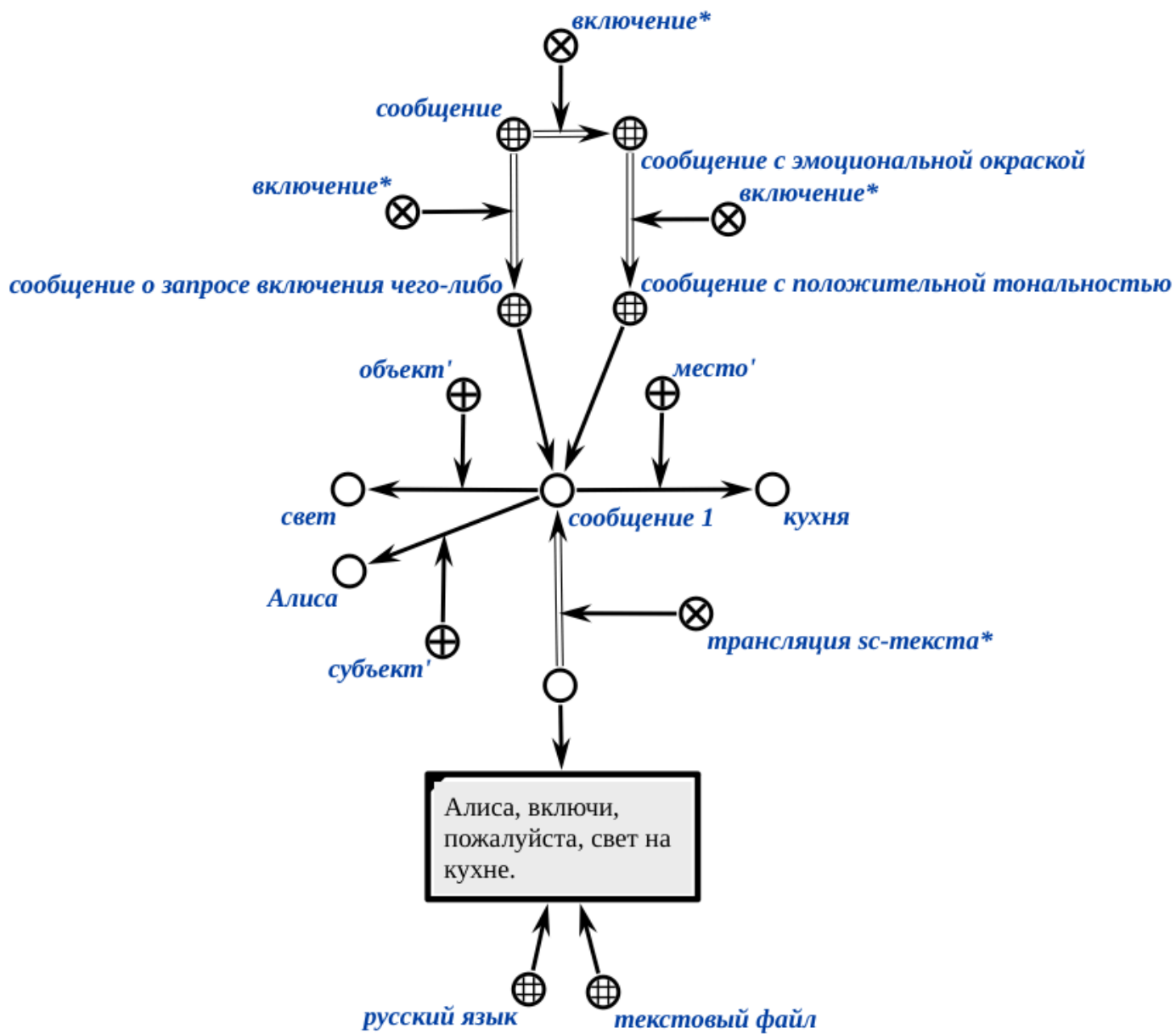
1. Что такое классификация? Какая цель задачи классификации? Как можно классифицировать сообщение?
2. Какие функциональные возможности у сервиса Wit.ai? Какой принцип работы Wit.ai?
3. Алфавит ядра SC-кода и алфавит SC-кода: что из себя представляют, в чем различие?
4. На чём основаны направления расширения Ядра SC-кода?
5. Приведите классификацию интеллектуальных систем.
6. Что такое экспертная система? В чём её отличительная особенность от других систем?
7. Какие существуют формы представления знаний? Какая форма представления знаний используется в ходе выполнения лабораторной работы?
8. Какие требования нужно учитывать при составлении и формализации диалога с компьютерной системой?

Задачи

1. Формализовать сообщение с выделением сущности:

- а. Поставь будильник на 7 утра.
- б. Сколько стоит арбуз на рынке?
- в. Сколько ядер у процессора Intel Core i9-12900K?
- г. Что ты знаешь о “9В” классе?
- д. Расскажи о треугольнике ABC.
- е. Алиса, включи, пожалуйста, свет на кухне.

Пример решения задачи “е”.



Лабораторная работа №3. Проектирование логических правил и фраз для ответа на сообщения системы NIKA

Цель

Изучить принципы работы механизма ответа на сообщения, используя логические правила и фразы, приобрести навыки проектирования логических формул для ведения диалога.

Задание

- 1) Спроектировать логические правила для ответа на сообщения, покрывая каждый класс фраз для ответа (см. ход работы);
 - а) правила должны использовать не только классы сообщений, но и выделяемые в сообщении сущности из первой лабораторной работы;
 - б) необходимо продемонстрировать различные ответы на одно и то же сообщение в зависимости от состояния базы знаний, для этого необходимо использовать фразы с шаблонами из второй лабораторной работы.
 - с) обязательным является условие получения ответа на сообщение, которое можно увидеть, используя пользовательский интерфейс системы NIKA.
- 2) Описать фразы ответных сообщений в базе знаний:
 - а) фразы сообщений не должны повторяться,
 - б) для каждого класса ответных фраз должно быть не менее 2 фраз и хотя бы 3 фразы на весь диалог с использованием шаблона ответа;
- 3) По результатам работы составить отчёт, где указать цель, задание, вариант и показать тестирование сценария диалога;
- 4) Уметь отвечать на вопросы и решать задачи, указанные в конце лабораторной работы.
- 5) Залить изменения в свой форк.

Алгоритм опубликования результатов работы:

1. Сделать форк проекта системы NIKA <https://github.com/ostis-apps/nika>
2. Установить форк проекта системы у себя локально:

```
git clone https://github.com/{имя_вашего_аккаунта}/nika
```

```
git checkout tpis-2023
```
3. Перейти в папку kb/extra и погрузить туда исходники фрагментов базы знаний;
4. Вернуться в корень проекта, сделать коммит:

```
git add kb/extra/
```

```
git commit -m "feat(kb): <название вашей предметной области>"
```

5. и сделать пуш изменений:

```
git push
```

6. Залить изменения в свой форк.

Критерии аттестации:

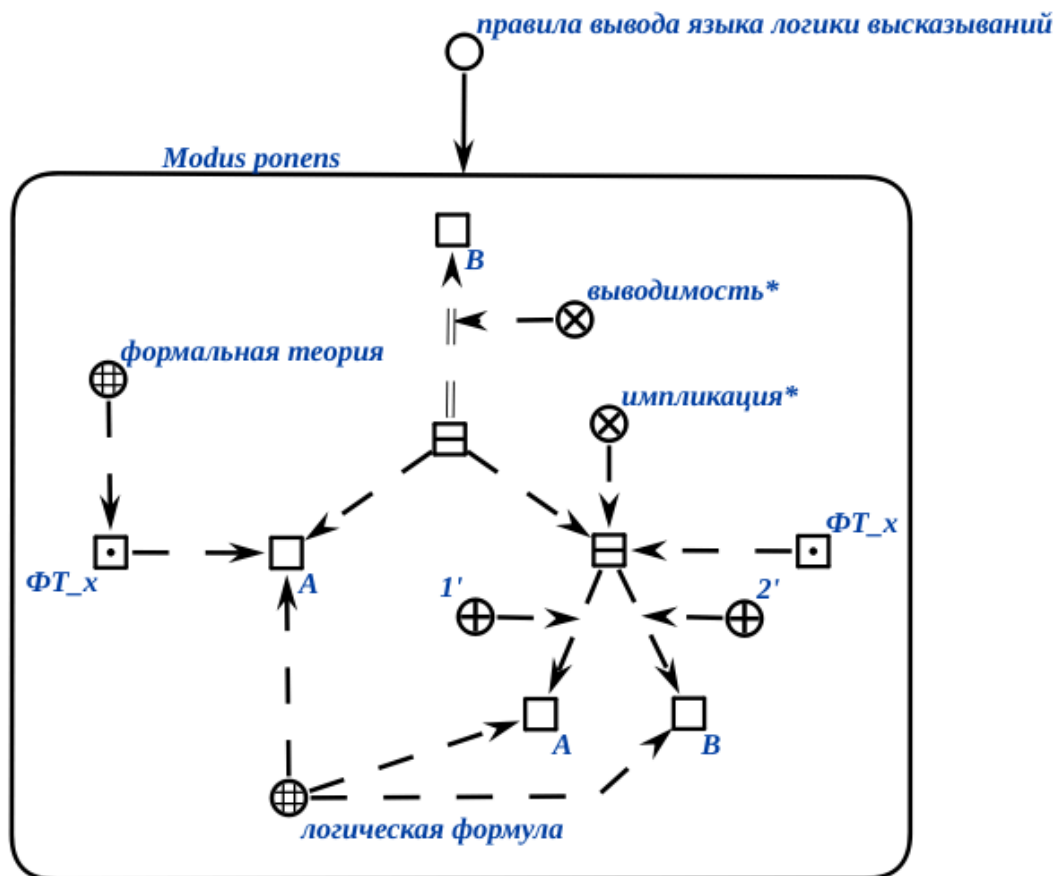
1. Осмысленность правил и фраз для сообщений в исследуемой предметной области (правила и фразы должны быть корректны и адекватны по смыслу) - до 2 баллов;
2. Сложность правил - до 2 баллов;
3. Качество отчёта - до 1 балла;
4. Умение отвечать на вопросы и решать задачи - до 3 баллов;
5. Срок выполнения - до 2 баллов.

Максимальный балл - 10.

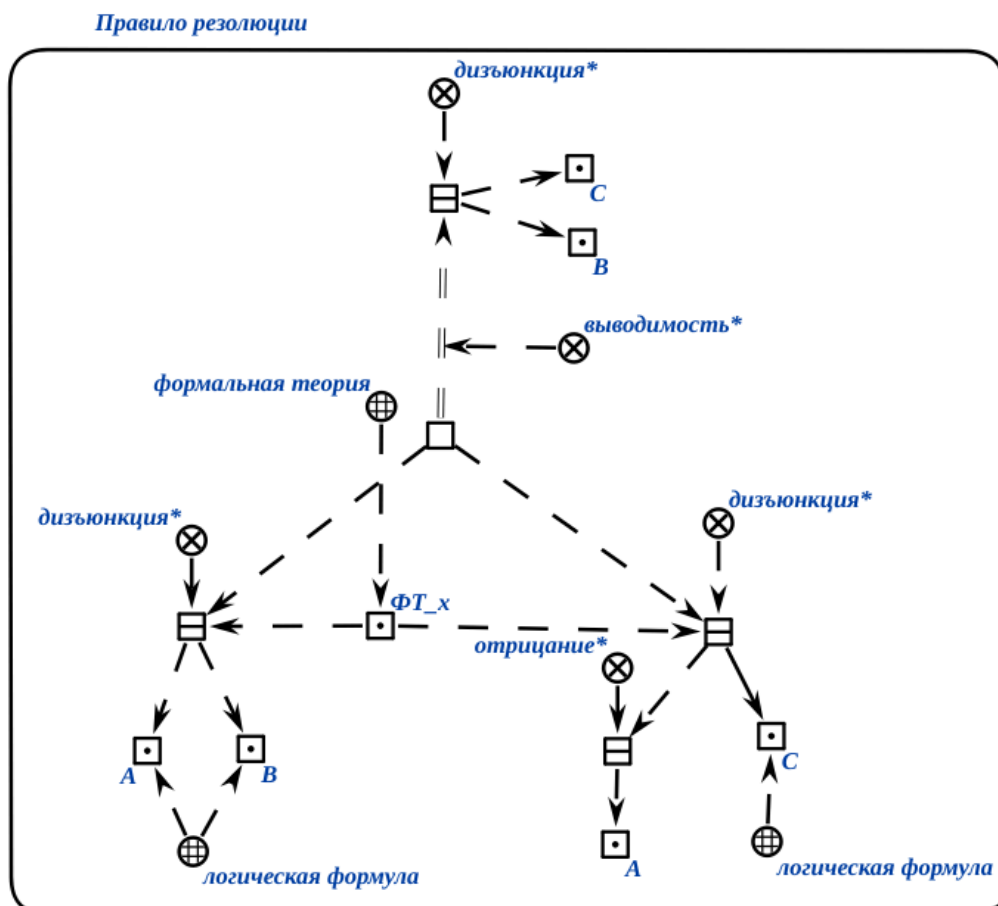
Срок сдачи - до 12 ноября.

Теоретические сведения

Экспертные системы моделируют работу эксперта, то есть человека, обладающего большим опытом в некоторой предметной области. Экспертные системы позволяют заменить дорогостоящего эксперта путём переноса его знаний в **базу знаний** экспертной системы. Наиболее ценными знаниями в такой ситуации являются высказывания (как фактографические, так и логические формулы). Используя высказывания, можно получать новые знания (новые формулы) на основе существующих с помощью аппарата логического вывода. Например, есть общая схема рассуждения, правило вывода **Modus ponens**, которое формально можно сформулировать следующим образом: если есть знание А, а также знание, что из А следует В (А влечёт В, А выводит В), то истинным является и знание В. Таким образом при наличии в базе знаний структур, соответствующих данному правилу вывода, можно получать новые знания в рамках некоторой формальной теории. Например, эксперт сформулировал такое правило из предметной области геометрии Евклида: если треугольник имеет угол, градусная мера которого равна 90 градусов, такой треугольник является прямоугольным. Другим примером может быть теорема Пифагора, теорема синусов, формула расчёта площади прямоугольного треугольника и другие. Используя такие правила и знания о предметных объектах, можно получить новые знания, изначально неочевидные.



Другим простейшим правилом вывода, используемым в экспертных системах, является *правило резолюции*. Оно гласит, что если истинно А или В, а также истинно С или не В, то истинно А или С.



Формальная теория — это множество высказываний, которые считаются истинными в рамках данной формальной теории. некоторые высказывания считаются аксиомами, а другие доказываются на основе других высказываний в рамках этой же формальной теории.

Под *высказыванием* понимается некоторая структура (в которую входят sc-константы из некоторой предметной области и/или sc-переменные) или логическая связка, которая может трактоваться как истинная или ложная в рамках какой-либо предметной области.

высказывание

=> разбиение*:

{

- атомарное высказывание
- неатомарное высказывание

}

=> разбиение*:

{

- фактографическое высказывание
- логическая формула

}

Атомарное высказывание — это высказывание, которое содержит хотя бы один sc-элемент, не являющийся знаком другого высказывания.

Неатомарное высказывание — это высказывание, в состав которого входят только знаки других высказываний. Нельзя говорить об истинности либо ложности неатомарного высказывания в рамках какой-либо формальной теории, в случае, когда невозможно установить истинность либо ложность любого из его элементов в рамках этой же формальной теории. Высказывание также может быть бессмысленным в случае, если в какое-либо атомарное высказывание входит какая-либо sc-константа, не являющаяся элементом предметной области, описываемой указанной формальной теории.

Под *фактографическим высказыванием* понимается атомарное высказывание, в состав которого не входит ни одна sc-переменная или атомарное высказывание, все элементы которого также являются фактографическими высказываниями.

логическая формула

=> разбиение*:

{

- атомарная логическая формула
- неатомарная логическая формула

}

Под логической формулой понимается:

- Атомарное высказывание, в состав которого входит хотя бы одна sc-переменная;
- Неатомарное высказывание, хотя бы один элемент которого является логической формулой.

Под атомарной логической формулой понимается атомарное высказывание, которое является логической формулой. Атомарная логическая формула не содержит логических связок.

Логическая связка - это отношение, связками которого являются высказывания.

Конъюнкция - это множество конъюнктивных высказываний, каждое из которых истинно в рамках некоторой формальной теории только в том случае, когда все его компоненты истинны в рамках этой же формальной теории.

Дизъюнкция – это множество дизъюнктивных высказываний, каждое из которых истинно в рамках некоторой формальной теории только в том случае, когда хотя бы один его компонент является истинным в рамках этой же формальной теории.

Отрицание – это множество высказываний об отрицании, каждое из которых истинно в рамках некоторой формальной теории только в том случае, когда его единственный элемент является ложным в рамках этой же формальной теории.

Импликация – это множество имплицативных неатомарных высказываний, каждое из которых состоит из посылки (первый компонент высказывания) и следствия (второй компонент высказывания). На рисунке 1 показан пример импликации, которая содержит следующую информацию: для любых переменных *_triangle* и *_angle* если *_triangle* является прямоугольным треугольником, то синус его внутреннего угла *_angle* равен единице.

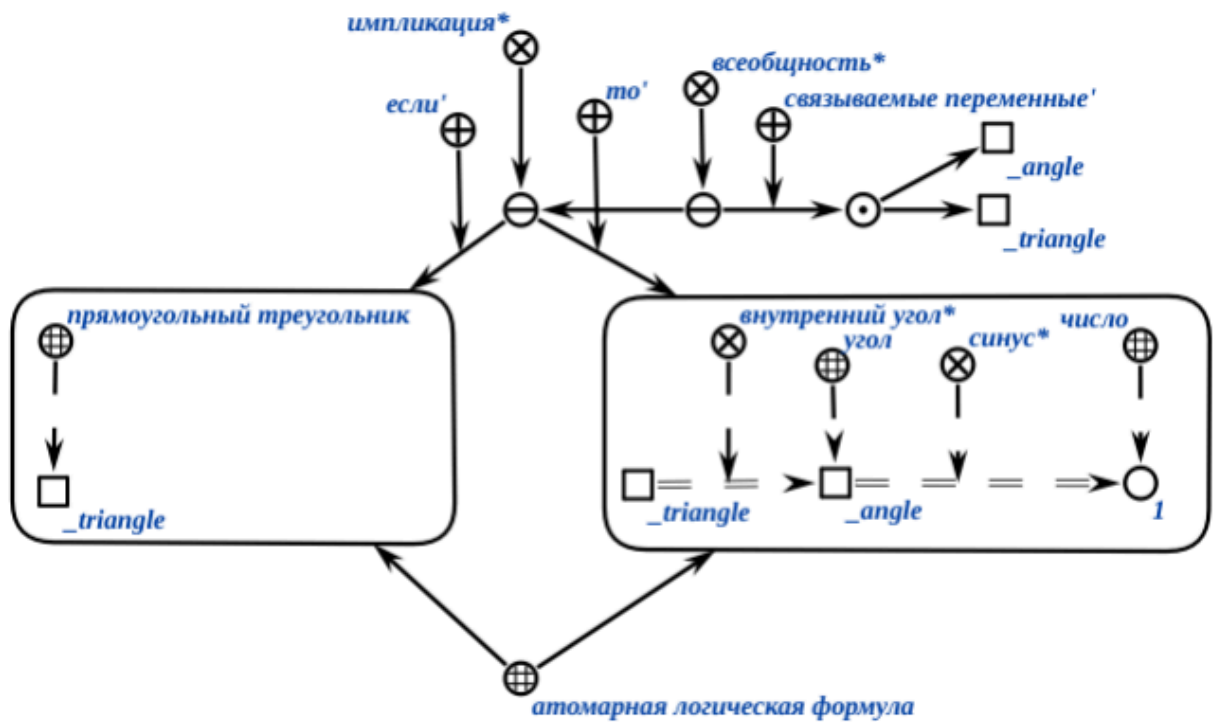


Рисунок 1. Пример импликации

По умолчанию атомарная логическая формула трактуется как высказывание о существовании, то есть наличия в памяти значений, соответствующих всем sc-переменным, входящим в состав данной формулы и не попадающих под действие какого-либо другого квантора (указанного явно или по умолчанию). Таким образом, на все sc-переменные, входящие в состав атомарной логической формулы и не попадающие под действие какого-либо другого квантора, неявно накладывается квантор существования. По умолчанию на все переменные, входящие в обе части высказывания об импликации (или хотя бы одну из подформул* каждой части) неявно накладывается квантор всеобщности*, при условии, что эти переменные не связаны другим квантором, указанным явно. То же самое справедливо для эквиваленции.

Для описания сложных закономерной в базе знаний используют неатомарные логические формулы, представляющие собой комбинацию различных логических связок и кванторов. На рисунке 2 представлен пример формулы, которая иллюстрирует теорему о сумме геометрических отрезков.

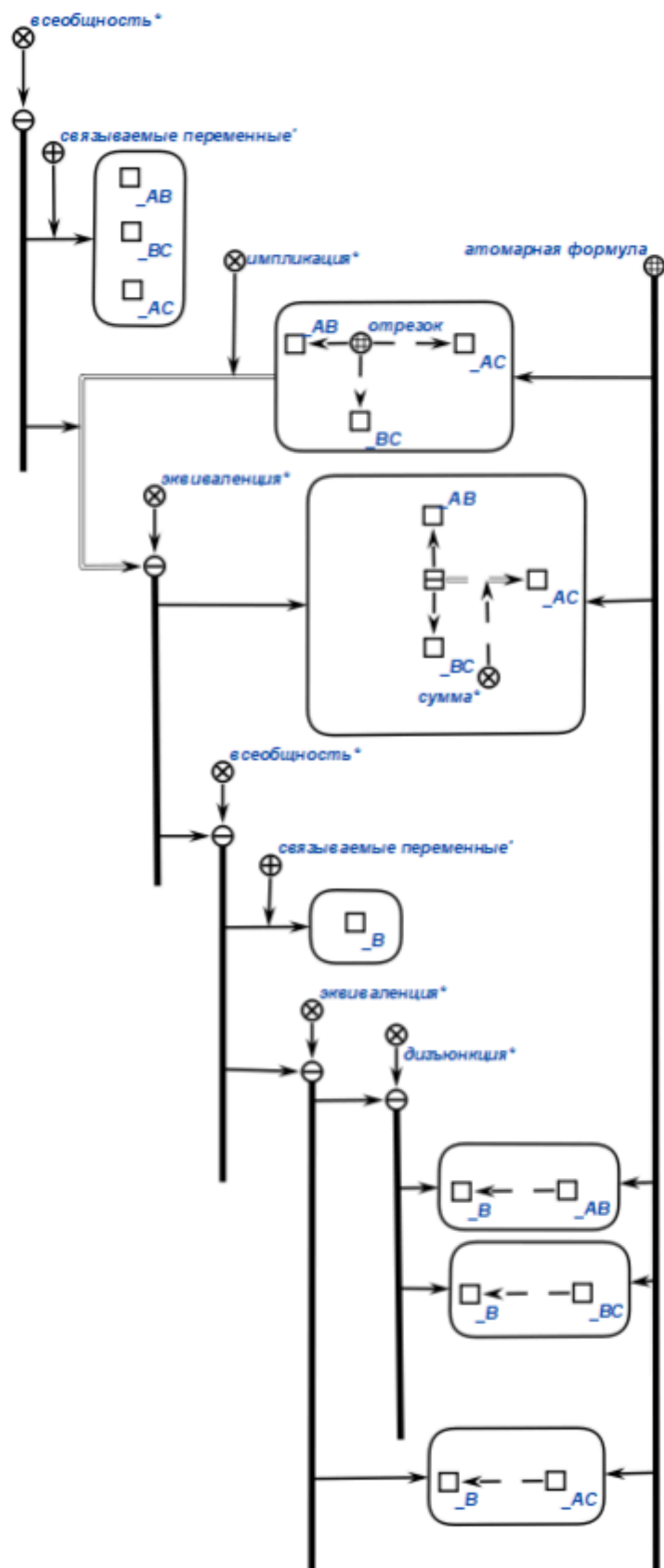


Рисунок 2. Пример логической формулы о сумме геометрических отрезков

Фразы для ответа

Фраза для ответа на сообщение представляет собой sc-ссылку, содержимое которой используется при генерации ответного сообщения на сообщение пользователя. Для фразы обязательно указывается язык путём добавления фразы во множество соответствующего языка (проведением дуги принадлежности от класса языка, например `lang_ru`, к фразе). Также для фразы указывается как минимум один класс, который описывает тему фразы (например, приветствие и позитивное приветствие). Фразы могут быть простыми (рис. 1), без использования шаблона, и сложные (рис. 2), с использованием шаблона фразы. Шаблоны позволяют учитывать состояние базы знаний при ответе на сообщение. Чем больше переменных во фразе, тем более вариативный и осмысленный получается ответ.

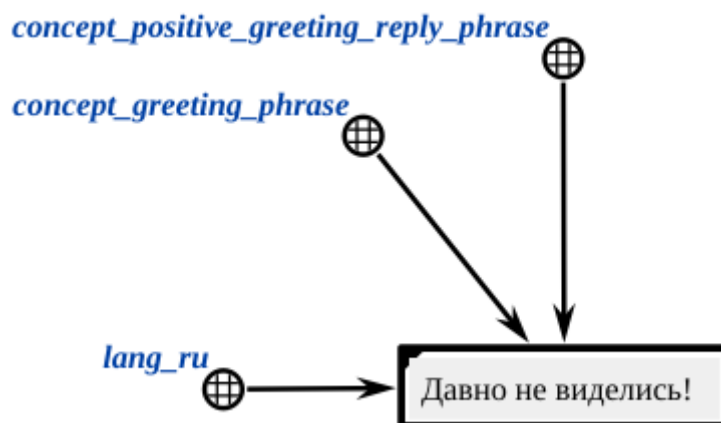


Рис. 1. Формализация фразы приветствия без использования шаблона.

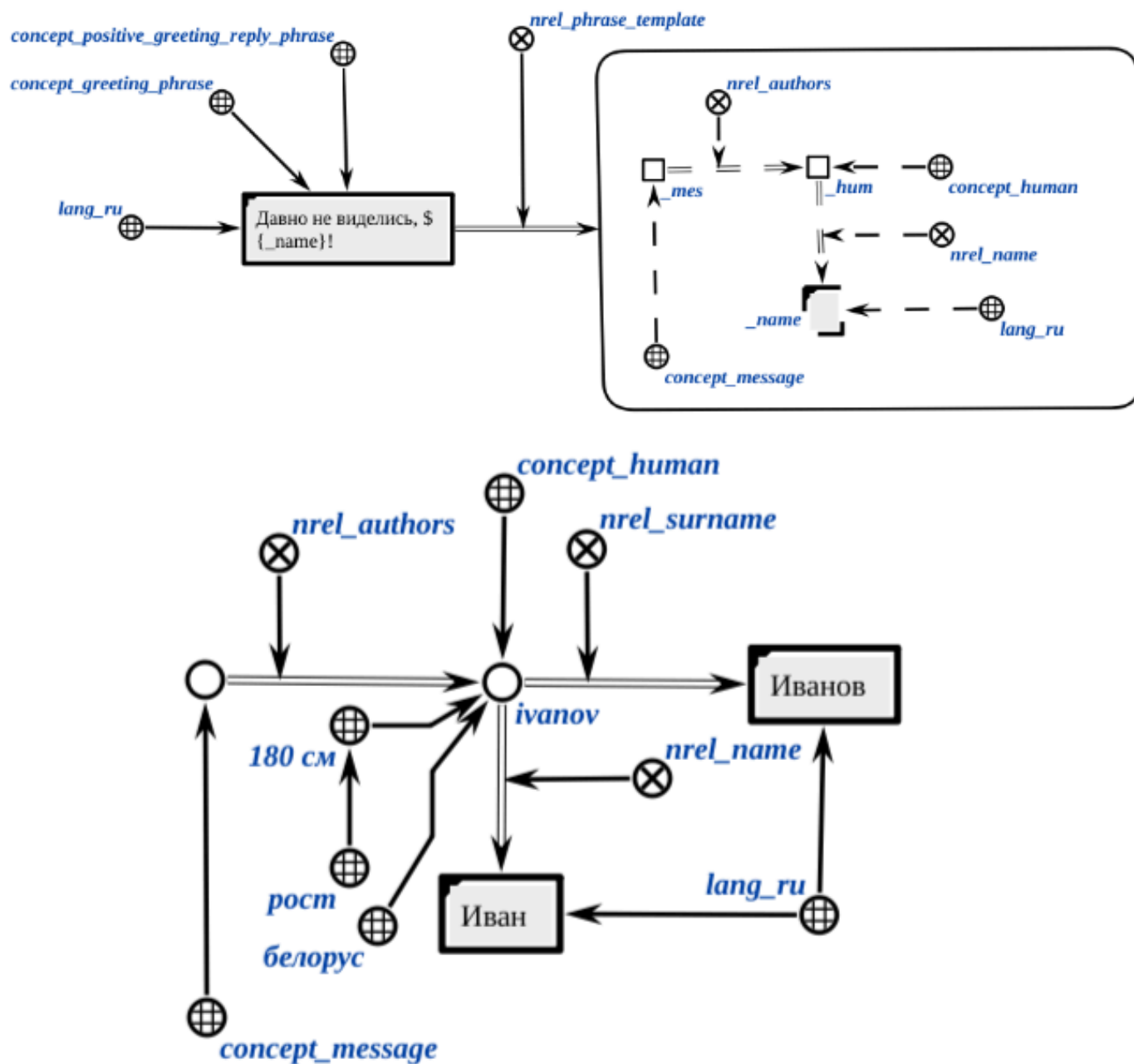


Рис. 2. Формализация фразы приветствия с использованием шаблона и соответствующий фрагмент базы знаний.

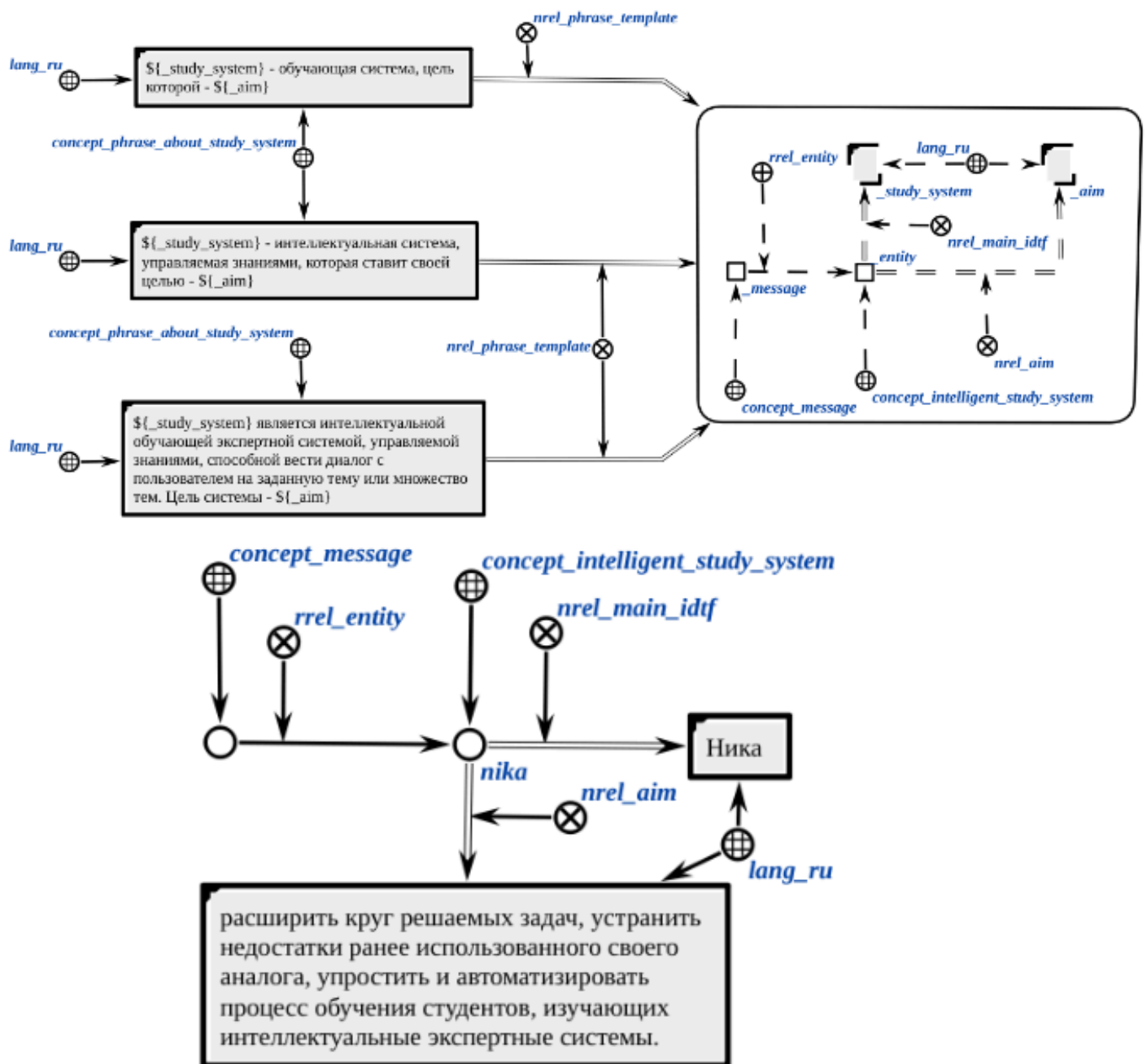


Рис. 3. Формализация фразы про обучающую систему с использованием шаблона.

Чтобы подставить в sc-ссылку ответной фразы содержимое ссылки из базы знаний, нужно использовать фигурные скобки, в которые заключён идентификатор той ссылки, содержимое которой подставляется в ответную фразу. На рис. 2 такой ссылкой является ссылка с идентификатором $_name$.

Для того, чтобы система при ответе на сообщение могла воспользоваться фразой с шаблоном, необходимо, чтобы в базе знаний хранились конструкции, которые находятся в шаблоне, иначе система не сможет ответить по желаемому правилу и ответом станет фраза непонимания (Я не понимаю Вас).

Ход работы

Формализация логических правил.

После выполнения первой и второй лабораторной работы, база знаний экспертной обучающей системы NIKA содержит формализованную предметную область, включая множество сообщений, с помощью которых система сможет отвечать на сообщения пользователя. В данной лабораторной работе предлагается спроектировать логические правила и фразы для ответа, с помощью которых система сможет использовать полученные данные фразы для ведения диалога. Пример простейшего правила приведён на рисунке 3.

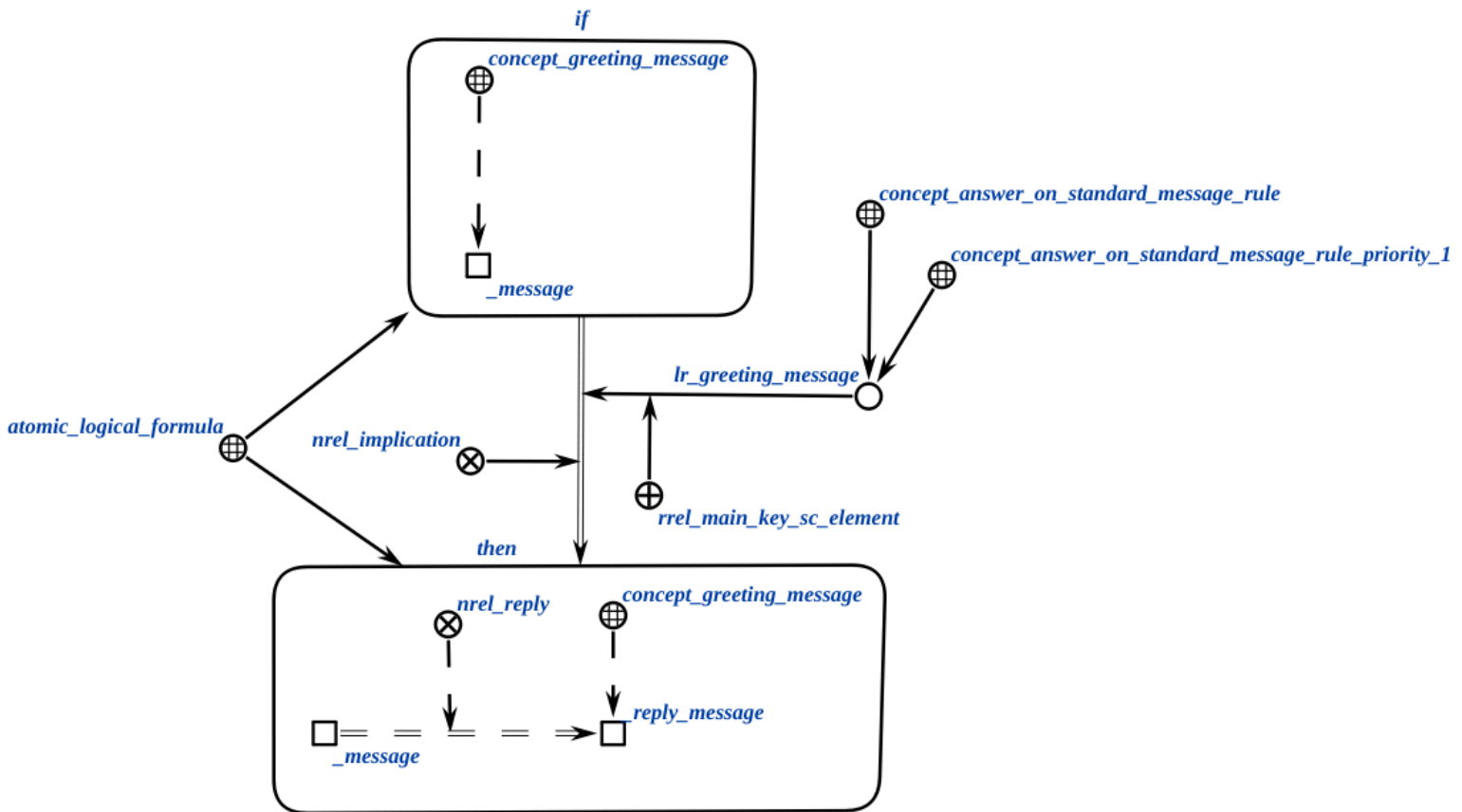
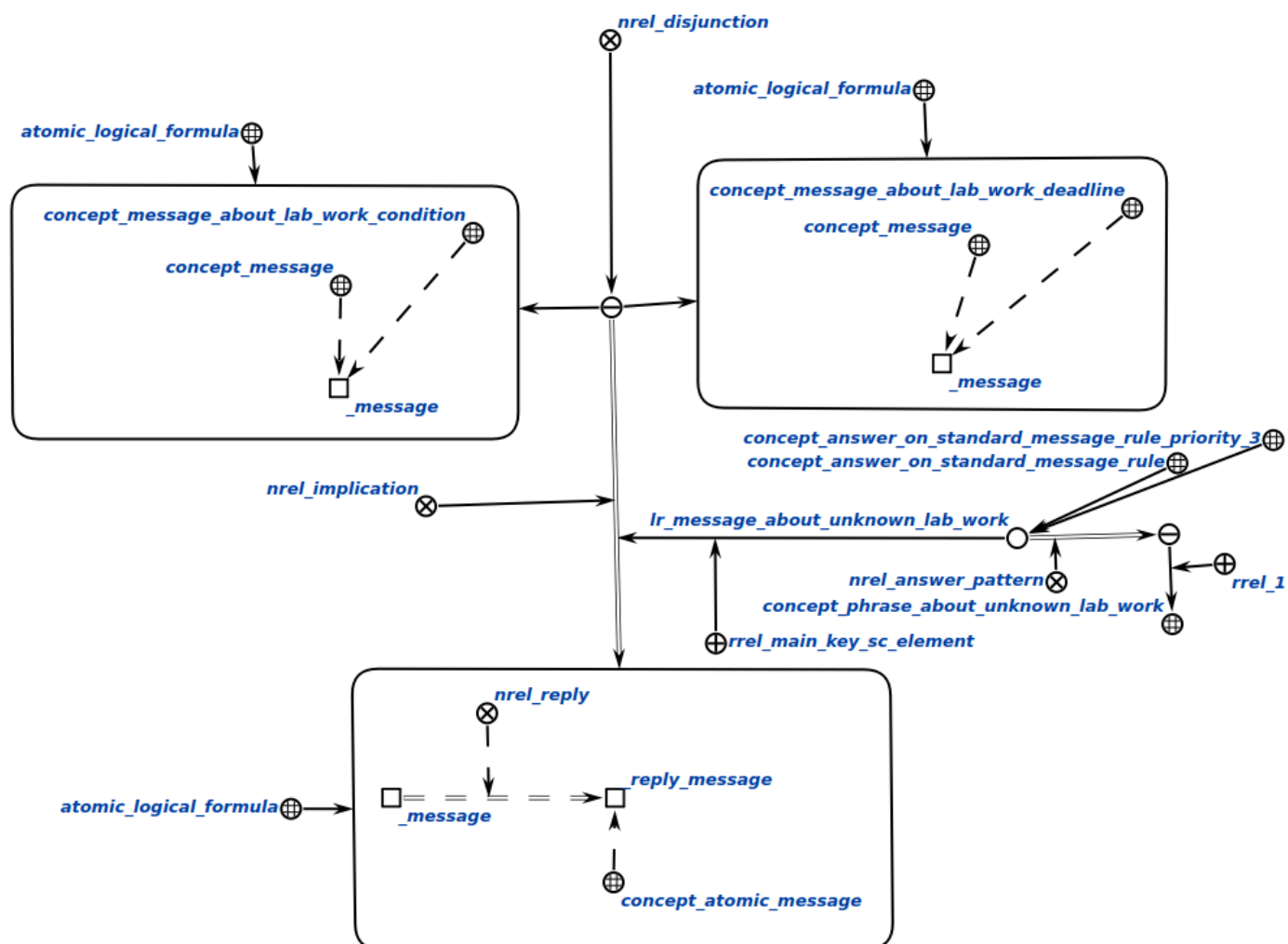


Рисунок 3. Пример логического правила для ответа на приветственное сообщение

Также можно использовать логическое правило для ответа на сообщение про условие или дедлайн неизвестной лабораторной работы с использованием логической связки “или”, выраженной с помощью неролевого отношения “nrel_disjunction”:



Логическое правило на рисунке 3 состоит из одной логической связи импликации (правило вида *если-то*). Посылкой (если) данной импликации является атомарная логическая формула (sc-контур с переменными), содержащий переменную *_message* и один класс, которому она принадлежит: *concept_greeting_message*. Следствием (то) данной формулы является контур, содержащий информацию о том, что ответом (*nrel_reply*) на такое сообщение должно быть сообщение класса *concept_greeting_message*. Таким образом, при появлении в памяти конструкции из посылки (то есть при отправке сообщения, которое было классифицировано как сообщение приветствия) генерируется конструкция из следствия (появляется новый узел сообщения, являющийся ответом на поступившее сообщение пользователя и принадлежащее классу приветственного сообщения). Как посылка, так и следствие могут быть насколько угодно большими и сложными, однако для того, чтобы правило сработало, обязательно в памяти системы должна быть конструкция из посылки импликации.

Каждое логическое правило должно иметь **шаблон ответа** (*nrel_answer_pattern*), это множество, которому принадлежит тот класс фраз, который используется при ответе на сообщение по данному правилу (рис. 4).

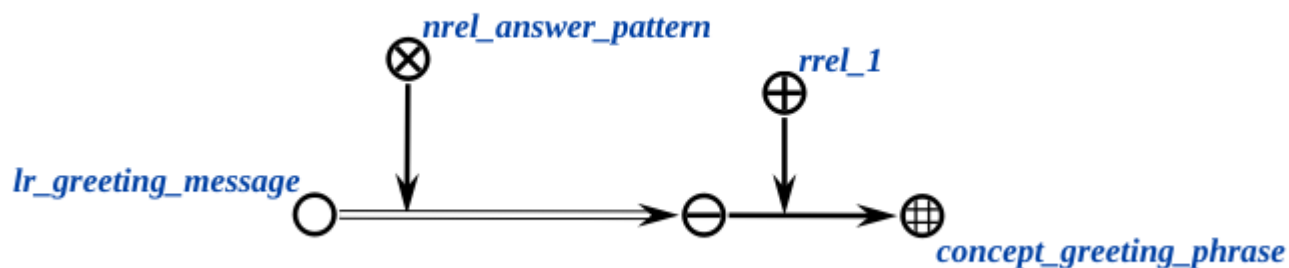


Рисунок 4. Шаблон ответа на сообщение по правилу

Таким образом, используя правило на рисунке 3, шаблон фразы на рисунке 4 и фразы на рисунке 5, можно получить ответ от системы, как на рисунке 6.

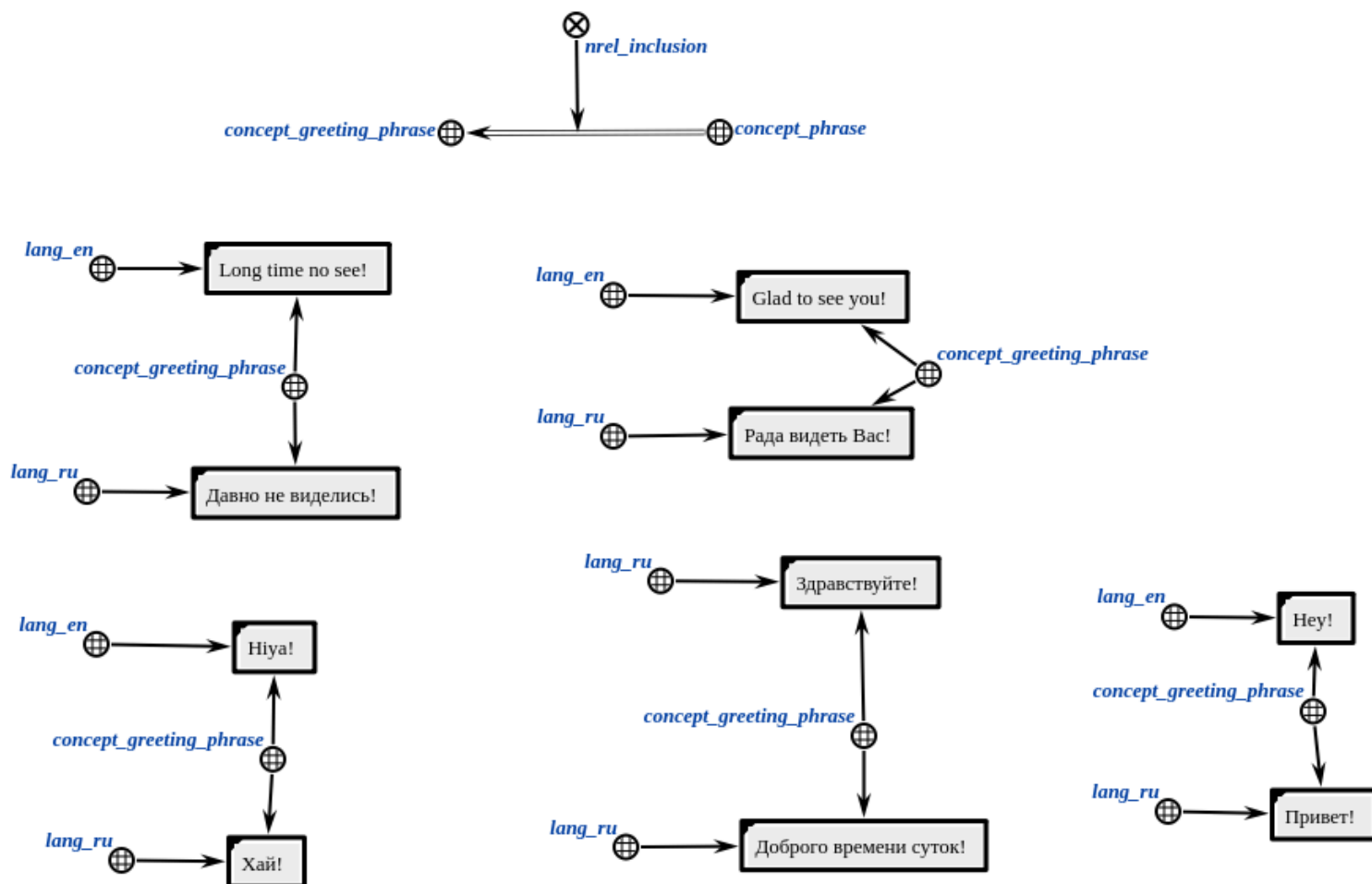


Рисунок 5. Формализация фраз приветствия.

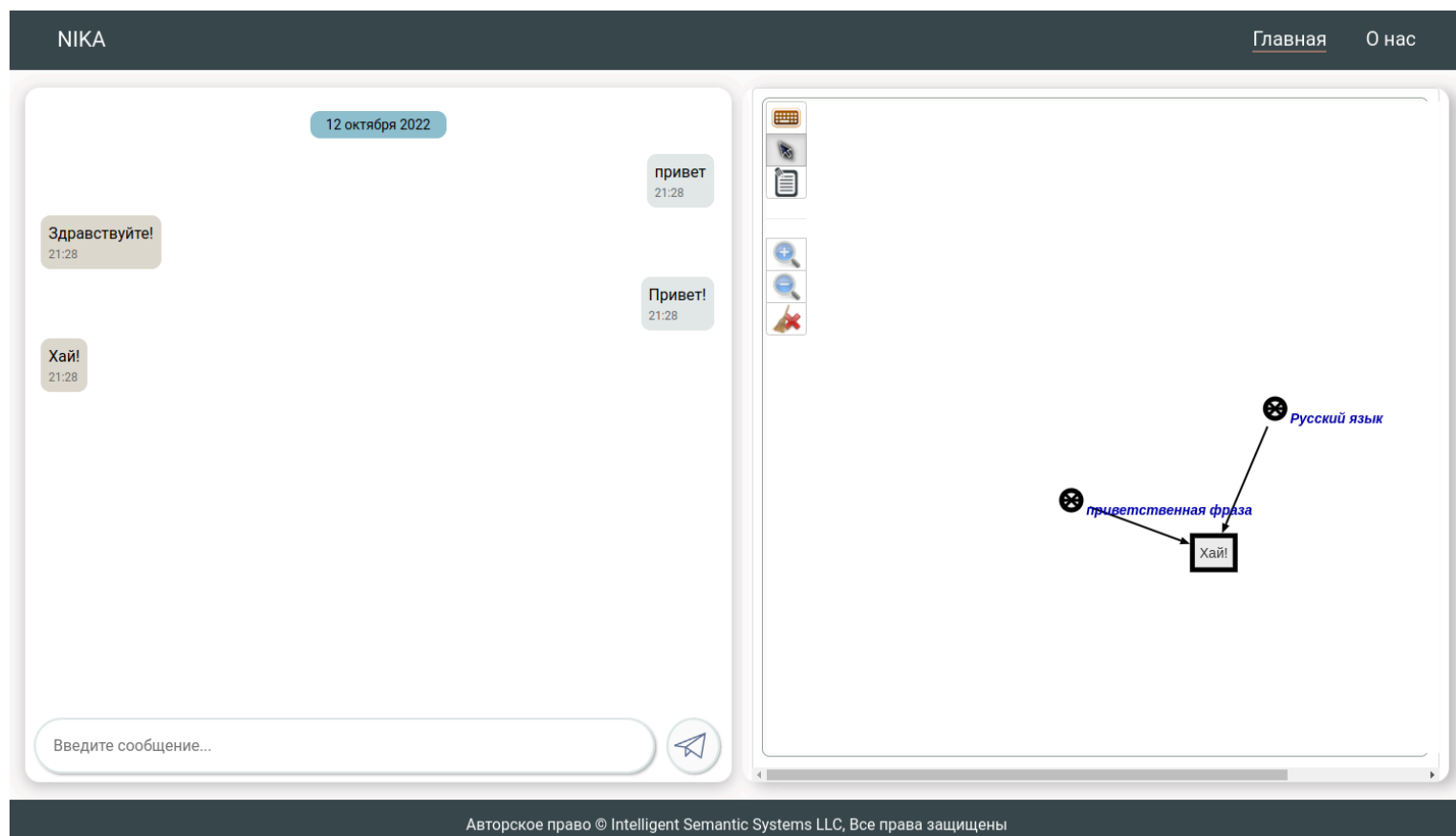


Рисунок 6. Пример ответа на сообщения с помощью логических правил

Сообщение для ответа выбирается случайным образом из множества фраз того класса, который указан в шаблоне ответа логического права.

Пример логической формулы с использованием сущности, выделяемой в сообщении, представлен на рисунке 7.

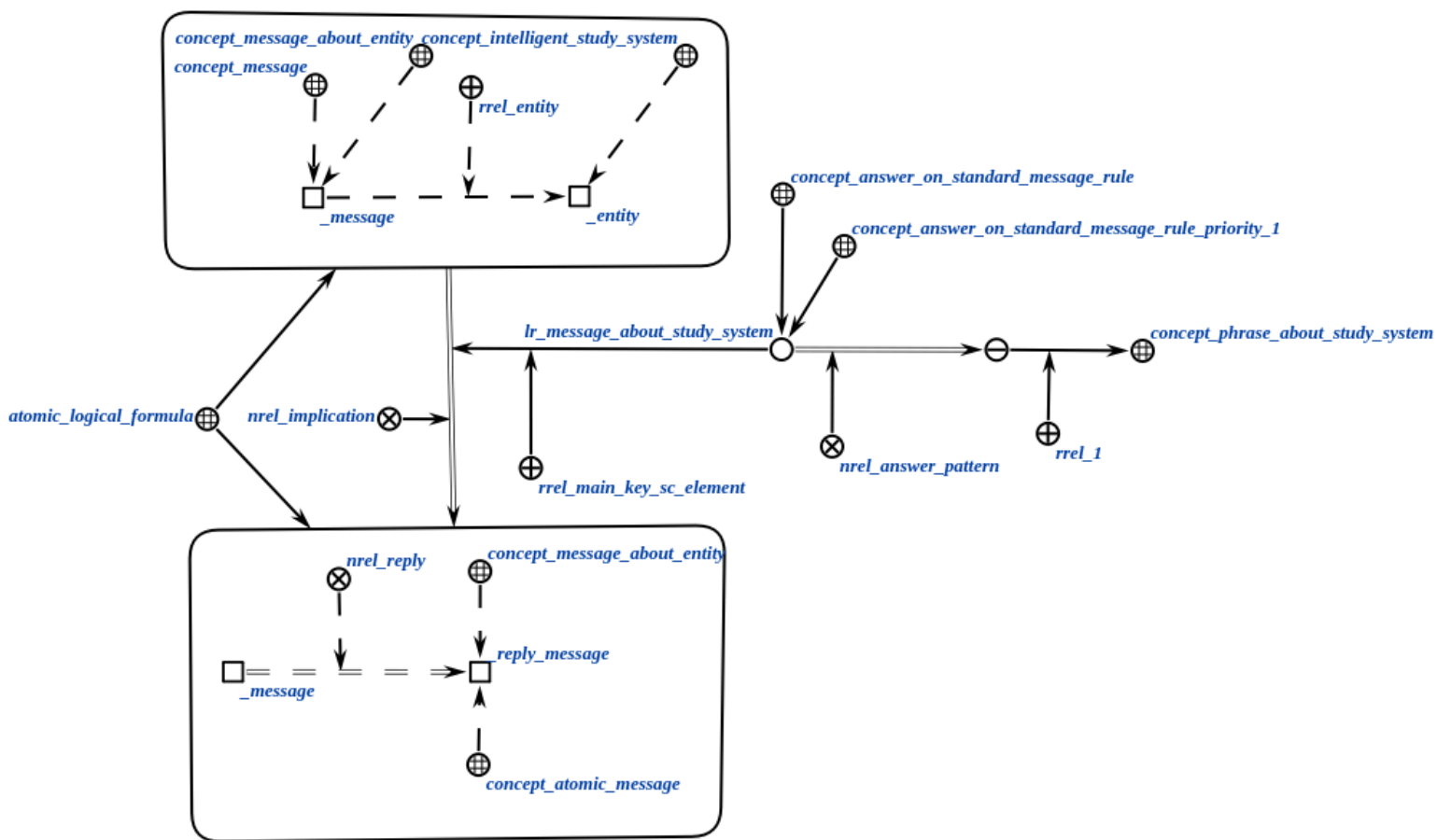


Рисунок 7. Пример правила с использованием сущности, выделяемой в сообщении

В данном примере показано, что если появилось сообщение (*concept_message*), классифицированное как сообщение о сущности (*concept_message_about_entity*) с выделенной сущностью (*rrel_entity*) *_entity*, которая принадлежит классу *concept_intelligent_study_system*, то генерируется ответное сообщение (связка *nrel_reply* от *_message* к *_reply_message*), принадлежащее классам *concept_message_about_entity* и *concept_atomic_message*. Ответной фразой по такому правилу может быть фраза с использованием шаблона, где конкретная сущность используется в ответе.

Формализация фраз ответа

Формализация фраз для ответа на классифицируемое сообщение (Рис. 8). При наличии нескольких фраз, принадлежащих одному классу фраз, система при ответе выбирает любую фразу случайным образом.

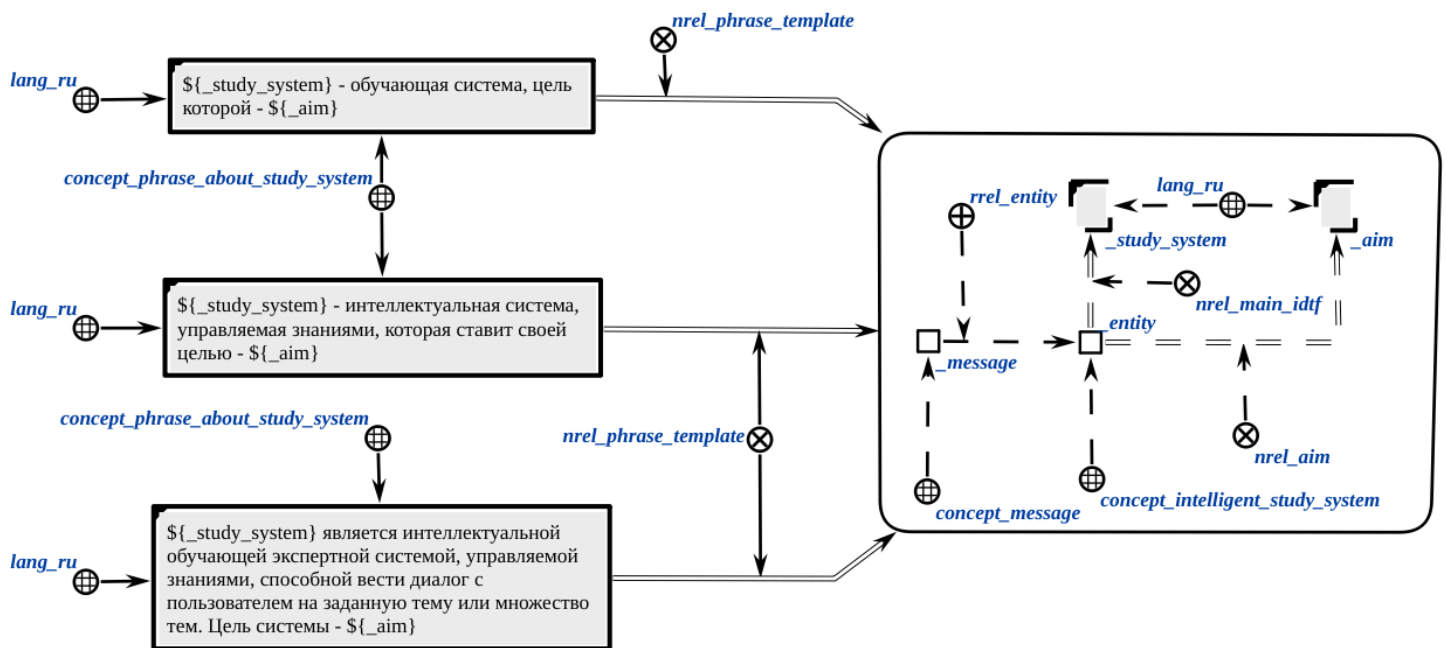


Рис. 8. Формализация фразы про обучающую систему с использованием шаблона.

В данном примере показана формализация фразы ответа с использованием переменных.

К защите

1. Протестировать диалог, используя спроектированные правила. Убедиться, что каждое правило может быть применено;
2. Протестировать правила и фразы в базе знаний системы (навигация по sc-web);
3. Сделать отчёт (электронная версия) с иллюстрацией тестирования;
4. Залить изменения в свой форк на GitHub со всеми выполненными заданиями;
5. Подготовиться к вопросам и задачам. Повторить теоретический материал.

Ход защиты

1. Проверка отчёта;
2. Иллюстрация и тестирование правил путём получения ответа на сообщение;
3. Ответы на вопросы и решение задач от преподавателя;
4. Выставление оценки.

Вопросы

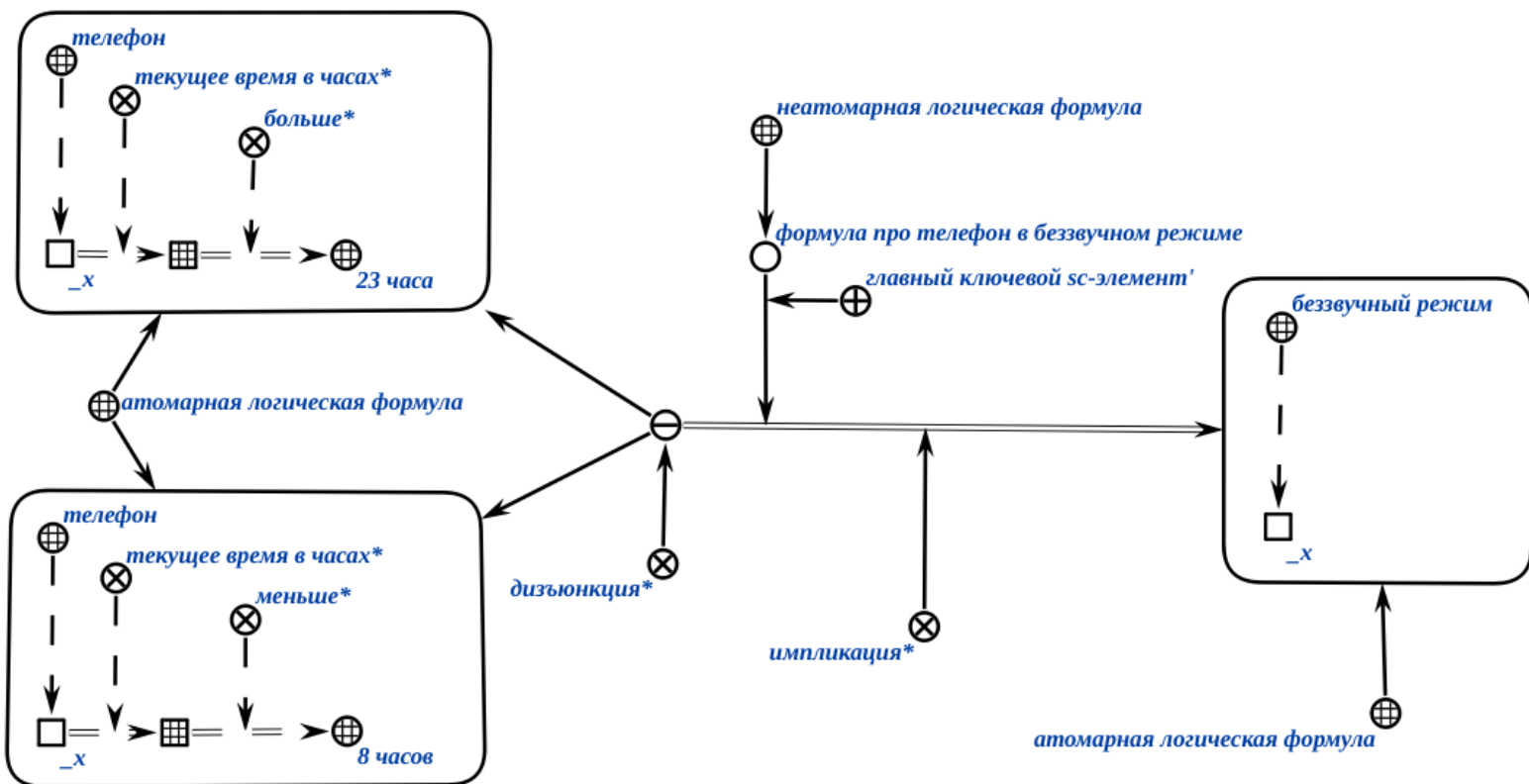
1. Что такое формальная теория? Приведите пример.
2. Что такое высказывание, какими они бывают?
3. В чем отличие фактографического высказывания от логической формулы?
4. Какие бывают логические связки?
5. В каком случае высказывание является бессмысленным?
6. Чем можно заменить конъюнкцию двух атомарных формул?
7. Что такое шаблон, для чего он нужен? Какие операции возможны с шаблонами?
8. Какие кванторы накладываются по умолчанию на атомарные логические формулы и на импликацию?

Задачи

1. Формализовать логические правила:

1. Если текущее время на телефоне больше 23 часов или меньше 8 часов, то телефон в беззвучном режиме.
2. Если температура воды не меньше 100 градусов по Цельсию, вода закипает.
3. Если сообщение пришло после того, как рабочий день закончился, ответа на сообщение не будет.
4. Если человек x является предком человека y и человек y является предком человека z , то человек x является предком человека z .
5. Если некоторое яблоко или красное, или невкусное и это яблоко или зелёное, или вкусное, то оно красное или зелёное.
6. Если студент ИИ понял главную проблему современного искусственного интеллекта, то он проучился не зря.
7. Нет дыма без огня.
8. Без труда не выловишь и рыбку из пруда.
9. Дважды в год лето не бывает.
10. Если гардероб работает и у студента идёт занятие, куртка студента должна быть сдана в гардероб.

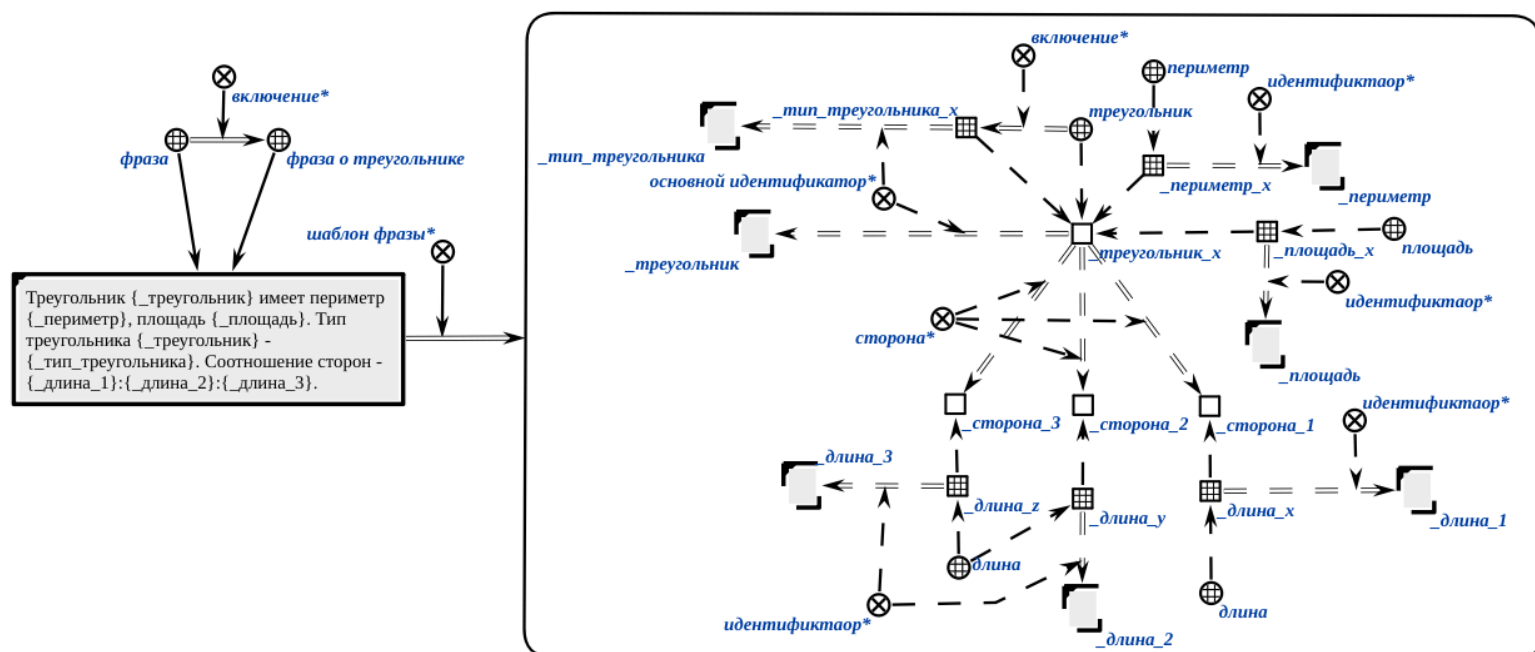
Пример решения задачи 1



2. Формализовать фразу с использованием шаблона:

1. В классе "9В" учится 25 учеников, из которых 14 отличников, 7 хорошистов и 4 двоечника. "9В" класс занял второе место в рейтинге классов школы. Класса "9В" углубленно изучает математику.
2. Студент Иванов учиться на 4 курсе, имеет средний балл 8.1, живёт на съёмной квартире, работает в компании Google.
3. В аудитории 612 22 стола, 15 компьютеров, ответственный за пожарную безопасность - Петров.
4. Университет БГУИР основан в 1964 году, главный корпус находится по адресу ул. П. Бровки, 6. Количество учащихся - 17 000, специализация университета - информационные технологии, радиотехника, электроника и телекоммуникации.
5. Треугольник ABC имеет периметр 12 см, площадь 6 см^2 . Тип треугольника ABC - прямоугольный. Соотношение сторон - 3:4:5.

Пример решения задачи 5.



Лабораторная работа №4. Разработка модуля адаптации знаний для системы NIKA¹

Цель

- Изучить принципы работы и проектирования решателей задач интеллектуальных систем, освоить навыки проектирования и реализации программ с помощью различных видов представления знаний и моделей решения задач, научиться анализировать и выводить знания.

Задание

¹ Лабораторная работа №4 не является обязательной

1) Выбрать вариант (вариант должен быть уникален в рамках потока), также можно предложить свою идею реализации агента, которой нет в подготовленных вариантах (в этом случае необходимо согласовать свой вариант с преподавателем);

2) Спроектировать и реализовать агент согласно выбранному варианту. Агент должен:

- получать знания из внешних или локальных источников и погружать в базу знаний;
- обрабатывать знания из локальных или внешних источников и выводить новые;
- вывод новых знаний должен происходить на знаниях, погруженных в БЗ.

3) Интегрировать агент с общей программой обработки системы;

4) Протестировать работу агента при помощи диалога с системой;

5) Задokumentировать реализацию агента:

- специфицировать условие инициирования и результат агента;
- описать входы и выходы агента;
- описать функционал (процедуры) агента;
- составить схему алгоритма агента.

6) По результатам работы составить отчёт, где указать:

- цель выполненной работы,
- задание лабораторной работы,
- индивидуальный вариант,
- документацию агента из пункта (5),
- скриншоты функционала агента,
- иллюстрацию работы агента,
- выводы.

7) Уметь отвечать на вопросы и решать задачи, указанные в конце лабораторной работы.

8) Залить изменения в свой форк на github.

Алгоритм опубликования результатов работы:

1. Сделать форк проекта системы NIKA <https://github.com/ostis-apps/nika>

2. Установить форк проекта системы у себя локально:

```
git clone https://github.com/{имя\_вашего\_аккаунта}/nika  
git checkout tpis-2023
```

3. Перейти в папку kb/extra и погрузить туда исходники фрагментов базы знаний;

4. Вернуться в корень проекта, сделать коммит:

```
git add problem-solver/{cxx, py}/папка_модуля_с_агентом  
git commit -m "feat(solver): implement название агента"
```

и сделать пуш изменений:

```
git push
```

5. Залить изменения в свой форк.

Критерии аттестации

- 1) Логика агента - до 2 баллов;
- 2) Документация агента - до 2 баллов;
- 3) Тестируемость - до 1,5 балла;
- 4) Качество отчёта - до 1,5 балла;
- 5) Умение отвечать на вопросы - до 2 баллов (два вопроса по 1 баллу);
- 6) Срок защиты - до 1 балла.

Максимальный балл - 10.

Срок сдачи - до конца текущего семестра.

Варианты:

Вариантом данной лабораторной работы должен являться вектор экземпляров выбранных категорий.

Класс 1. Локальность используемых источников:

- 1) Локальные источники (база знаний системы Nika, локальные структуры данных, файлы)
- 2) Внешние сервисы, доступные по сети Internet
- 3) Web-страницы (скраперы)

Класс 2. Структура внешней информации:

- 1) структурированные:
 - a) excel
 - b) sql
 - c) graph-ql
 - d) rdf/owl
 - e) erp
- 2) полуструктурированные:
 - a) xml
 - b) json
 - c) html
 - d) no-sql
 - e) csv

3) неструктурированные:

- a) txt
- b) docs
- c) pdf
- d) image
- e) audio

Класс 3. Модель обработки знаний:

- 1) процедурные (алгоритмические) модели
- 2) нейросетевые модели
- 3) логические модели
- 4) генетические алгоритмы
- 5) интеграция нескольких моделей

Класс 4. Язык программирования:

- 1) C++
- 2) Python
- 3) TypeScript

Также можно выбрать вариант реализации агента из следующего списка:

- 1. Сделать получение начальной формы для сущностей в классификации
- 2. Запоминать имя пользователя из диалога, использовать его во фразах.
Сделать агент получения конкретных знаний из диалога (узнать имя и использовать его в ходе диалога).
- 3. Агент инициирования диалога.
- 4. Агент синтеза и распознавания речи.
- 5. Агент обработки чисел в wit response.
- 6. Обучалка wit-ai
- 7. Изменение языка ведения диалога, поддержка нескольких одновременно.
- 8. Учитывать знания из предыдущих сообщений.

Теоретические сведения

В основе **решателя задач** каждой *ostis*-системы лежит многоагентная система, агенты которой взаимодействуют между собой только(!) через общую для них *sc*-память посредством спецификации в этой памяти выполняемых ими

действий в sc-памяти. При этом пользователи ostis-системы также считаются агентами этой системы. Кроме того, sc-агенты делятся на внутренние, рецепторные и эффекторные. Взаимодействие между агентами через общую sc-память сводится к следующим видам действий:

- К использованию общедоступной для соответствующей группы sc-агентов части хранимой базы знаний;
- К формированию (генерации) новых фрагментов базы знаний и/или к корректировке (редактированию) каких-либо фрагментов доступной части базы знаний;
- К интеграции (погружению) новых и/или обновленных фрагментов в состав доступной части базы знаний.

Подчеркнем, что sc-агенты не общаются между собой напрямую путем отправки сообщений, как это делается в большинстве современных подходов к построению многоагентных систем. Кроме того, sc-агенты имеют доступ к общей для них базе знаний за счет чего гарантируется семантическая совместимость (взаимопонимание) между агентами, включая и пользователей ostis-систем.

Пользователь ostis-системы не может сам непосредственно выполнить какое-либо действие в sc-памяти, но он может средствами пользовательского интерфейса инициировать построение (генерацию, формирование в sc-памяти) sc-текста, являющегося спецификацией действия в sc-памяти, выполняемого либо одним атомарным sc-агентом за один акт, либо одним атомарным sc-агентом за несколько актов, либо коллективом sc-агентов (неатомарным sc-агентом). В спецификации каждого такого действия в sc-памяти, инициированного пользователем, этот пользователь указывается как заказчик этого действия. Таким образом, пользователь ostis-системы дает поручения (задания, команды) sc-агентам этой системы на выполнение различных специфицируемых им действий в sc-памяти.

Каждый sc-агент, выполняя некоторое действие в sc-памяти, должен "помнить", что sc-память, над которой он работает, является общим ресурсом не только для него, но и для всех остальных sc-агентов, работающих над этой же sc-памятью. Поэтому sc-агент должен соблюдать определенную этику поведения в коллективе таких sc-агентов, которая должна минимизировать помехи, которые он создает другим sc-агентам.

Деятельность каждого агента ostis-системы дискретна и представляет собой множество элементарных действий (актов). При этом при выполнении каждого акта агент может устанавливать блокировки нескольких типов на фрагменты базы знаний. Указанные блокировки позволяют запретить другим агентам изменять указанный фрагмент базы знаний или вообще сделать его "невидимым" для других агентов. Блокировки устанавливаются самим агентом

при выполнении соответствующего акта и снимаются им же на последнем этапе выполнения этого акта или раньше, если это возможно.

Таким образом, **sc-модель машины обработки знаний** любой ostis-системы представляет собой неатомарный абстрактный sc-агент, являющийся результатом объединения всех абстрактных sc-агентов, входящих в состав этой ostis-системы, в один. Другими словами, под sc-моделью машины обработки знаний понимается коллектив всех sc-агентов, входящих в состав заданной ostis-системы.

Под **агентом** понимается некоторый субъект действия, частности процедура, который обладает такими качествами, как:

- автономность (независимость) агентов, что позволяет локализовать изменения, вносимые в систему при ее эволюции, и снизить соответствующие трудозатраты;
- децентрализация обработки, т.е. отсутствие единого контролирующего центра, что также позволяет локализовать вносимые в систему изменения;
- возможность параллельной работы информационного процесса, соответствующему этому агенту, то есть возможность распределенного решения задач;

В системах, ориентированных на агентную обработку решаемых задач, сами задачи формулируются в декларативном ключе.

Ход работы

Программная реализация платформы интерпретации sc-моделей написана на языках C/C++ (<https://github.com/ostis-ai/sc-machine>). На данный момент существует два способа подключения к ней:

- На уровне платформы предоставляется интерфейс, с помощью которого можно реализовать модули и агентные программы на языке C++, зависящие от реализации платформы. Под модулем подразумевается некоторый коллектив агентов, тесно связанных между собой (стоит отметить, что деление может быть и другим).
- К sc-памяти можно подключиться по сети, с помощью клиентов, подключенных к так называемому sc-серверу, работающему по websocket-протоколу. Такой вариант подключения не зависит от реализации платформы, но зависит от данных, которые передаются по сети. Данные по сети передаются в json-формате.

C++ разработка

В качестве примера рассмотрим [реализацию](#) агента нахождения сущностей, начинающихся на определенную букву на C++. Для реализации данного агента нужно выполнить следующие шаги:

1) В директории */nika/problem-solver/cxx* необходимо создать директорию под реализацию модуля и агентов этого модуля. Например, можно создать папку *messageProcessingModule*.

2) После, в созданной директории необходимо подключить зависимости, а также указать исходники для сборки. Для указания зависимостей необходимо создать конфигурационный файл CMakeLists.txt и поместить в эту же директорию. Содержание может быть следующим:

```
***
# добавление всех исходных файлов C++ в списки для цели сборки
set(SOURCES
  "MessageProcessingModule.cpp"
  "keynodes/MessageProcessingKeynodes.cpp"
  "agent/FindWordInSetByFirstLetterAgent.cpp"
)

set(HEADERS
  "MessageProcessingModule.hpp"
  "keynodes/MessageProcessingKeynodes.hpp"
  "agent/FindWordInSetByFirstLetterAgent.hpp"
)

# включение исходных файлов sc-памяти
include_directories(
  ${CMAKE_CURRENT_LIST_DIR}
  ${SC_MEMORY_SRC}
  ${SC_KPM_SRC}
  ${DIALOG_CONTROL_MODULE_SRC}
  ${SC_COMMON_MODULE_SRC}
)

# создание библиотеки `messageProcessingModule` на базе собранных исходных файлов
add_library(messageProcessingModule SHARED ${SOURCES} ${HEADERS})

# линковка зависимости `messageProcessingModule` библиотек от
# `sc-memory` и `sc-agents-common`
target_link_libraries(messageProcessingModule sc-memory sc-agents-common common
dialogControlModule)

# запуск генератора кода (необходим для создания тел
# sc-агентов и ключевых элементов)
sc_codegen_ex(messageProcessingModule ${CMAKE_CURRENT_LIST_DIR}
${CMAKE_CURRENT_LIST_DIR}/generated)
***
```

3) После подключения библиотек *sc-machine* необходимо прописать логику регистрации модуля агентов. Для этого необходимо создать класс модуля *MessageProcessingModule* со следующим содержанием:

- файл *MessageProcessingModule.hpp*

```
#pragma once

#include "sc-memory/sc_memory.hpp"
#include "sc-memory/sc_module.hpp"

#include "keynodes/MessageProcessingKeynodes.hpp"
#include "agent/FindWordInSetByFirstLetterAgent.hpp"
#include "utils/ActionUtils.hpp"

#include "MessageProcessingModule.generated.hpp"

namespace messageProcessingModule
{
class MessageProcessingModule : public ScModule
{
    SC_CLASS(LoadOrder(100))
    SC_GENERATED_BODY()

    // Метод инициализации модуля и его агентов
    virtual sc_result InitializeImpl() override;

    // Метод деинициализации модуля и его агентов
    virtual sc_result ShutdownImpl() override;
};

} // namespace messageProcessingModule
```

- файл *MessageProcessingModule.cpp*

```
#include "MessageProcessingModule.hpp"

namespace messageProcessingModule
{
// Регистрация модуля
SC_IMPLEMENT_MODULE(MessageProcessingModule)

sc_result MessageProcessingModule::InitializeImpl()
{
    if (!MessageProcessingKeynodes::InitGlobal())
        return SC_RESULT_ERROR;

    // Регистрация агентов модуля
    ScMemoryContext ctx(sc_access_lvl_make_min, "MessageProcessingModule");
    if (ActionUtils::isActionDeactivated(&ctx,
MessageProcessingKeynodes::action_find_word_in_set_by_first_letter))
    {
        SC_LOG_ERROR("action_letter_search is deactivated");
    }
    else
    {
        SC_AGENT_REGISTER(FindWordInSetByFirstLetterAgent);
    }

    return SC_RESULT_OK;
}
```



```

}

sc_result MessageProcessingModule::ShutdownImpl()
{
    // Дeregистрация агентов модуля
    SC_AGENT_UNREGISTER(FindWordInSetByFirstLetterAgent);

    return SC_RESULT_OK;
}

} // namespace messageProcessingModule

```

4) Реализация необходимого функционала производится на уровне агента. Агент представляет собой асинхронно-вызываемую программу, решающую определённый класс задач и имеющую описанную в формальном виде спецификацию. В спецификации агента указываются: класс задач, решаемый агентом, первичное условие инициирования, условие инициирования этого агента и его целевой результат, ключевые элементы (отношения и классы), используемые агентом, и другие.

Перед началом написания реализации агента необходимо сформулировать решаемый им класс задач. Агент *FindWordInSetByFirstLetterAgent* умеет находить сущности в множестве, начинающиеся на определённую букву. Таким образом, действие агента может быть задано как *action_find_word_in_set_by_first_letter*, что означает класс таких действий, которое может быть выполнено некоторым агентом поиска сущностей на определённую букву при наличии соответствующего условия в памяти системы. При этом сам экземпляр действия агента должен принадлежать классу всех возможных действий *question*.

Поскольку агентная программа является асинхронной, то важно задать условие инициирования и результат агента. Агент считается инициированным тогда и только тогда, когда была создана, например, выходящая константная позитивная постоянная дуга принадлежности от класса *question_initiated* к sc-узлу действия этого агента.

5) Логика агента может описываться в любом стиле программирования и представляет собой класс, описываемый в .hpp и .cpp файлах.

- FindWordInSetByFirstLetterAgent.hpp

```

#pragma once

#include "sc-memory/kpm/sc_agent.hpp"
#include "sc-agents-common/keynodes/coreKeynodes.hpp"

#include "FindWordInSetByFirstLetterAgent.generated.hpp"
#include "searcher/MessageSearcher.hpp"

namespace messageProcessingModule

```

```

{
class FindWordInSetByFirstLetterAgent : public ScAgent
{
    // Тип события условия инициирования sc-агента -- появление в sc-памяти
    // выходящей дуги из `question_initiated` к действию
    SC_CLASS(Agent, Event(scAgentsCommon::CoreKeynodes::question_initiated,
ScEvent::Type::AddOutputEdge))

    SC_GENERATED_BODY();

private:
    bool checkActionClass(ScAddr const & actionAddr);

    ScAddrVector createAnswer(std::string const & linkContent) const;

    std::string getMessageText(ScAddr const & messageAddr) const;

    std::unique_ptr<class dialogControlModule::MessageSearcher> messageSearcher;
};
} // namespace messageProcessingModule

```

- FindWordInSetByFirstLetterAgent.cpp

```

// Подключение полезных утилит из sc-machine
#include "sc-agents-common/utils/AgentUtils.hpp"
#include "sc-agents-common/utils/CommonUtils.hpp"
#include "sc-agents-common/utils/IteratorUtils.hpp"
#include "sc-agents-common/keynodes/coreKeynodes.hpp"
#include "MessageKeynodes.generated.hpp"

#include "keynodes/MessageProcessingKeynodes.hpp"
#include "keynodes/MessageKeynodes.hpp"

#include "FindWordInSetByFirstLetterAgent.hpp"

using namespace scAgentsCommon;
using namespace utils;

namespace messageProcessingModule
{
SC_AGENT_IMPLEMENTATION(FindWordInSetByFirstLetterAgent)
{
    // Проверка условия инициирования sc-агента
    ScAddr const & actionAddr = otherAddr;
    if (!checkActionClass(actionAddr))
    {
return SC_RESULT_OK;
    }

    // Вывод сообщения о начале работы агента
    SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent started");

    ScAddr const & messageAddr =
        utils::IteratorUtils::getAnyByOutRelation(&m_memoryCtx, actionAddr,
scAgentsCommon::CoreKeynodes::rrel_1);
    if (!messageAddr.IsValid())

```

```

    {
        SC_LOG_ERROR("FindWordInSetByFirstLetterAgent: the message isn't valid");
        utils::AgentUtils::finishAgentWork(&m_memoryCtx, actionAddr, false);
        return SC_RESULT_ERROR;
    }

    if (!m_memoryCtx.HelperCheckEdge(
MessageProcessingKeynodes::concept_message_about_find_word_by_first_letter,
        messageAddr,
        ScType::EdgeAccessConstPosPerm))
    {
        SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent: the message isn't about letter
search");
        utils::AgentUtils::finishAgentWork(&m_memoryCtx, actionAddr, false);
        return SC_RESULT_ERROR;
    }

    ScAddrVector answerElements;
    try
    {
        ScIterator3Ptr const & agentAnswerLinkIterator = m_memoryCtx.Iterator3(
            MessageProcessingKeynodes::word_starts_with_required_letter_answer_phrase,
            ScType::EdgeAccessConstPosPerm,
            ScType::LinkConst);
        if (agentAnswerLinkIterator->Next())
        {
            m_memoryCtx.EraseElement(agentAnswerLinkIterator->Get(2));
        }

        messageSearcher =
std::make_unique<dialogControlModule::MessageSearcher>(&m_memoryCtx);
        std::string messageText = getMessageText(messageAddr);
        ScAddr const & entityAddr = utils::IteratorUtils::getAnyByOutRelation(
            &m_memoryCtx, messageAddr,
dialogControlModule::MessageKeynodes::rrel_entity);
        // Создание итератора для поиска элементов к котором есть связь с entity
        ScIterator3Ptr const & entityNodesIterator =
            m_memoryCtx.Iterator3(entityAddr, ScType::EdgeAccessConstPosPerm,
ScType::NodeConst);

        std::string firstLetter;
        std::string firstWordLetter;

        firstLetter = messageText.substr(messageText.size() - 3, 2);
        std::stringstream resultStream;
        std::string result;
        std::string word;

        if (entityNodesIterator->Next())
        {
            word = utils::CommonUtils::getMainIdtf(
                &m_memoryCtx, entityNodesIterator->Get(2),
{scAgentsCommon::CoreKeynodes::lang_ru});
            firstWordLetter = word.substr(0, 2);
            if (firstLetter == firstWordLetter)
            {
                resultStream << word;

```

```

        SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent: found word " << word);
    }
}
while (entityNodesIterator->Next())
{
    word = utils::CommonUtils::getMainIdtf(
        &m_memoryCtx, entityNodesIterator->Get(2),
{scAgentsCommon::CoreKeynodes::lang_ru});
    firstWordLetter = word.substr(0, 2);
    if (firstLetter == firstWordLetter)
    {
        if (resultStream.str().size() == 0)
            resultStream << word;
        else
            resultStream << ", " << word;
        SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent: found word " << word);
    }
}

if (resultStream.str().empty())
{
    result = "Таких слов нет";
}
else
{
    result = resultStream.str();
}
answerElements = createAnswer(result);
SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent: reply message is generated");
}
catch (utils::ScException const & exception)
{
    SC_LOG_ERROR(exception.Description());
    utils::AgentUtils::finishAgentWork(&m_memoryCtx, actionAddr, answerElements,
false);
    SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent: finished with an error");
    return SC_RESULT_ERROR;
}

    utils::AgentUtils::finishAgentWork(&m_memoryCtx, actionAddr, answerElements,
true);

    SC_LOG_DEBUG("FindWordInSetByFirstLetterAgent finished");
    return SC_RESULT_OK;
}

bool FindWordInSetByFirstLetterAgent::checkActionClass(ScAddr const & actionAddr)
{
    return m_memoryCtx.HelperCheckEdge(
        MessageProcessingKeynodes::action_find_word_in_set_by_first_letter,
actionAddr, ScType::EdgeAccessConstPosPerm);
}

ScAddrVector FindWordInSetByFirstLetterAgent::createAnswer(std::string const &
linkContent) const
{
    ScAddr const & answerLink = m_memoryCtx.CreateLink(ScType::LinkConst);
    m_memoryCtx.SetLinkContent(answerLink, linkContent);
}

```

```

        ScAddr const & edgeAccessConstPosPerm = m_memoryCtx.CreateEdge(
            ScType::EdgeAccessConstPosPerm,
            MessageProcessingKeynodes::word_starts_with_required_letter_answer_phrase,
            answerLink);
        return {
            answerLink, edgeAccessConstPosPerm,
            MessageProcessingKeynodes::word_starts_with_required_letter_answer_phrase};
    }

    std::string FindWordInSetByFirstLetterAgent::getMessageText(ScAddr const &
messageAddr) const
    {
        ScAddr const & messageLink = messageSearcher->getMessageLink(messageAddr);
        if (!messageLink.IsValid())
        {
            SC_THROW_EXCEPTION(utils::ExceptionItemNotFound,
"FindWordInSetByFirstLetterAgent: message link is not found.");
        }
        return utils::CommonUtils::getLinkContent(&m_memoryCtx, messageLink);
    }

} // namespace messageProcessingModule

```

6) Также необходимо создать класс, содержащий необходимые сущности для работы агента:

- **MessageProcessingKeynodes.hpp**

```

#pragma once

#include "sc-memory/sc_addr.hpp"
#include "sc-memory/sc_object.hpp"
#include "MessageProcessingKeynodes.generated.hpp"

namespace messageProcessingModule
{
    class MessageProcessingKeynodes : public ScObject
    {
    {
        SC_CLASS()
        SC_GENERATED_BODY()

    public:
        SC_PROPERTY(Keynode("action_find_word_in_set_by_first_letter"), ForceCreate)
        static ScAddr action_find_word_in_set_by_first_letter;

        SC_PROPERTY(Keynode("concept_message_about_find_word_by_first_letter"),
ForceCreate)
        static ScAddr concept_message_about_find_word_by_first_letter;

        SC_PROPERTY(Keynode("word_starts_with_required_letter_answer_phrase"),
ForceCreate)
        static ScAddr word_starts_with_required_letter_answer_phrase;
    };

} // namespace messageProcessingModule

```

- MessageProcessingKeynodes.cpp

```
#include "MessageProcessingKeynodes.hpp"

namespace messageProcessingModule
{
    ScAddr MessageProcessingKeynodes::action_find_word_in_set_by_first_letter;
    ScAddr
MessageProcessingKeynodes::concept_message_about_find_word_by_first_letter;
    ScAddr
MessageProcessingKeynodes::word_starts_with_required_letter_answer_phrase;
} // namespace messageProcessingModule
```

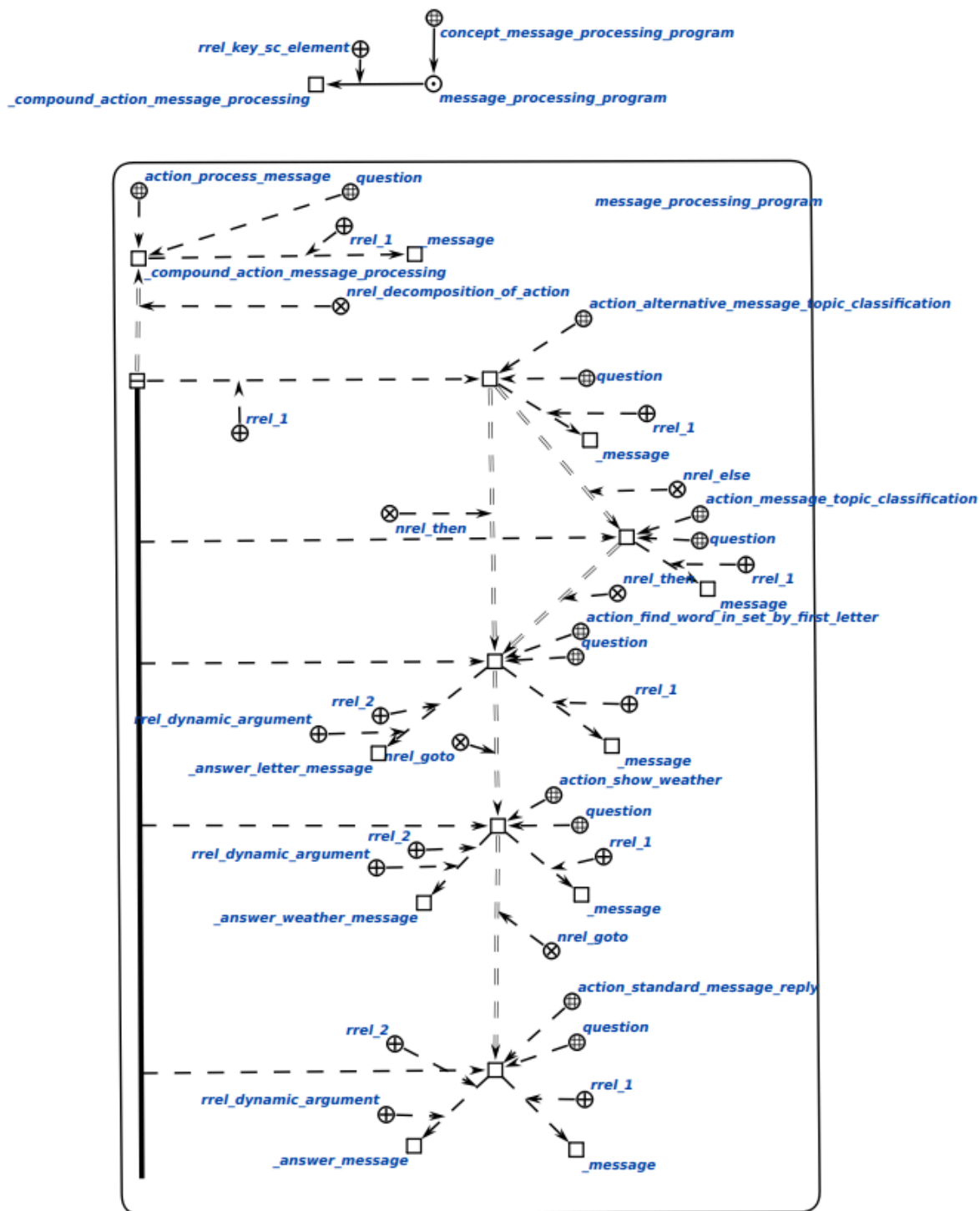
Стоит отметить, что программный код приводится в качестве примера, в реальных логика и спецификация агента могут быть иными.

Как видно из листинга кода, агент в случае ошибки возвращает код ошибки, а в случае успешной отработки - код успеха и ответную структуру, например структуру, которой принадлежит ссылка, включающая в себя найденные слова.

После реализации программного кода агента нужно совершить пересборку решателя задач. Для этого выполнить в `nika/scripts` скрипт `./build_problem_solver.sh` (либо `docker compose build` при использовании Docker).

При выполненной данной работы требуется использовать не только стандартные библиотеки `ostis`-платформы, но и сторонние библиотеки (например, библиотеку для обращения к API некоторого облачного сервиса).

Для того, чтобы подключить разработанный агент в общую последовательность плана выполнения агентов системы Nika нужно изменить [указанную](#) программу обработки:



Документацию агентов программы обработки системы Nika можно найти [по ссылке](#).

Python и TypeScript разработка

Взаимодействовать с памятью ostis-системы можно не только при помощи программного API. Для решения более широкого круга задач была разработана подсистема взаимодействия с sc-памятью на основе WebSocket и JSON. Обработка запросов к памяти реализуется на стороне sc-сервера, являющегося частью sc-machine. Для обращения к sc-памяти через sc-сервер можно использовать клиенты, реализованные на разных языках. Больше информации о sc-сервере можно найти [по ссылке](#).

Существует 2 открытых клиента, подключаемых к sc-серверу и удобных в использовании:

- Клиент на Python - [py-sc-client](#);
- Машина обработки знаний на Python - [py-sc-kpm](#);
- Клиент на TypeScript - [ts-sc-client](#).

Для разработки агента на языке Python в систему добавлен модуль и агент, который можно использовать в качестве примера. Пример доступен по [ссылке](#).

Для TypeScript разработки на данный момент рекомендуется самостоятельно реализовать подсистему с агентом, используя клиент для работы с памятью.

К защите

1. Протестировать работоспособность агента через диалог с системой;
2. Сделать отчёт (электронная версия) с иллюстрацией тестирования и документации агента;
3. Залить изменения в форк на GitHub со всеми выполненными заданиями;
4. Подготовиться к вопросам. Повторить теоретический материал.

Ход защиты

1. Проверка отчёта;
2. Иллюстрация и тестирование цепочки агентов;
3. Ответы на вопросы от преподавателя;
4. Выставление оценки.

Вопросы

1. Что такое многоагентная система? Приведите пример.
2. Какие бывают многоагентные системы?

3. Зачем нужно создавать многоагентные системы? Возможно ли разрабатывать интеллектуальные системы, не являющиеся многоагентными?
4. Что такое интеллектуальная система?
5. Что такое агент? Какие бывают агенты?
6. Что входит в спецификацию любого агента?
7. Какие задачи позволяет решать текущая реализация платформы OSTIS?

Дополнительно

В данном пункте будут представлены все вопросы, на которые умеет отвечать Ника, а также кратко показано, что нужно сделать для того, чтобы Ника смогла на них ответить.

Во-первых, чтобы узнать, на какие вопросы Ника умеет отвечать, можно запустить проект. Вы увидите диалоговое окно. Введите пустое сообщение (нажмите Enter). В качестве сообщения-подсказки будет выбрано сообщение: “Что Ника умеет?”. В ответе Ники будут примеры вопросов, на которые она умеет отвечать.

Что умеет Ника?
19:09

Ника умеет:

- отвечать на приветствие:
 - "Привет",
 - "Хай",
- изменять цвет компонентов интерфейса, где вместо #00bfff можно подставить hex код для любого цвета:
 - "Измени цвет хедера на #00bfff",
 - "Поменяй цвет компонента футер на #00bfff",
 - "Измени цвет компонента основная часть на #00bfff";
- отвечать на вопросы о сущности:
 - "Что такое ...?",
 - "Кто такой ...?",
- отвечать на вопросы о навыках и умениях:
 - "Что умеет ...?",
 - "Что делает ...?",
- отвечать на вопросы о разбиении:
 - "На что декомпозируется ...?",
 - "Каким бывает ...?",
- находить слова, которые начинаются с заданной пользователем буквы:
 - "Что в ... начинается на букву ...?",
- а также Ника может научиться отвечать на любые новые вопросы.

19:09

Что умеет Ника?



Для того, чтобы Ника смогла ответить на Ваш вопрос, ей вначале нужно выделить *ключевые элементы сообщения* – сущности. Далее в своей базе знаний Ника может найти семантическую окрестность данных сущностей и выделить *класс сообщения* (класс сообщения должен быть формализован в системе). Потом Ника по правилу *логического вывода* (Modus Ponens) пытается ответить на задаваемый ей вопрос (фраза ответа на сообщение должна быть формализована в системе). В этом случае фраза может быть получена по атомарной логической

формуле (шаблону), где для каждой sc-переменной будет подставлен в соответствие sc-элемент из семантической окрестности выделенной сущности. В конце данная фраза будет выведена в диалоговом окне как ответное сообщение.

Если Ника не сможет выделить ключевой элемент сообщения (найти его семантическую окрестность), то в диалоговом окне появится ответная фраза о том, что Ника не может ответить на вопрос.



При создании ответа на вопрос можно выделить следующие шаги:

1. Формализация необходимого класса сообщений.
2. Обучение wit.ai на сообщения необходимого класса.
3. Формализация правила классификации сообщений необходимого класса.
4. Формализация правила ответа сообщений необходимого класса.
5. Формализация фразы ответа на сообщения необходимого класса.

Под сообщением необходимого класса понимается вопрос (напр. Что Ника умеет?), использование слова *класс* указывает на то, что Ника в итоге научится отвечать не только на конкретный вопрос (Что Ника умеет?), но и на схожие по формулировке вопросы (Что делает Ника?, Что Ника может?, “Что может робот?” и т.д.).

Далее разберем формализацию следующих вопросов: “Что такое ...?”, “Что умеет ...?”, “На что декомпозируется ...?” в соответствии с выделенными шагами.

“Что такое ...”?

1. Наш класс сообщений лучше формализовать в отдельном файле с расширением .scs.

Название файла должно совпадать с названием класса сообщений, а сам файл расположен по следующему пути:
“/nika/kb/section_subject_domain_of_messages/{nanka с вашей ПрО}/concept_message_about_{название типа сообщения}.scs”

Ключевым элементом вопроса является какая-то сущность (не обязательно о Нике), поэтому тип вопроса – о сущности.
“concept_message_about_entity.scs”.

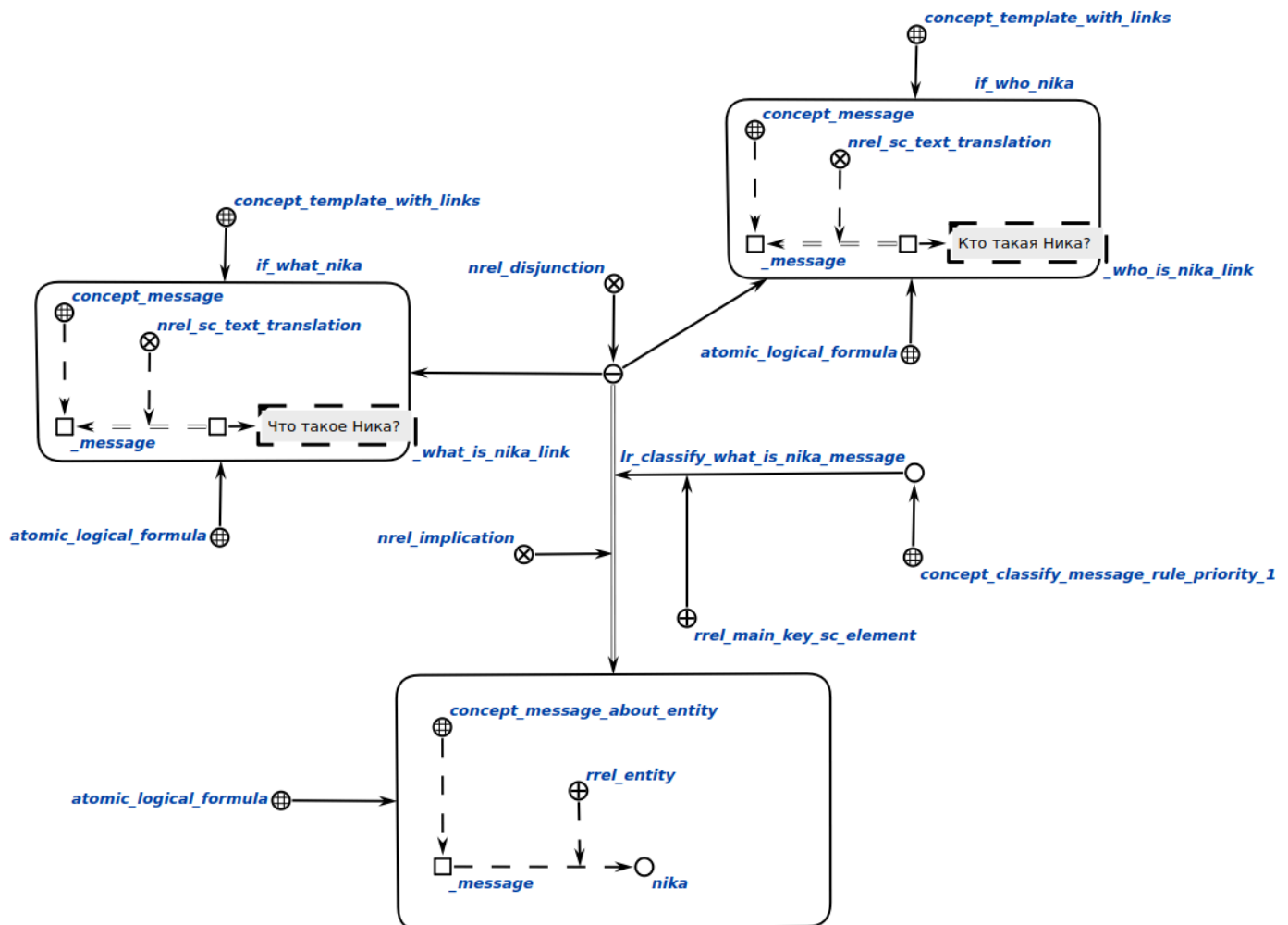
Содержимое файла с формализацией класса можно увидеть в проекте.

```
concept_message_about_entity
<- sc_node_class;
=> nrel_main_idtf:
  [сообщение о сущности]
    (* <- lang_ru;; *);
  [message about entity]
    (* <- lang_en;; *);
=> nrel_wit_ai_idtf:
  [about_entity];
<- rrel_key_sc_element:
  ...
  (*
=> nrel_main_idtf:
  [Опр. (сообщение о сущности)]
    (* <- lang_ru;; *);
  [Def. (message about entity)]
    (* <- lang_en;; *);;
<- definition;;
<= nrel_sc_text_translation:
  ...
  (*
  -> rrel_example:
    [Сообщение о сущности – атомарное сообщение, которое содержит
    сущность, информацию о которой необходимо найти.]
      (* <- lang_ru;; *);;
  *);;
  ...
  (*
  -> rrel_example:
    [Message about entity is an atomic message that
    contains entity information about which you need to find.]
      (* <- lang_en;; *);;
  *);;
<= nrel_using_constants:
  {
    concept_atomic_message
  };;
*);;
```

Обучение на wit.ai необходимо для классификации сообщений и выделения сущностей сообщения. Подробно данный этап рассмотрен в лабораторной работе №2. Фразы для обучения wit.ai могут быть следующими: “Что такое Ника?”, “Что Ника такое?”, “Кто такая Ника?”, “Что такое цветок?”, “Кто такой космонавт?” и т. д.

2. Формулы (правила) классификации лучше всего формализовать на SCg-коде в файлах с расширением .gwf.

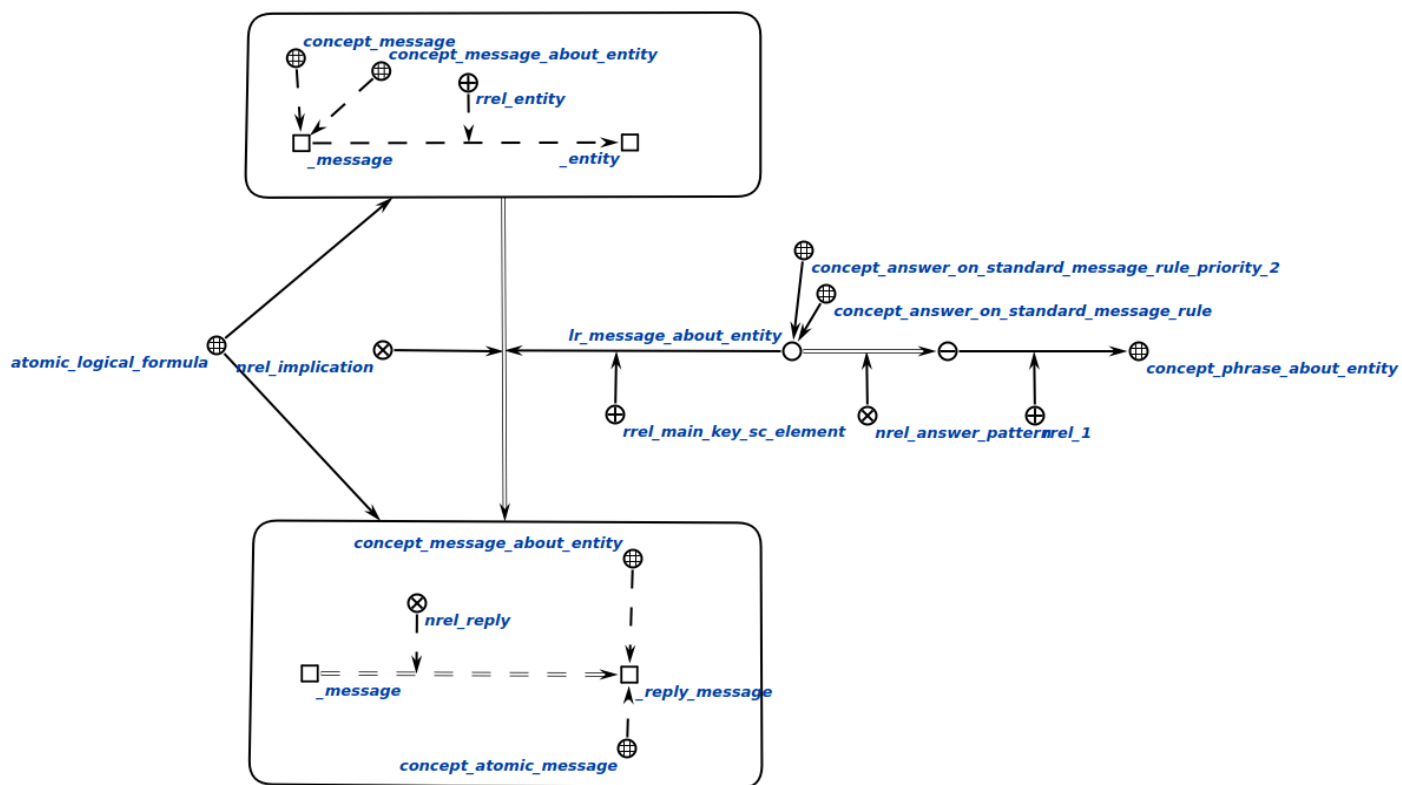
Название файла: “*lr_message_about_{тип сообщения}.gwf*”. Приставка “lr” означает “logic rule”, то есть “логическое правило” (логическая формула). Пример формализации можно увидеть по следующему пути: “*nika/kb/section_subject_domain_of_dialogues/section_subject_domain_of_dialog_control/section_subject_domain_of_dialogue_control_using_template_responses/logic_rules/lr_alternative_classification/lr_message_about_entity.gwf*”



3. Формулы ответа на сообщения заданного класса, как и формулы классификации, лучше формализовывать на SCg-коде. Пример расположен по пути:

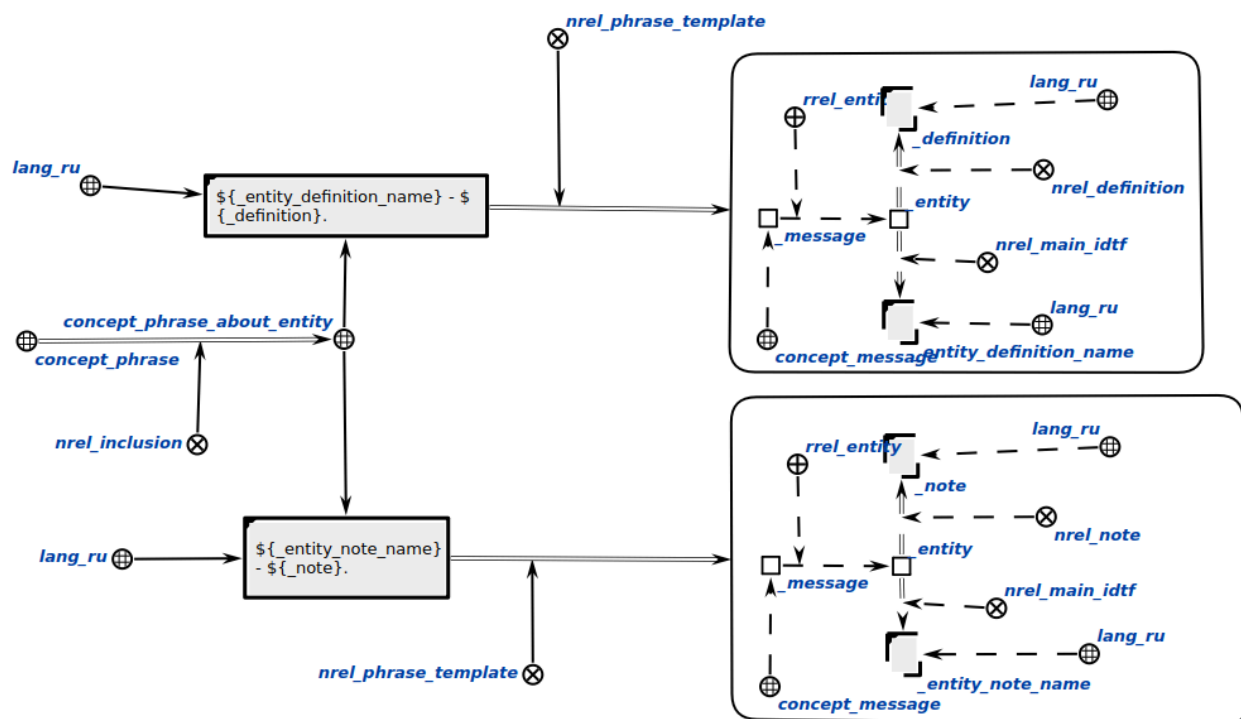
“*nika/kb/section_subject_domain_of_dialogues/section_subject_domain_of_dialog_control/section_subject_domain_of_dialogue_control_using_template_responses/logic_rules/lr_message_about_entity/lr_message_about_entity.gwf*”

Логическая формула представляет собой связку импликации между двумя логическими формулами и данное правило необходимо для формирования ответа на сообщение, которое было успешно классифицировано.



4. Формулы генерации фразы ответа тоже лучше формализовывать на SCg-коде. Пример расположен по пути:

“nika/kb/section_subject_domain_of_messages/subject_domain_of_phrases/examples/concept_phrase_about_entity/concept_phrase_about_entity.gwf”



“Что умеет ...?”

1. Класс сообщений — `concept_message_about_skill.scs`

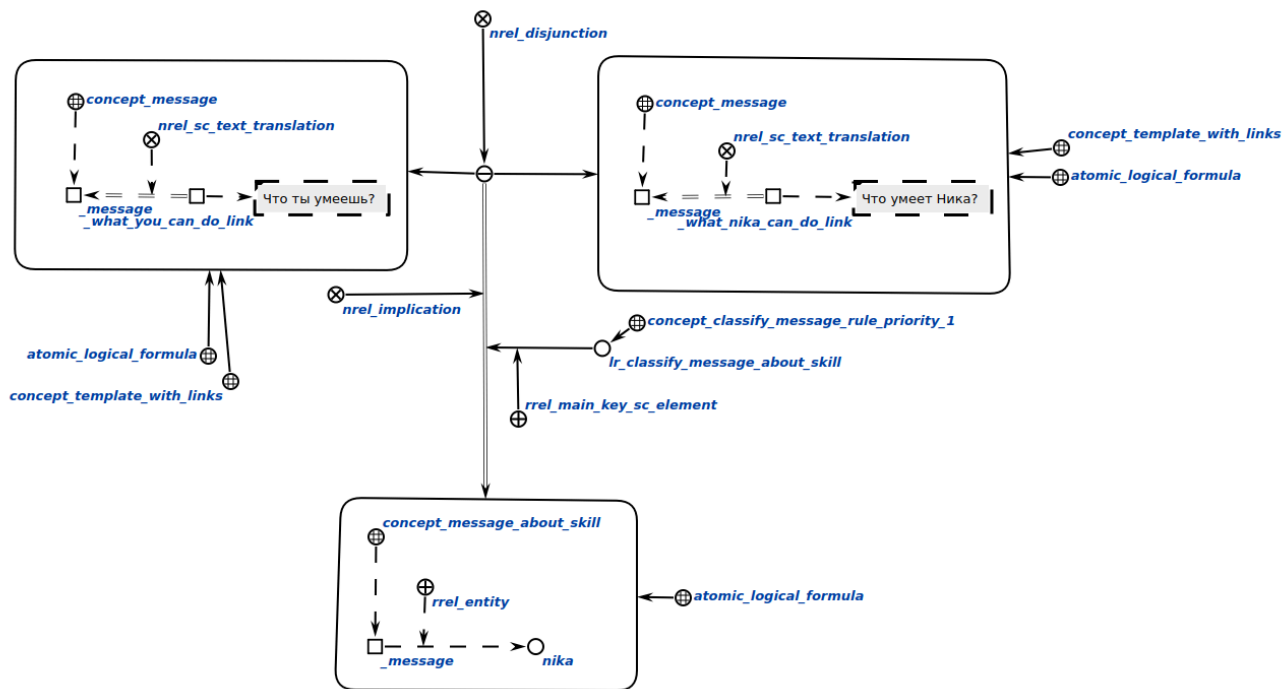
```

concept_message_about_skill
<- sc_node_class;
=> nrel_main_idtf:
  [сообщение о навыке]
    (* <- lang_ru;; *);
  [message about skill]
    (* <- lang_en;; *);
=> nrel_wit_ai_idtf:
  [about_skill];
<- rrel_key_sc_element:
  ...
  (*
=> nrel_main_idtf:
  [Опр. (сообщение о навыке)]
    (* <- lang_ru;; *);
  [Def. (message about skill)]
    (* <- lang_en;; *));
<- definition;;
<= nrel_sc_text_translation:
  ...
  (*
-> rrel_example:
    [Сообщение о навыке - атомарное сообщение, которое
содержит навык, информацию о котором необходимо найти.]
    (* <- lang_ru;; *));
  *);
  ...
  (*
-> rrel_example:
    [Message about skill is an atomic message that
contains skill information about which you need to find.]
    (* <- lang_en;; *));
  *);
<= nrel_using_constants:
  {
    concept_atomic_message
  };
  *);

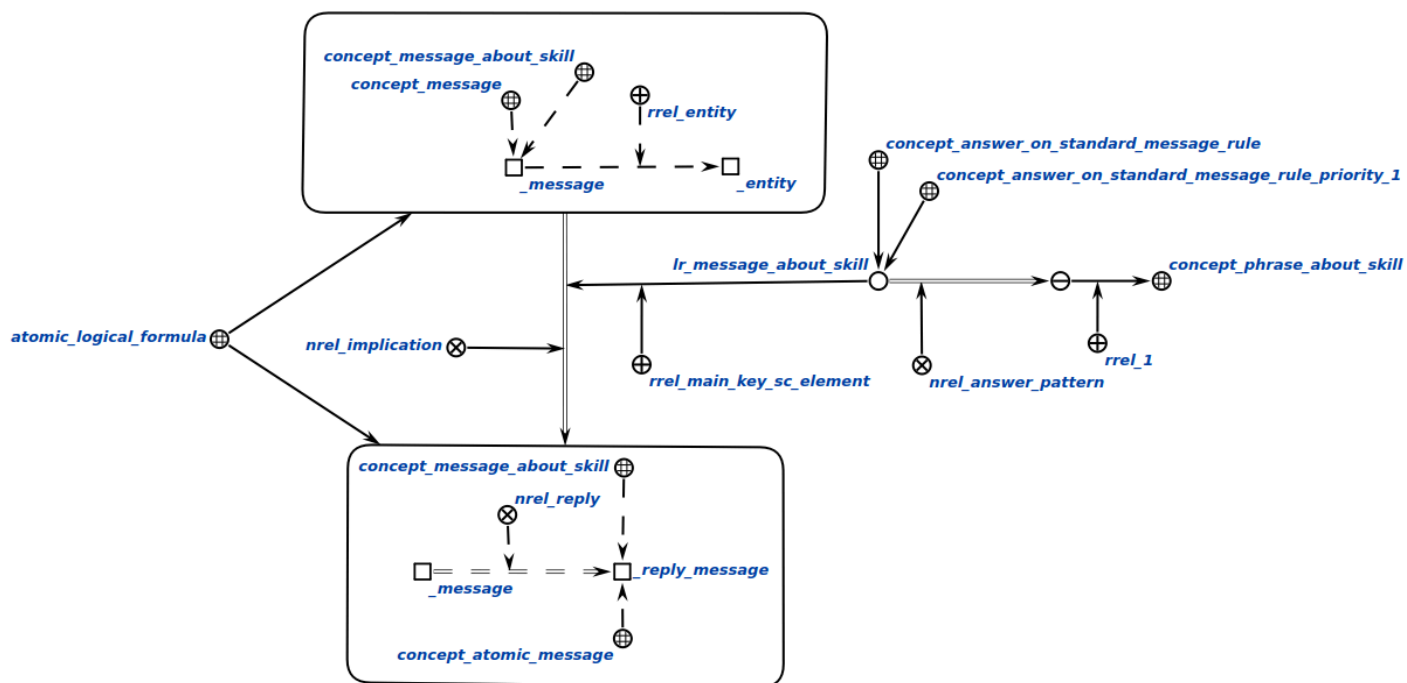
```

2. Фразы для обучения: “Что может Ника?”, “Что Ника умеет?”, “Что Ника делает?”, “Что может робот?” и т.д.

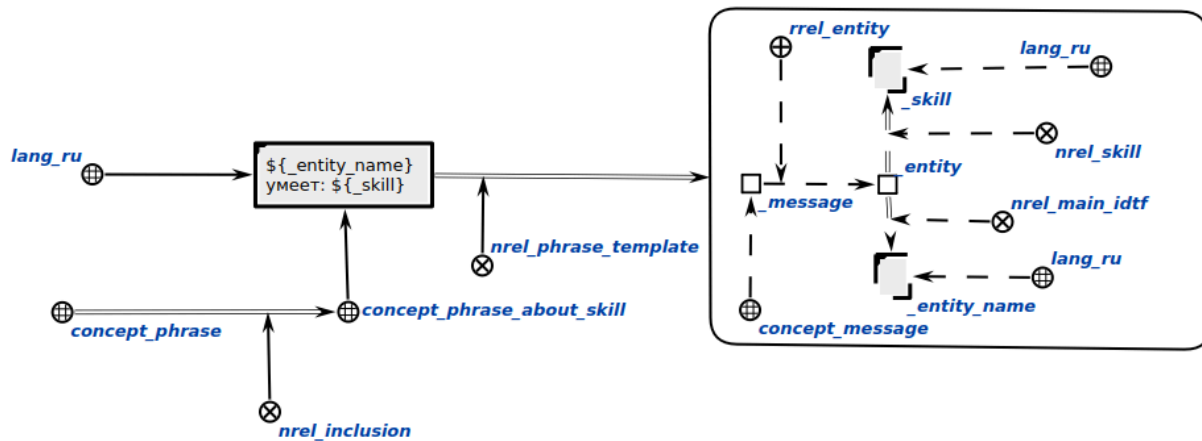
3. Формула классификации — lr_message_about_skills.gwf



4. Формула ответа — lr_message_about_skills.gwf



5. Формулы генерации фразы ответа — concept_phrase_about_skills.gwf



“На что декомпозируется ...?”

1. Класс сообщений — concept_message_about_subdividing.scs

```

concept_message_about_subdividing
<- sc_node_class;
=> nrel_main_idtf:
  [сообщение о разбиении]
  (* <- lang_ru;; *);
  [message about subdividing]
  (* <- lang_en;; *);
=> nrel_wit_ai_idtf:
  [subdividing];
<- rrel_key_sc_element:
  ...
  (*
    => nrel_main_idtf:
      [Опр. (сообщение о разбиении)]
      (* <- lang_ru;; *);
      [Def. (message about subdividing)]
      (* <- lang_en;; *);
      <- definition;;
      <= nrel_sc_text_translation:
        ...
        (*
          -> rrel_example:
            [сообщение о разбиении - атомарное сообщение,
            в котором говорится о разбиении какой-то сущности.]
            (* <- lang_ru;; *);
          *);
        ...
        (*
          -> rrel_example:

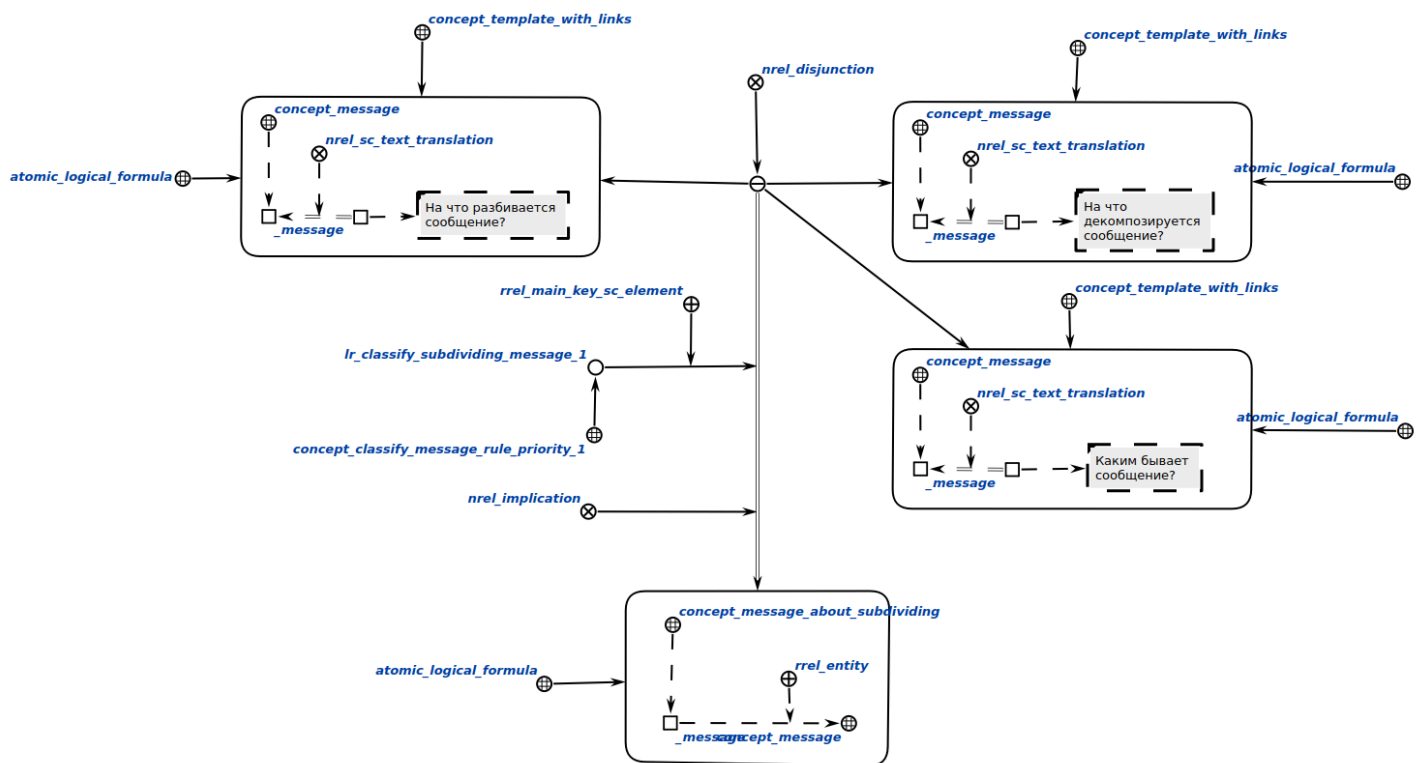
```


[message about subdividing is an atomic message, that is about of an entity subdivision.]

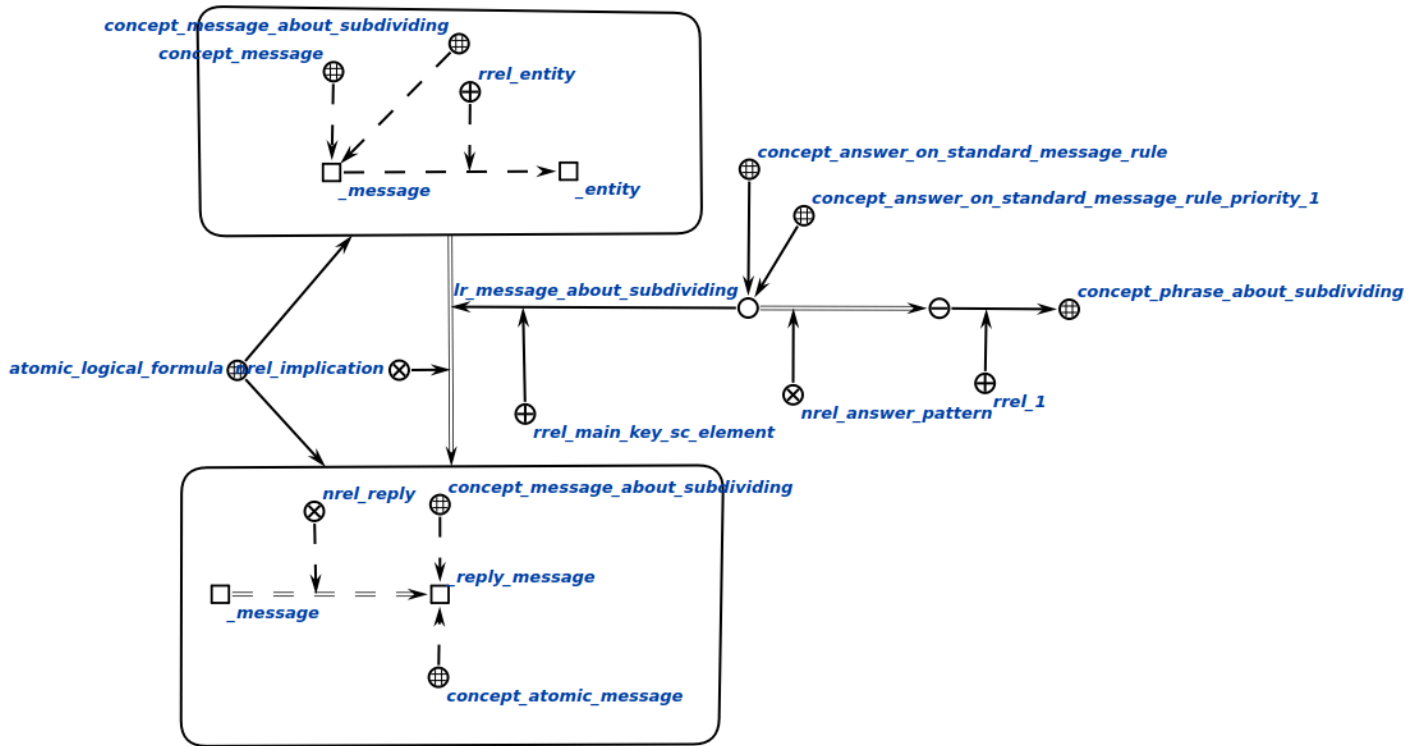
```
(* <- lang_en;; *);;
<= nrel_using_constants:
{
    concept_atomic_message;
    nrel_subdividing
};;
*);;
```

2. Фразы для обучения: “Каким бывает сообщение?”, “На что разбивается сообщение?”, “Что может быть разбиением сообщения?”, “Каким бывает действие?” и т.д.

3. Формула классификации — lr_message_about_subdividing.gwf



4. Формула ответа — lr_message_about_subdividing.gwf



5. Формулы генерации фразы ответа –
concept_phrase_about_subdividing.gwf

